

Building a Feed-forward Characteristic Function Matching Generator

A project submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science Honours

in

Mathematical Statistics

Department of Statistics

Rhodes University

by

Nicholas Loyd Szczawinski

November 6, 2025

Supervisor: Jeremy Baxter

Declaration

By submitting this thesis electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Rhodes University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

I hereby declare that ethics approval was not required for my research as the data used in this study, as introduced in chapter 4, is simulated.

Date: 5th November 2025

Abstract

This paper discusses the construction of a neural network based sampler, called a generator, which is able to produce sample estimates of a target distribution by incorporating its characteristic function in the loss equation. By back propagating the squared error of the target distribution's characteristic function and an empirical estimate of the generator's last layer output, a mapping is learned which transforms standard normal noise into sample estimates for a particular univariate target distribution. Testing was done by matching the theoretical mean and variance of the target distribution and the actual mean and variance of the simulated samples. Distributional shape was observed by overlapping a histogram of 10 000 simulated samples with the target distribution's density curve. Results showed that depending on the chosen set of parameters, the generator could reproduce most of the common univariate distributions including bimodal mixtures, except for the Cauchy distribution, and struggled to handle parameter combinations that produced distributions with large means or heavy tails.

Contents

Declaration	i
Abstract	ii
List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
1 Introduction	1
1.1 Neural Networks and Modern Sampling Techniques	1
1.2 Problem Statement	1
1.3 Research Question	2
1.4 Hypothesis	2
1.5 Objective	2
1.6 Approach	2
1.7 Method	2
1.8 Structure of the Paper	3
2 Sampling, Simulation, and Characteristic Functions	4
2.1 Importance of Sampling and Simulation	4
2.2 Sampling Algorithms	5
2.2.1 Traditional Sampling Algorithms	5
2.2.2 Neural Network Samplers	6
2.3 The Characteristic Function	6
2.3.1 Examples of Univariate Characteristic Functions	8
2.3.2 Characteristic Functions of Mixture Distributions	8
2.3.3 The Empirical Characteristic Function	9

2.3.4	The Multivariate Empirical Characteristic Function	11
2.4	Sampling from Complex Distributions	13
2.5	Theoretical Benefits of a Characteristic Function-based Method	14
2.6	Caveats of a Characteristic Function-Based Neural Network Sampler	15
3	Constructing The Feed-forward Characteristic Function Matching Generator	17
3.1	Algorithm	17
3.2	Arguments (Objects) of the Generator	19
3.2.1	Input Matrix	19
3.2.2	Target Matrix	20
3.2.3	Weight Matrices	20
3.2.4	Bias Vectors	21
3.2.5	Linear Pre-activation Function (Accumulator)	22
3.2.6	Post-activation Function (Layer Output)	22
3.2.7	Empirical Characteristic Function Computation	24
3.3	Loss Function	25
3.4	The Optimising Algorithm	26
3.5	Back Propagation	27
3.5.1	General Algorithm	27
3.5.2	Deriving the Generator Derivatives	29
3.6	Post Convergence Form	35
3.7	Choice of Activation Function	36
3.7.1	Advantages of using ReLU functions:	36
3.7.2	Disadvantages of using ReLU functions:	37
4	Data Analysis and Selected Results	38
4.1	Hyperparameters	38
4.2	Univariate Distribution Matching	39
4.2.1	Description of the Data	39
4.2.2	Reporting	40
4.2.3	Normal Distributions	41
4.2.4	Gamma Distributions	44
4.2.5	Chi-Square Distributions	47
4.2.6	Exponential Distributions	50
4.2.7	T Distributions	53

4.2.8	Cauchy Distributions	56
4.2.9	Normal Inverse Gaussian Distributions	59
4.2.10	Equally Weighted Normal Inverse Gaussian Distribution Mixtures	62
4.2.11	Differently Seeded Simulations	65
5	Discussion and Conclusion	68
5.1	The Research Objective	68
5.2	Evaluation of Results	68
5.3	Implications and Applications of Results	69
5.4	Strengths and Weaknesses of Analysis	70
5.4.1	Strengths	70
5.4.2	Weaknesses	70
5.5	Conclusion	71
A	The R Code Structure	73
A.1	The ‘define_architecture’ Function	73
A.2	The ‘nn_layer’ Function	74
A.3	The ‘execute’ Function	75
A.4	The ‘push’ and ‘generate_samples’ Functions	78
A.5	The Auxiliary Functions	79
A.6	The Function Wrapper	87
	References	91
	Research Ethics Declaration	98

List of Figures

3.1	Training Cycle	18
3.2	Sample Generation	19
3.3	Example FCFMG Graph	25
4.1	0 Mean Normal Distribution	42
4.2	5 Mean Normal Distribution	42
4.3	10 Mean Normal Distribution	43
4.4	15 Mean Normal Distribution	43
4.5	20 Mean Normal Distribution	44
4.6	Right Skew Heavy Tailed Gamma Distribution	45
4.7	Moderate Right Skew Gamma Distribution	45
4.8	Moderate Symmetric Gamma Distribution	46
4.9	Moderate Symmetric Heavy Tailed Gamma Distribution	46
4.10	Large Mean Gamma Distribution	47
4.11	Right Skew Chi-Square Distribution	48
4.12	Moderate Skew Chi-Square Distribution	48
4.13	Moderate Symmetric Chi-Square Distribution	49
4.14	Large Mean Moderate Symmetric Chi-Square Distribution	49
4.15	Large Mean Normal Approximating Chi-Square Distribution	50
4.16	Heavy tailed Exponential Distribution	51
4.17	Moderate Spread Exponential Distribution	51
4.18	Standard Exponential Distribution	52
4.19	Narrow Exponential Distribution	52
4.20	Short Tailed Exponential Distribution	53
4.21	Standard Cauchy (T) Distribution	54
4.22	Infinite Variance T Distribution	54
4.23	Finite Variance Heavy Tailed T Distribution	55
4.24	Moderate Tailed T Distribution	55

4.25	Approximately Normal T Distribution	56
4.26	Standard Cauchy Distribution	57
4.27	Narrow Peak Cauchy Distribution	57
4.28	Wide Peak Cauchy Distribution	58
4.29	Shifted Cauchy Distribution	58
4.30	Shifted Heavy Tailed Cauchy Distribution	59
4.31	Heavy Tailed Symmetric NIG Distribution	60
4.32	Right Skew NIG Distribution	60
4.33	Left Skew Heavy Tailed NIG Distribution	61
4.34	Large Spread NIG Distribution	61
4.35	Highly Right Skew NIG Distribution	62
4.36	Moderate Symmetric Bimodal Equally Weighted NIG Distribution . . .	63
4.37	Highly Skewed Bimodal Equally Weighted NIG Distribution	63
4.38	Highly Skewed Heavy Tailed Bimodal Equally Weighted NIG Distribution	64
4.39	Moderate Symmetric Equally Weighted NIG Distribution	64
4.40	Highly Skewed Shifted Bimodal Equally Weighted NIG Distribution . .	65
4.41	20 Mean Normal Distribution (seed = 4)	66
4.42	Large Mean Gamma Distribution (seed = 2)	66
4.43	Large Mean Normal Approximating Chi-Square Distribution (seed = 4)	67
4.44	Short Tailed Exponential Distribution (seed = 2)	67

List of Tables

4.1	Normal Distribution True vs Simulated Results	41
4.2	Gamma Distribution True vs Simulated Results	44
4.3	Chi-Square Distribution True vs Simulated Results	47
4.4	Exponential Distribution True vs Simulated Results	50
4.5	T Distribution True vs Simulated Results	53
4.6	Cauchy Distribution True vs Simulated Results	56
4.7	NIG True vs Simulated Results	59
4.8	Equally Weighted NIG True vs Simulated Results	62
4.9	Selected Different Seeded True vs Simulated Results	65

List of Abbreviations

FCFMG Feed-forward Characteristic Function Matching Generator

GAN Generative Adversarial Network

i.i.d Independent and identically distributed

MCMC Markov Chain Monte-Carlo

NIG Normal Inverse-Gaussian

ReLU Rectified Linear Unit

sup Supremum

tanh Hyperbolic tangent

Chapter 1

Introduction

1.1 Neural Networks and Modern Sampling Techniques

Advancements in computational capacity have enabled a variety of alternative methods in which to generate samples from distributions. These modern approaches make use of neural networks, trained on sets of data, to capture features of a particular distribution (Alzubaidi et al., 2021; Nowozin, Cseke, & Tomioka, 2016). The end result is a function which is able to transform an input into sample estimates which closely mimic those from a target distribution. In essence, a mapping is learned from one set of objects to another.

These methods have risen in popularity due to their ability to handle high dimensional distributions as well as un-normalised densities which commonly arise in real world applications (Perekrestenko, Eberhard, & Bölcskei, 2021). Traditional sampling methods generally tend to struggle with high dimensional spaces and are limited to one-at-a-time sample generation such as Markov Chain Monte-Carlo (MCMC) methods (Roberts & Rosenthal, 1998).

1.2 Problem Statement

Although neural network based approaches provide batch sample generation in high dimensions, these methods still require some sort of training and measure of distance between samples from a *true* distribution, and estimates generated by the network. On the other hand and as seen in Roberts & Rosenthal (1998), other traditional methods, in particular MCMC provides asymptotic guarantees about the samples they produce despite the aforementioned shortfalls.

This paper derives inspiration from the work of Brück (2025) and investigates the application and construction of a neural network sampler which uses the characteristic function of a target distribution as a measure of distance. As per Billingsley (1995), The characteristic function exists for all real valued random variables and provides global information about the underlying distribution of a particular random variable.

1.3 Research Question

The research question for this paper is whether it is indeed feasible and practical to incorporate the characteristic function in the loss of a feed-forward fully connected neural network. The aim is to produce high quality sample estimates of a particular target distribution in a univariate setting. High quality is a term used in this paper to refer to the degree by which the first and second moments (mean and variance matching) of the estimated samples agree with the true samples of the target distribution.

1.4 Hypothesis

Due to the global properties that the characteristic function provides, it is hypothesised that matching characteristic functions through squared error loss is enough to learn a mapping from input samples to sample estimates of a target distribution.

1.5 Objective

The objective for this paper is to construct a neural network sampler, called a generator, inspired by the work of Brück (2025) which learns to map standard normal input samples of arbitrary dimension to samples from a univariate target distribution via the squared error loss of the characteristic function. The ideal output would be samples estimates which match both the target distribution’s theoretical mean and variance.

1.6 Approach

The approach this paper takes is inspired by the work of Brück (2025). Brück (2025) trained a feed forward neural network to generate samples of a target density function via its corresponding characteristic function. At no point in the process is it necessary to know the density function itself. Brück (2025) incorporated the characteristic function into the loss function during back propagation to train the network on random input samples. After training, it was possible to produce close estimates of the samples from a target density function by pushing random samples from an easily sample-able distribution through the network. This avoids having to deal with highly complex densities.

This paper details the creation of a feed-forward characteristic function matching generator (FCFMG). When given an input of standard normal samples, this generator learns to produce synthetic samples of a target density without utilising the target density itself.

1.7 Method

- The generator, or model is constructed as a fully connected feed-forward neural network whose loss function is the squared error between an empirical characteristic

function of the input sample data and a true characteristic function of the target distribution.

- The model is trained on standard normal input samples, and by utilising the gradient descent algorithm during back propagation, learns a representation of the characteristic function of the target distribution in frequency space.
- The model is then tested by generating sample estimates and comparing these with samples drawn from the true distribution. If the first and second moments of the estimated samples is close to those of the true samples then the model can be deemed successful.

1.8 Structure of the Paper

This paper is separated into six parts. The introduction in chapter 1 presents a brief idea of modern sampling techniques and introduces the use of neural network based methods for sample generation. This chapter also provides particulars on what the paper aims to achieve in the form of a problem statement, research question, hypothesis, objective, approach, and method. The literature in chapter 2 details the importance of sampling and simulation, and why statisticians need specific algorithms and tools to make predictions and forecasts. Traditional and neural network based sampling algorithms are also discussed to understand the fundamental differences between the two, and to compare their respective strengths and weaknesses. Lastly, this chapter focuses on explaining the characteristic function in theoretical terms by deriving necessary forms that are used in the creation of a neural network sampler.

The methodology in chapter 3 describes the process of creating the neural network sampler. The arguments of the sampler are defined which detail how a mapping from an input to an output is computed using weights and bias terms, and how these can be transformed via an activation function to introduce nonlinear approximation capabilities to the network. The loss function, optimising and back propagation algorithms are also introduced in this chapter which detail how the neural network sampler learns to adjust the weights and bias terms iteratively to achieve a desired mapping. Lastly, a complete derivation of the back propagation terms is conducted followed by a justification for the use of the activation function. The results and data analysis in chapter 4 establishes the hyper-parameters used in the simulation of univariate sample estimates from common distributions at varying parameter values. Distributions reconstructed include the normal, gamma, chi-square, exponential, Student's t, Cauchy, a normal inverse-Gaussian, and an equally weighted normal inverse-Gaussian mixture.

The discussion in chapter 5 investigates the simulation results and highlights potential applications of a neural network sampler or lack thereof in a univariate context. The strengths and weaknesses of the analysis are also detailed, with emphasis on whether the research objective had been achieved. An appendix comprising of the full R code used to create the neural network sampler is included at the end of the paper, with the functions of the sampler being discussed component wise.

Chapter 2

Sampling, Simulation, and Characteristic Functions

2.1 Importance of Sampling and Simulation

Sampling and simulation methods are vital tools for any statistical workflow, they allow statisticians to engage with problems that are analytically intractable (no closed form solutions) by providing numerical approximations instead and provide a convenient way to conduct inference procedures (Bürkner, Schmitt, & Radev, 2025). For example, real world data rarely follows model assumptions of normality (in small samples), but because of mathematical complexity, it is convenient to work with less representative yet easier models than it is to try and model the data perfectly (Nascimento, Ramos, Elal-Olivero, Cortes-Araya, & Louzada, 2021). Classical statistical inference relies on asymptotic results for this specific reason (Bürkner et al., 2025).

Another application can be seen in the collection of samples themselves. A large component of statistical inference rests on the ability to repeat a process many times and observe results. It would be inconvenient to have to repeatedly and manually collect data to run inside a model say 100 000 times. As a result, it is common for statisticians to make an assumption about the nature of the data, and use a distribution to generate samples almost instantly. There is a trade off occurring between representational accuracy of the random process and tractability (ease of computation).

An example of this can be found in Odeyar, Apel, Hall, Zon, & Skrzypkowski (2022), where analysing the time it takes for a machine to break down can be parametrically modeled using an exponential distribution. Instead of having to collect thousands of real world samples, it would be easier to collect a small sample and make an assumption that the time it takes for a machine to break down is exponentially distributed. The small initial sample informs a parameter, in this case the rate, of a known distribution which is easily sampled from. Sampling thus allows statisticians to make inference through simulation based approaches which allow them to explore uncertainty and make predictions. This process becomes more involved however in contexts where the desired quantities, for example moments of a random variable, are complex and impossible to solve analytically (Kleiber, 2014).

2.2 Sampling Algorithms

2.2.1 Traditional Sampling Algorithms

Traditional sampling algorithms such as the inverse sampling method, rejection sampling, and compound transformations (multistage stochastic representations) have served as the primary tools for generating samples from target distributions and typically involve an analytic transformation of samples drawn from simpler distributions to more complex ones (Al-Labadi & Ly, 2024). For example, to generate exponentially distributed samples, the inverse transform method is generally used. Uniform samples are initially drawn, which are then transformed analytically via the inverse cumulative distribution function of the exponential distribution (Devroye, 2006). This is how the ‘rexp()’ function in R works (R Core Team, 2025).

An algebraic example from Wilks (2011) of this method applied to the exponential distribution would proceed firstly by setting the cumulative density function equal to a uniform random variable $F_X(x) = 1 - e^{-\lambda x} = U$, and then re-writing the expression to isolate x , yielding $x = -\frac{1}{\lambda} \ln(1 - U) = F_X^{-1}(U)$. Since $U \sim U(0, 1)$, it is possible to say $(1 - U) \sim U(0, 1)$. Letting $1 - U = U^*$ yields $x = -\frac{1}{\lambda} \ln(U^*)$ which is equivalent to $x = \frac{1}{\lambda} \ln(\frac{1}{U^*})$. Letting $\lambda = 1$ yields $x = \ln(\frac{1}{U^*})$ which is the form seen in Devroye (2006). Exponentially distributed random variables are then generated by sampling from the uniform distribution and transforming them thereafter.

Rejection sampling involves choosing a proposal distribution, $q(x)$, which is easily sampled from, and finding a constant c , which satisfies the condition $p(x) \leq cq(x) \quad \forall x$ where $p(x)$ is the target distribution. The algorithm samples a particular x from $q(x)$, samples a $u \sim U(0, 1)$ and accepts x if $u \leq \frac{p(x)}{cq(x)}$ (Caffo, Booth, & Davison, 2002). To generate samples from a gamma distribution, a special form of rejection sampling is typically used called the Marsaglia and Tsang method which involves a transformation step and where $q(x)$ is a standard normal distribution (Marsaglia & Tsang, 2000).

Compound transformations work by sampling from intermediate distributions and using these results in further sample generation. For example, in order to generate a sample from a Normal Inverse-Gaussian (NIG) distribution, an initial sample is drawn from an inverse-Gaussian distribution, upon which a further sample is drawn from a Normal distribution that has been conditioned on that initial sample (Rydberg, 1997). The parameters of the second sample’s distribution depends on the first sample.

These methods are efficient for univariate random variables, but struggle in higher dimensions and require analytic forms of the underlying distribution (Dolgov, Anaya-Izquierdo, Fox, & Scheichl, 2020; Tanizaki, 2004). For example, inverse transform sampling would require an analytic multidimensional inverse cumulative distribution function which would need to be evaluated each time a new sample is generated. Rejection sampling would require a good proposal distribution and proper choice of c which would cover the entire space of $p(x)$ which would be difficult in high dimensions (Li & Bickel, 2005). Furthermore, in rejection sampling, a single point in the probability space is either accepted or rejected at a time, meaning that the sample generation process can become taxing for high dimensional random vector variates (Dymetman, Bouchard, & Carter, 2012).

2.2.2 Neural Network Samplers

In contrast to these traditional sampling algorithms, modern alternatives in the form of neural network samplers have emerged to address some of the issues particularly revolving around high dimensional distributions. These neural networks learn an implicit mapping from some simple distribution to a more complex one, but enjoy the benefits of amortised, independent, and parallel sample generation at the cost of exactness (the samples produced are estimates to those of the target distribution) (Hu et al., 2018; Zammit-Mangion, Sainsbury-Dale, & Huser, 2025).

Amortised means that the computational cost of the sample generation procedure is incurred during the training phase of the network (Choi, Mittal, Elvira, Park, & Malkin, 2025). Once the network has learned the mapping, then producing the actual sample estimates is quick. This occurs because once optimal parameters for the network have been found during the training phase, sample generation is a matter of pushing new random samples through a deterministic function to transform them into target sample estimates. An example of this procedure can be seen in figures 3.1 and 3.2.

The property of independence arises from the independent random initialisation of the input samples. The neural network sampler acts as a deterministic transformation only, meaning that if the input samples were independently sampled, the output will retain that independence property (Cunningham, Cobb, & Jha, 2022). This is in contrast to traditional sampling methods such as Markov Chain Monte-Carlo (MCMC) methods which produce correlated samples since each new sample is dependent on the previous sample (Roberts & Rosenthal, 1998).

Parallel in this context refers to the ability to produce many samples in batches. This property comes from the structure of a neural network, where it is able to take as input a matrix of samples of arbitrary dimension (Kosiorek, 2024). The matrix algebra involved in the deterministic function representation of the network works to transform each row of the input matrix to become target sample estimates at the same time or in parallel. This is in contrast to rejection sampling which accepts or rejects one point at a time.

A downside however is that the samples produced by neural network samplers are not *true* to the target distribution, rather they are estimates. In essence, the benefits of using a neural network sampler revolve around not needing an analytic transformation to be known in order to complete the mapping from input samples to output samples. Once trained, these networks can be very fast at producing samples which mimic those of a target distribution (Hu et al., 2018).

2.3 The Characteristic Function

The characteristic function of the random variable X has the following form:

$$\phi(t) = E[e^{itX}] = \int_{\mathbb{R}} e^{itX} f_X(x) dx = \int_{-\infty}^{\infty} e^{itx} f(x) dx. \quad (2.1)$$

This function uniquely defines the probability distribution of X , and is equivalent to the probability density and distribution functions of X since knowing one informs the other

completely (Billingsley, 1995). Characteristic functions are convenient to work with since they always exist for any real valued random variable, since

$$|e^{itX}| = |\cos(tX) + i\sin(tX)| = \sqrt{\cos(tX)^2 + \sin(tX)^2} = 1.$$

As a result

$$E[|e^{itX}|] = 1 < \infty \quad \forall t, X \in \mathbb{R}$$

which implies that $E[e^{itX}]$ always exists and is finite (Billingsley, 1995).

The characteristic function is the fourier transform of the density function with a variable change and sign reversal. This is why knowing the characteristic function is equivalent to knowing the density or distribution functions since it is possible to transform between all three in theory. In practice this can be non-trivial since the integrals may have no closed form solutions, meaning Monte-Carlo methods would need to be used which introduces numerical complications (Brück, 2025; Lamonaca, 2025).

Given the conventional fourier transform of a function $f(x)$

$$\hat{f}(u) = \int_{-\infty}^{\infty} e^{-i2\pi ux} f(x) dx, \quad \forall u \in \mathbb{R}, \quad (2.2)$$

the corresponding inverse fourier transform is:

$$f(x) = \int_{-\infty}^{\infty} e^{i2\pi ux} \hat{f}(u) du, \quad \forall u \in \mathbb{R}. \quad (2.3)$$

Let $t = -2\pi u$, then $u = -\frac{t}{2\pi}$ where $du = -\frac{1}{2\pi} dt$. As a result, equation (2.2) becomes

$$\hat{f}\left(-\frac{t}{2\pi}\right) = \int_{-\infty}^{\infty} e^{itx} f(x) dx = \phi(t).$$

As $u \rightarrow \infty$ then $t \rightarrow -\infty$, and as $u \rightarrow -\infty$ then $t \rightarrow \infty$ which means that the bounds of integration flip.

Applying equation (2.3) with the variable change yields

$$\begin{aligned} f(x) &= \int_{u=-\infty}^{u=\infty} \hat{f}(u) e^{i2\pi ux} du, \quad \forall x \in \mathbb{R} \\ &= \int_{t=\infty}^{t=-\infty} \hat{f}\left(-\frac{t}{2\pi}\right) e^{i2\pi\left(-\frac{t}{2\pi}\right)x} \left(-\frac{1}{2\pi}\right) dt \\ &= -\frac{1}{2\pi} \int_{\infty}^{-\infty} \phi(t) e^{-itx} dt \\ &= \frac{1}{2\pi} \int_{-\infty}^{\infty} \phi(t) e^{-itx} dt \end{aligned} \quad (2.4)$$

Equation (2.4) is the inverse fourier transform of $f(x)$ in terms of $\phi(t) = \hat{f}\left(-\frac{t}{2\pi}\right)$.

Equation (2.1) can be generalised for a random vector variate $\mathbf{X}$

$$\phi(\mathbf{t}) = E[e^{it'\mathbf{X}}] \quad \text{where } \mathbf{t} \in \mathbb{R}^p. \quad (2.5)$$

2.3.1 Examples of Univariate Characteristic Functions

Univariate Normal

$$\phi(t) = e^{i\mu t - \frac{1}{2}\sigma^2 t^2}.$$

Gamma

$$\phi(t) = \left(1 - \frac{it}{\lambda}\right)^{-\alpha}.$$

Exponential

$$\phi(t) = \frac{\lambda}{\lambda - it}.$$

Chi-Square

$$\phi(t) = (1 - 2it)^{-k/2}.$$

Student's T

$$\phi(t) = \frac{(\sqrt{\nu}|t|)^{\nu/2} K_{\nu/2}(\sqrt{\nu}|t|)}{\Gamma(\nu/2) 2^{\nu/2-1}}$$

where $K_{\nu/2}(\sqrt{\nu}|t|)$ is a modified Bessel function of the second kind, that is

$$K_{\nu/2}(\sqrt{\nu}|t|) = \int_0^\infty e^{-\sqrt{\nu}|t| \cosh(t)} \cosh(\nu t/2) dt.$$

The t is not the frequency in the above definition, it is an integration dummy variable.

Cauchy

$$\phi(t) = e^{i\mu t - \gamma|t|}.$$

Normal Inverse Gaussian

$$\phi(t) = e^{i\mu t + \delta \left(\gamma - \sqrt{\alpha^2 - (\beta + it)^2} \right)}.$$

2.3.2 Characteristic Functions of Mixture Distributions

A mixture distribution has a density function which is a convex combination of two or more density functions (McLachlan, Lee, & Rathnayake, 2019; Zeevi & Meir, 1997). A convex combination is a linear combination where each coefficient is positive and whose sum is equal to 1. An example of a convex combination will be:

$$Y = a_1 X_1 + a_2 X_2 + \cdots + a_n X_n \text{ where } \sum_{i=1}^n a_i = 1 \text{ and } a_i \geq 0 \quad \forall i.$$

Let M be a mixture distribution, $F = f_1, \dots, f_n$ be a set of n different probability density functions, π_h be the mixture weights such that $\sum_{h=1}^n \pi_h = 1$ and $\pi_h \geq 0 \quad \forall h$, and X be a random variable from the distribution M . Then from Al-Labadi & Ly (2024), the distribution function of M , the mixture distribution, is given by

$$f(x) = \sum_{h=1}^n \pi_h f_h(x) \quad \text{for } h = 1, \dots, n,$$

where $X \sim f_h$ with probability π_h , and each f_h has characteristic function ϕ_h .

Theorem 1. *The characteristic function of any finite mixture distribution is the convex combination of its component characteristic functions.*

$$\begin{aligned} \phi(t) &= E[e^{itX}] \\ &= \int_{-\infty}^{\infty} e^{itx} \left(\sum_{h=1}^n \pi_h f_h(x) \right) dx \\ &= \sum_{h=1}^n \pi_h \int_{-\infty}^{\infty} e^{itx} f_h(x) dx \\ &= \sum_{h=1}^n \pi_h \phi_h(t). \end{aligned}$$

Consider an example of an equally weighted Normal Inverse Gaussian mixture distribution, let $\phi_1(t)$ be the characteristic function of the first NIG distribution with parameters $\alpha_1; \beta_1; \delta_1; \mu_1$, and $\phi_2(t)$ be the characteristic function of the second NIG distribution with parameters $\alpha_2; \beta_2; \delta_2; \mu_2$.

From theorem 1 the characteristic function of this mixture distribution is given by

$$\phi(t) = \frac{1}{2} \left(e^{i\mu_1 t + \delta_1 \left(\gamma_1 - \sqrt{\alpha_1^2 - (\beta_1 + it)^2} \right)} + e^{i\mu_2 t + \delta_2 \left(\gamma_2 - \sqrt{\alpha_2^2 - (\beta_2 + it)^2} \right)} \right).$$

2.3.3 The Empirical Characteristic Function

In circumstances where the true characteristic function is not known, an empirical counterpart can be constructed to estimate it. The main difference between computing the empirical estimate and the true characteristic function is that in the former, some sample data would need to be collected to construct it (Yu, 2004). The reason this is not the case in the latter is because by definition (2.1), the true characteristic function integrates out all of the data.

Let $X_1, X_2, \dots, X_n$ be independent and identically distributed (i.i.d) random variables from a probability distribution with characteristic function $\phi(t)$. The empirical characteristic function $\hat{\phi}(t)$ is given by:

$$\hat{\phi}(t) = \frac{1}{n} \sum_{h=1}^n e^{itX_h}. \quad (2.6)$$

It can be observed that this is just the sample average of the true characteristic function (Yu, 2004).

Theorem 2. *The empirical characteristic function is an unbiased.*

$$\begin{aligned} E[\hat{\phi}(t)] &= E\left[\frac{1}{n} \sum_{h=1}^n e^{itX_h}\right] = \frac{1}{n} \sum_{h=1}^n E[e^{itX_h}] \\ &= \frac{1}{n} \sum_{h=1}^n \phi(t) \quad \text{since } X_h \text{ are i.i.d.} \rightarrow E[e^{itX_h}] = \phi(t) \quad \forall h \\ &= \frac{1}{n} n\phi(t) = \phi(t). \end{aligned}$$

Theorem 3. *The empirical characteristic function is consistent.*

$$Y_h := e^{itX_h} \text{ for } h = 1, \dots, n,$$

and

$$|Y_h| = |e^{itX_h}| = 1 \rightarrow E[|Y_h|] = 1 < \infty.$$

Since the Y_h are bounded and i.i.d, because the X_h are i.i.d, by the strong law of large numbers,

$$\frac{1}{n} \sum_{h=1}^n Y_h \xrightarrow{a.s.} E[Y] \rightarrow \frac{1}{n} \sum_{h=1}^n e^{itX_h} \xrightarrow{a.s.} E[e^{itX}] \leftrightarrow \hat{\phi}(t) \xrightarrow{a.s.} \phi(t) \quad \text{for fixed } t.$$

Almost sure (a.s) convergence is stronger than convergence in probability and means that after some point, the empirical estimate will stay close to the true characteristic function forever as n gets large. Convergence in probability on the other hand means that the probability of the empirical estimate deviating from the true characteristic function gets smaller as n gets large, but there is no condition on the empirical estimate being close to the true characteristic function forever after a finite point, there will always be small deviation. Almost sure convergence implies convergence in probability (Billingsley, 1995; Cox, 2020).

Theorem 4. *The empirical characteristic function is uniform convergent.*

Since X_h are i.i.d and $E[|X|] = 1 < \infty$,

$$\sup_{t \in [-T, T]} |\hat{\phi}_n(t) - \phi(t)| \xrightarrow{a.s.} 0.$$

Theorem 4 comes from Csörgő (1981) and involves the supremum instead of the maximum because it is not necessarily the case that the maximum will exist in the set given that $\hat{\phi}$ is a function of random samples X_h . This means that $\hat{\phi}$ can get arbitrarily close to ϕ and stay there forever, but this occurs asymptotically, implying that the greatest deviation between the two functions may never obtain a definite maximum value.

An example can be given from introductory real analysis, consider the set $(0, 1)$. This set has no maximum nor minimum value since it is possible to get arbitrarily close to 1 or 0 forever. It is possible to say however that the supremum of the set is 1 and the infimum of the set is 0. For intuitive purposes, theorem 4 can be interpreted as the biggest difference across all values t between the empirical characteristic function and the true characteristic function tends to zero as $n \rightarrow \infty$ (Binkowski & Kozek, 2010). Uniform convergence ensures that the shape of the function $\hat{\phi}(t)$ approaches $\phi(t)$ on all points t at the same rate.

2.3.4 The Multivariate Empirical Characteristic Function

Expanding upon equation (2.6) for a multivariate context, denote $\mathbf{x} \in \mathbb{R}^p$ as the random variable of interest. Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ denote the data matrix whose rows are samples of $\mathbf{x}$ and whose columns are features of $\mathbf{x}$, $\mathbf{x}_h \in \mathbb{R}^p$ for $h = 1, \dots, n$ denote the h^{th} sample from $\mathbf{X}$, $\mathbf{t}_j \in \mathbb{R}^p$ for $j = 1, \dots, m$ denote the j^{th} vector of frequencies, that is scalar values at which to evaluate the characteristic function, and $\mathbf{1} \in \mathbb{R}^n$ denote a vector of 1's.

The empirical characteristic function evaluated at point $\mathbf{t}_j$ is given by

$$\hat{\phi}(\mathbf{t}_j) = \frac{1}{n} \sum_{h=1}^n e^{it'_j \mathbf{x}_h} = \frac{1}{n} \left(\sum_{h=1}^n \cos(\mathbf{t}'_j \mathbf{x}_h) + i \sum_{h=1}^n \sin(\mathbf{t}'_j \mathbf{x}_h) \right). \quad (2.7)$$

This can be split into real and imaginary parts respectively as

$$\hat{\phi}(\mathbf{t}_j) = \left[\frac{1}{n} \sum_{h=1}^n \cos(\mathbf{t}'_j \mathbf{x}_h) \quad \frac{1}{n} \sum_{h=1}^n \sin(\mathbf{t}'_j \mathbf{x}_h) \right]_{1 \times 2}. \quad (2.8)$$

If all of the $\mathbf{x}_h$ samples are collected into the matrix $\mathbf{X}$, then

$$\mathbf{X} \mathbf{t}_j = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}_{n \times p} \begin{bmatrix} t_{j1} \\ t_{j2} \\ \vdots \\ t_{jp} \end{bmatrix}_{p \times 1} = \begin{bmatrix} \mathbf{t}'_j \mathbf{x}_1 \\ \vdots \\ \mathbf{t}'_j \mathbf{x}_n \end{bmatrix}_{n \times 1},$$

on which we define

$$\cos(\mathbf{X} \mathbf{t}_j) := \begin{bmatrix} \cos(\mathbf{t}'_j \mathbf{x}_1) \\ \vdots \\ \cos(\mathbf{t}'_j \mathbf{x}_n) \end{bmatrix},$$

and

$$\sin(\mathbf{X} \mathbf{t}_j) := \begin{bmatrix} \sin(\mathbf{t}'_j \mathbf{x}_1) \\ \vdots \\ \sin(\mathbf{t}'_j \mathbf{x}_n) \end{bmatrix}.$$

As a result

$$\hat{\phi}(\mathbf{t}_j) = \left[\frac{1}{n} \mathbf{1}' \cos(\mathbf{X} \mathbf{t}_j) \quad \frac{1}{n} \mathbf{1}' \sin(\mathbf{X} \mathbf{t}_j) \right]_{1 \times 2} \quad (2.9)$$

Now introduce $\mathbf{T} \in \mathbb{R}^{m \times p}$ whose elements are $t_{jl} \in \mathbb{R}$ for $j = 1, \dots, m$ and $l = 1, \dots, p$ such that:

$$\mathbf{X} \mathbf{T}' = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}_{n \times p} \begin{bmatrix} t_{11} & t_{21} & \cdots & t_{m1} \\ t_{12} & t_{22} & \cdots & t_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ t_{1p} & t_{2p} & \cdots & t_{mp} \end{bmatrix}_{p \times m} = \begin{bmatrix} \mathbf{t}'_1 \mathbf{x}_1 & \mathbf{t}'_2 \mathbf{x}_1 & \cdots & \mathbf{t}'_m \mathbf{x}_1 \\ \mathbf{t}'_1 \mathbf{x}_2 & \mathbf{t}'_2 \mathbf{x}_2 & \cdots & \mathbf{t}'_m \mathbf{x}_2 \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{t}'_1 \mathbf{x}_n & \mathbf{t}'_2 \mathbf{x}_n & \cdots & \mathbf{t}'_m \mathbf{x}_n \end{bmatrix}_{n \times m}.$$

Applying the function $f(\cdot)$ element-wise, dividing by the total number of samples n , and summing over the columns yields

$$\begin{aligned} \frac{1}{n} \mathbf{1}' f(\mathbf{X} \mathbf{T}') &= \frac{1}{n} \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix}_{1 \times n} \begin{bmatrix} f(\mathbf{t}'_1 \mathbf{x}_1) & \cdots & f(\mathbf{t}'_m \mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ f(\mathbf{t}'_1 \mathbf{x}_n) & \cdots & f(\mathbf{t}'_m \mathbf{x}_n) \end{bmatrix}_{n \times m} \\ &= \frac{1}{n} \begin{bmatrix} \sum_{h=1}^n (f(\mathbf{t}'_1 \mathbf{x}_h)) & \cdots & \sum_{h=1}^n (f(\mathbf{t}'_m \mathbf{x}_h)) \end{bmatrix}_{1 \times m}. \end{aligned} \quad (2.10)$$

Using equation (2.9) it is possible to re-write equation (2.10) as

$$\frac{1}{n} \mathbf{1}' f(\mathbf{X} \mathbf{T}') = \left[\frac{1}{n} \mathbf{1}' f(\mathbf{X} \mathbf{t}_1) \quad \cdots \quad \frac{1}{n} \mathbf{1}' f(\mathbf{X} \mathbf{t}_m) \right]_{1 \times m}. \quad (2.11)$$

Using $\cos(\cdot)$ as $f(\cdot)$ in equation (2.11) yields

$$\frac{1}{n} \mathbf{1}' \cos(\mathbf{X} \mathbf{T}') := \left[\frac{1}{n} \mathbf{1}' \cos(\mathbf{X} \mathbf{t}_1) \quad \cdots \quad \frac{1}{n} \mathbf{1}' \cos(\mathbf{X} \mathbf{t}_m) \right]_{1 \times m}, \quad (2.12)$$

and using $\sin(\cdot)$ as $f(\cdot)$ in equation (2.11) yields

$$\frac{1}{n} \mathbf{1}' \sin(\mathbf{X} \mathbf{T}') := \left[\frac{1}{n} \mathbf{1}' \sin(\mathbf{X} \mathbf{t}_1) \quad \cdots \quad \frac{1}{n} \mathbf{1}' \sin(\mathbf{X} \mathbf{t}_m) \right]_{1 \times m} \quad (2.13)$$

Equations (2.12) and (2.13) are the real and imaginary parts respectively for all $h = 1, \dots, n$ samples and $j = 1, \dots, m$ frequencies. These can be transposed to obtain $m \times 1$ vectors, and combined to form a matrix. This is now the representation of the empirical characteristic function for all h, j

$$\hat{\phi}(\mathbf{T}) = \begin{bmatrix} \frac{1}{n} \cos(\mathbf{T} \mathbf{X}') \mathbf{1} & \frac{1}{n} \sin(\mathbf{T} \mathbf{X}') \mathbf{1} \end{bmatrix} = \begin{bmatrix} \Re(\hat{\phi}(\mathbf{t}_1)) & \Im(\hat{\phi}(\mathbf{t}_1)) \\ \vdots & \vdots \\ \Re(\hat{\phi}(\mathbf{t}_m)) & \Im(\hat{\phi}(\mathbf{t}_m)) \end{bmatrix} = \begin{bmatrix} \hat{\phi}_{11} & \hat{\phi}_{12} \\ \vdots & \vdots \\ \hat{\phi}_{m1} & \hat{\phi}_{m2} \end{bmatrix}_{m \times 2}. \quad (2.14)$$

2.4 Sampling from Complex Distributions

Consider an example of modelling the portfolios of stocks. Alayón et al. (2017) discuss that many models in finance have used the normal distribution for to model returns, with the rationale being that the distribution is well studied, easily sampled from and whose moments are known. But this would be a poor fit since real world portfolio returns have fat tails, or non-negligible probability in the lower and upper end of the spectrum of values, and are generally skew in distribution (De Domenico, Livan, Montagna, & Nicrosini, 2023; Scherer, Rachev, Kim, & Fabozzi, 2012).

A better approach would be to use the Normal Inverse-Gaussian distribution which is popular in the financial literature and is able to capture fat tails and skewness (Barndorff-Nielsen, 1997; Pacheco-González, 2009). The issue however is that the NIG has no closed form cumulative distribution function, it has a non trivial probability density function, and requires a multistage sampling procedure since it is a variance-mean mixture of a normal distribution with an inverse Gaussian mixing variable (Pacheco-González, 2009). What this means is that conducting sampling and running simulations will require more effort since the NIG random variable will need to be constructed through a stochastic representation in a process which requires variable transformation (Rydberg, 1997).

The probability density function of the NIG is given by

$$f(x) = \frac{\alpha \delta K_1 \left(\alpha \sqrt{\delta^2 + (x - \mu)^2} \right)}{\pi \sqrt{\delta^2 + (x - \mu)^2}} e^{\delta \gamma + \beta(x - \mu)} \quad \forall x \in \mathbb{R}$$

where $\gamma := \sqrt{\alpha^2 - \beta^2}$ and the modified Bessel function of the second kind with 1 degree of freedom (FPW, 1923) has the form

$$K_1(z) = \int_0^\infty e^{-z \cosh(t)} \cosh(t) dt.$$

The parameters in this distribution are:

- $\mu \in \mathbb{R}$ is the location parameter.
- $\alpha \in \mathbb{R}$ where $\alpha > 0$, is the tail heaviness parameter. Larger values provide steeper density.
- $\beta \in \mathbb{R}$ where $|\beta| < \alpha$, is the asymmetry parameter. $\beta = 0$ provides a symmetric distribution.
- $\delta \in \mathbb{R}$ where $\delta > 0$, is the scale parameter.

The NIG has characteristic

$$\phi(t) = e^{i\mu t + \delta \left(\gamma - \sqrt{\alpha^2 - (\beta + it)^2} \right)}$$

The modified Bessel function of the second kind with one degree of freedom K_1 , or per Barndorff-Nielsen (1997), the third kind in older literature, makes sampling methods such as rejection sampling numerically unstable. This is because per Casella, Robert, & Wells (2004), rejection sampling requires the density function to be evaluated multiple times,

with the Bessel function being an integral needing approximation or a series of many terms to be evaluated. As z gets large, the Bessel function exponentially decays to 0 and floating point precision cannot accommodate output values (Cuingnet, 2023). Similarly, when z gets small, then the integration computation requires a large domain of t in order to capture tail behaviour as the integrand becomes flat, leading to increased computational cost (Plesner, Sørensen, & Hauberg, 2024). These issues are called underflow and overflow respectively.

Additionally, per Hörmann & Leydold (2003), inverse transform sampling requires the cumulative distribution function to be known, which poses a problem considering the NIG has no closed form distribution function (He & Wang, 2021). What is known and relatively simple is its characteristic function. Recall from equation (2.6) that the characteristic function is the sample average of the true characteristic function. This means that the characteristic function can always be estimated. This means that the actual form of the function does not need to be known. This enables a procedure which takes advantage of the characteristic function and suggests that it is not necessary to know the density of the NIG in order to sample from it, thus mitigating the trade off of representational accuracy and tractability. It is then possible to consider applying a neural network sampler which takes advantage of the characteristic function in order to simulate samples from the target distribution.

2.5 Theoretical Benefits of a Characteristic Function-based Method

The characteristic function captures global properties of the underlying density, and as such, incorporating it into the loss function can allow a neural network sampler to learn over the entire support (Brück, 2025; Du & Dang, 2025). What this means is that a characteristic function based method should be able to capture information about distributions which have heavy tails. Take for example distributions used in the field of insurance, where although most probability mass is concentrated in a particular domain window, there is a non-negligible mass distributed in the tails (Eling, 2012). Conventional sampling needs extra care, for example larger samples and importance sampling, in dealing with such distributions to pick up rare events (Hult & Svensson, 2009).

Employing characteristic functions in the loss can also theoretically mitigate issues around heavy tail behaviour. Other types of loss functions such as likelihood based loss prioritise regions where the density is concentrated, hence potentially ignoring tail behaviour. This is because the likelihood is dominated by the data which is concentrated wherever the bulk of the distribution may be as opposed to the tails (Joe & Li, 2024). Similarly and according to Rangwani, Jaswani, Karmali, Jampani, & Babu (2022), traditional Generative Adversarial Networks (GANs) can ignore tail behaviour due to skewed training data and the presence of small gradients resulting from the vanishing gradient phenomenon (Arjovsky, Chintala, & Bottou, 2017).

The characteristic function on the other hand is the expectation over the full distribution, hence using it in the loss implies that all parts of the distribution in question influence the loss which gets back propagated into the network parameters (Brück, 2025). What this means is that if it is possible to capture the behaviour of the characteristic function

in frequency space, then it should not matter what the tail behaviour of the underlying density is in samples space, as knowing one implies information about the other. What would be important however is that large frequencies of t are captured to model the tails accurately.

A primary difference between a characteristic function matching approach versus a naive generative neural sampler is that in the latter case, the model tries to match in sample space. What this means, is that it replicates one particular realisation of a distribution. The generator discussed in this paper tries to match in frequency space, meaning that it is not learning a specific realisation of a target distribution but a function describing its global properties. The difference is that the frequency space of the distribution is deterministic and does not change. Each realisation of a particular distribution’s samples however is random.

What this means is that there are guarantees of the closeness of the generator’s reconstruction of the target distribution and the target distribution itself which are discussed thoroughly in Brück (2025). This is because the characteristic function by definition uniquely ‘characterises’ the underlying random variable (Billingsley, 1995). In essence, if the error reaches perfect 0, then the generator’s reconstruction and the target distribution are the same.

2.6 Caveats of a Characteristic Function-Based Neural Network Sampler

In order to make use of the uniqueness of the characteristic function, it is necessary to have a large domain of frequencies t , as choosing only a window or small subset will only allow the model to capture where the bulk of the probability mass is concentrated, and it will not be possible to model the tails accurately. This result is trivial to understand when thinking about the characteristic function as any polynomial which is being approximated. The characteristic function is evaluated point wise at every t , and so it is beneficial to try and cover as much of the function with the domain as possible. Weighting the loss function would also generally be necessary as uniform weighting, or not weighting the loss at all, would mean that no attention is being paid to where the data is dense and where it is sparse. Brück (2025) makes use of kernels for this reason.

Recall the definition of the characteristic function in equation (2.1), it can be observed that as t gets large, then $\phi(t)$ tends to 0. Consider the characteristic function of the normal distribution $\phi(t) = e^{i\mu t - \frac{1}{2}t^2\sigma^2}$. It is clear that as t increases, then $\phi(t)$ decreases quadratically to zero. Conversely, consider the Cauchy distribution with characteristic function $\phi(t) = e^{i\mu t - \gamma|t|}$. As t increases, $\phi(t)$ decreases to zero but at a slower rate. This behaviour suggests that heavy tailed distributions have slower decaying characteristic functions.

As discussed in Jordão (2020), the smoother the function, the faster the fourier transform decays and the less smooth a function, the slower the fourier transform decays. Specifically, functions which belong to Lipschitz or Besov space, enjoy faster decay in their fourier transform. Considering that the characteristic function is a fourier transform of the probability density function with a sign and variable change, the idea of fast vs slow decay in frequency space applies here and has consequences. If the distribution is tail

heavy, then this would correspond to a slow decaying characteristic function which would be harder to approximate in frequency space. This suggests that capturing tail heaviness is not as easy as switching to the frequency space.

Numerical instability, such as divergent gradient updates, can become an issue when imitating certain distributions since the fundamental updating algorithm of a fully connected feed-forward neural network relies on gradient descent (Karner, Kazeev, & Petersen, 2024). This means the model is not immune to getting stuck in local minimums. For example, characteristic functions with high oscillatory behaviour such as the large mean normal distribution in figure 4.5 can cause gradients to be sensitive to small changes in t and make back propagation unstable (Sitzmann, Martel, Bergman, Lindell, & Wetzstein, 2020).

If the true characteristic function is also not known and has to be estimated with the empirical equivalent, then working in high dimensional space and or with sparse samples will make for a poor target for the loss function. This is because the empirical characteristic function is not exact and only matches the true characteristic function in the limit (Ferré & Whittaker, 2004). Hence, having two empirical estimates in the loss instead of the empirical/true combination will introduce variance into the model.

Since the generator is a neural network, all of the caveats of a neural network carry over to the idea of using a generator (Porkodi, Sarada, Maik, & Gurushankar, 2022). Most notably that the generator needs to be trained first, there is no human interpretation of the parameters of the generator (black box), the generator may not converge, and it is very sensitive to parameter initialisation (Sharma, Kumar, Sharma, & Biju, 2024).

Like most numerical optimisation methods, there needs to be an initial starting point for the optimising algorithm to work from. In the context of neural networks, these initial starting points are the individual weights in each weight matrix and every associated bias term. If the network is large, this could mean initialising millions of individual weights. What this means is that running a neural network with the same hyper-parameters, such as the amount layers, neurons, passes, etc, can lead to very different results because the weights get reassigned random values each time the network is initialised (Dodge et al., 2020).

Chapter 3

Constructing The Feed-forward Characteristic Function Matching Generator

The FCFMG is a neural network sampler which is able to learn a mapping which takes random noise as input and transforms it into samples which mimic those of a target distribution after a single forward push once it has converged. Consider a generator G where Ω is a set of the generator's parameters, A is the target distribution, and θ is a set of the target distribution's parameters.

$$G_{\Omega} : \mathbb{R}^{n \times p} \rightarrow \mathbb{R}^{n \times p}, \text{ with rows of } \mathbf{X} \stackrel{\text{i.i.d.}}{\sim} N_p(\mathbf{0}, \mathbf{I}) \mapsto \hat{\mathbf{Y}} = G_{\Omega}(\mathbf{X}), \text{ with rows of } \hat{\mathbf{Y}} \sim A(\theta) \quad (3.1)$$

Definition (3.1) is interpreted as a generator called G which is a function of parameters Ω , which transforms an input of dimension $\mathbb{R}^{n \times p}$ to an output of dimension $\mathbb{R}^{n \times p}$. The input is a matrix called $\mathbf{X}$ whose rows are the noise samples of dimension p and are each drawn from a standard normal distribution. The output is a matrix called $\hat{\mathbf{Y}}$ whose rows are transformed samples of dimension p which mimic samples that come from a distribution A which has parameter vector θ . Since the generator G is a transformation on $\mathbf{X}$, we say that the output is a function of the generator $\hat{\mathbf{Y}} = G_{\Omega}(\mathbf{X})$.

3.1 Algorithm

The network is fed an input of noise samples drawn from a standard normal distribution, and outputs a transformation of this noise which is then used to compute an empirical characteristic function. The empirical characteristic function is then compared to a true characteristic function of a target distribution via a squared error loss function and the error is back propagated through the network.

Algorithm FCFMG

- 1: **Initialise** random noise $\mathbf{X}_{n \times p}$ where $\mathbf{x}_h \sim N_p(\mathbf{0}, \mathbf{I})$ for $h = 1, \dots, n$.
 - 2: **Initialise** random frequencies $\mathbf{T}_{m \times p}$ where $\mathbf{t}_j \sim N_p(\mathbf{0}, \mathbf{I})$ for $j = 1, \dots, m$.
 - 3: **Input:** $\mathbf{X}$, $\mathbf{T}$, characteristic function, distribution parameters, architecture, activation functions, loss type (SEL), learn rate, passes.
 - 4: **for** $i = 1, \dots, N$ **do**
 - 5: Push $\mathbf{X}$ through network to obtain $\hat{\mathbf{Y}}^{(i)}$.
 - 6: Compute $\hat{\phi}_{\hat{\mathbf{Y}}^{(i)}}(\mathbf{T}) := \hat{\phi}(\mathbf{T})$.
 - 7: Evaluate $L = \sum_{j=1}^m \|\phi(\mathbf{t}_j) - \hat{\phi}(\mathbf{t}_j)\|_F^2 = \|\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T})\|_F^2$.
 - 8: Back propagate L through the generator using the gradient descent algorithm.
 - 9: **end for**
 - 10: **Output:** new $\hat{\mathbf{Y}}$ after pushing new $\mathbf{X}$ through the network.
-

The following figure demonstrates the training algorithm for the generator. This process continues until a necessary stopping condition has been met. The stopping condition in this case is meeting a certain number of passes.

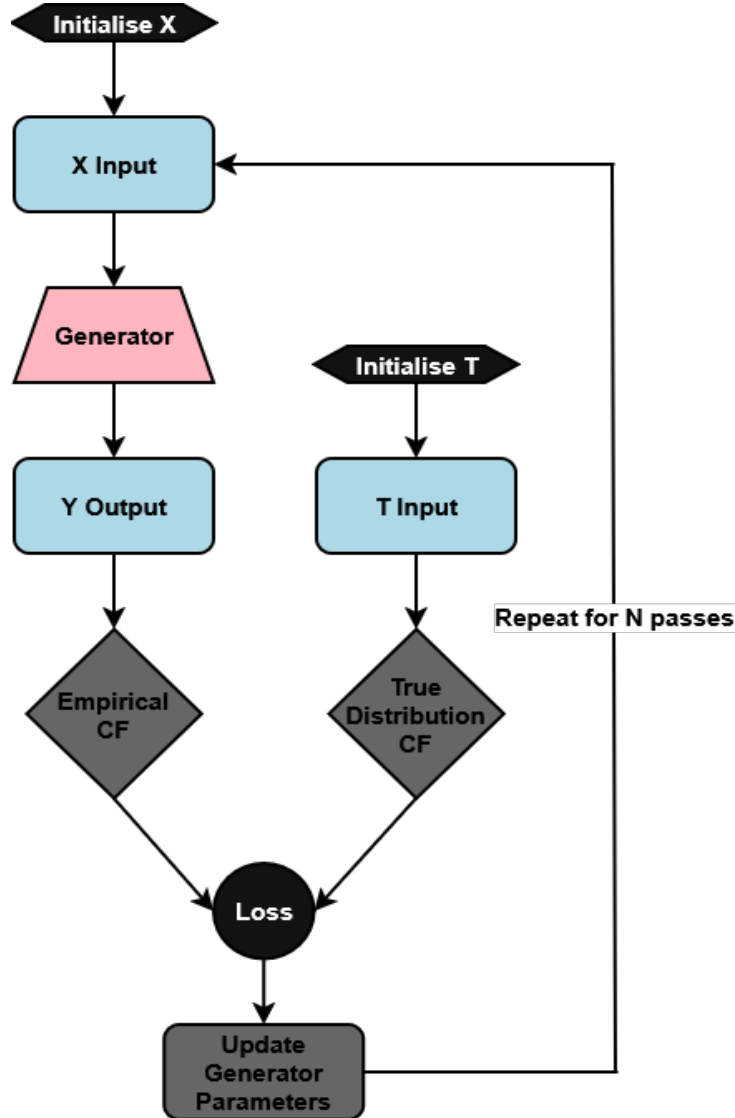


Figure 3.1: Training Cycle

Once the training has been complete, all that is left is the generator itself and new random sample inputs. The generator in this case has become a deterministic function mapping from the input samples to target sample estimates.

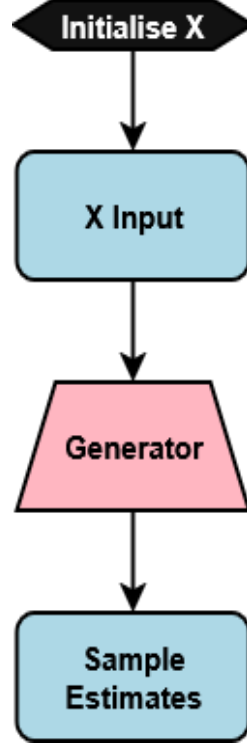


Figure 3.2: Sample Generation

3.2 Arguments (Objects) of the Generator

3.2.1 Input Matrix

$\mathbf{X} \in \mathbb{R}^{n \times p}$ is the input matrix to the generator

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} = \begin{bmatrix} \text{1st sample } (x_1, x_2, \cdots x_p) \\ \text{2nd sample } (x_1, x_2, \cdots x_p) \\ \vdots \\ \text{nth sample } (x_1, x_2, \cdots x_p) \end{bmatrix}.$$

This matrix represents the input noise samples which eventually get transformed into the synthetic samples of the target density. Each row of the input matrix is an observation or additional sample, and each column is a dimension of that sample. In the context of this paper, this matrix is populated with random noise samples from a standard normal distribution such that $x_{hj} \sim N(0, 1)$ for $h = 1, \dots, n$ and $j = 1, \dots, p$. Note that it is possible to work with either univariate x_h or multivariate $\mathbf{x}_h$, in which case the former would collapse this matrix down to a vector of length n .

3.2.2 Target Matrix

Let $\phi \in \mathbb{R}^{m \times 2}$ be the target matrix of the generator such that:

$$\phi = \begin{bmatrix} \phi_{11} & \phi_{12} \\ \phi_{21} & \phi_{22} \\ \vdots & \vdots \\ \phi_{m1} & \phi_{m2} \end{bmatrix} = \begin{bmatrix} \text{1st target } (\phi_1, \phi_2) \\ \text{2nd target } (\phi_1, \phi_2) \\ \vdots \\ \text{mth target } (\phi_1, \phi_2) \end{bmatrix}$$

Note that in most cases for generic feed-forward neural networks, the target matrix does not need to have the same dimensions as the input matrix. In the context of this paper however, the target matrix will always have dimension $m \times 2$ regardless of the dimension of the input matrix $\mathbf{X}$ because the target matrix is the collection of real and imaginary true characteristic values evaluated at $\mathbf{t}_j$, $j = 1, \dots, m$. It has nothing to do with the inputs. Specifically

$$\phi(\mathbf{t}_j) = [\Re(\phi(\mathbf{t}_j)) \quad \Im(\phi(\mathbf{t}_j))]_{1 \times 2}$$

or

$$\phi(\mathbf{T}) = \begin{bmatrix} \Re(\phi(\mathbf{t}_1)) & \Im(\phi(\mathbf{t}_1)) \\ \vdots & \vdots \\ \Re(\phi(\mathbf{t}_m)) & \Im(\phi(\mathbf{t}_m)) \end{bmatrix} = \begin{bmatrix} \phi_{11} & \phi_{12} \\ \vdots & \vdots \\ \phi_{m1} & \phi_{m2} \end{bmatrix}_{m \times 2}.$$

The reason why this matrix is of dimension $m \times 2$ is because it is the characteristic function evaluated at m frequency points split into 2 parts for the real and imaginary component. In the univariate case, the characteristic function is evaluated at particular scalar values t for which there are m of them, that is t_j for $j = 1, \dots, m$. In the multivariate case, the characteristic function is evaluated at particular vector values $\mathbf{t}$ for which there are m of them, that is $\mathbf{t}_j$ for $j = 1, \dots, m$. This is the same idea as evaluating any polynomial point wise by plugging in m values from the domain.

The choice of ϕ for the notation of this matrix is deliberate. It is not a true label for the last layer output $\mathbf{Y}$ as would be found in a standard neural network. The generator works by comparing a function $\hat{\phi}$ of the last layer of the neural network which estimates $\hat{\mathbf{Y}}$ which is then compared to the target ϕ , that is $\|\phi - \hat{\phi}\|^2$, and not $\|\mathbf{Y} - \hat{\mathbf{Y}}\|^2$ as per “standard” neural networks.

3.2.3 Weight Matrices

Let $\mathbf{W} \in \mathbb{R}^{\alpha \times \beta}$, with initialisation rule, $w_{ij} \sim \text{Uni}(-1, 1)$ for $i = 1, \dots, \alpha$ and $j = 1, \dots, \beta$ be a weight matrix of the generator,

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1\beta} \\ w_{21} & w_{22} & \cdots & w_{2\beta} \\ \vdots & \vdots & \ddots & \vdots \\ w_{\alpha 1} & w_{\alpha 2} & \cdots & w_{\alpha \beta} \end{bmatrix} = \begin{bmatrix} \text{1st neuron weights in the layer} \\ \text{2nd neuron weights in the layer} \\ \vdots \\ \alpha\text{th neuron weights in the layer} \end{bmatrix}.$$

The dimensions of the weight matrix correspond to the number of neurons in the current layer (α) and in the previous layer (β). These α, β parameters are unique per weight matrix. What this means is that each weight matrix in the network would have subscript k for being in the k^{th} layer, that is $\mathbf{W}_k$ is the weight matrix in the k^{th} layer. $\mathbf{W}_k$ will have α, β parameters which are different to those of $\mathbf{W}_{k-1}$ which will have its own α, β parameters. The choice of this notation is used to avoid subscripts on the α, β parameters. For example, if $\mathbf{W}_1$ is of dimension $\alpha_1 \times (\beta_1 = p)$, then $\mathbf{W}_2$ is of dimension $(\alpha_2 = p) \times \beta_2$.

There are various approaches to choosing how these weights are pre-populated as mentioned in Le, Jaitly, & Hinton (2015) and Glorot & Bengio (2010), such as drawing $w_{ij} \sim \text{Uni}\left(-\sqrt{\frac{6}{\beta}}, \sqrt{\frac{6}{\beta}}\right)$ for ReLU activation functions or $w_{ij} \sim \text{Uni}\left(-\sqrt{\frac{6}{\beta+\alpha}}, \sqrt{\frac{6}{\beta+\alpha}}\right)$ for sigmoid or tanh activation functions.

These choices of initial weights aim to address the vanishing gradient phenomenon, where in deeper networks, the gradient which is back propagated through the network becomes negligibly small. As a result the layers in the beginning of the network effectively stop ‘learning’. This can be thought of as dropping a stone into a large body of water, you would see local ripples where the stone has broken the water surface, but by the time those ripples have gotten to the coast, they are negligible if noticeable at all (H. Lee, Kim, Yang, & Choi, 2024).

3.2.4 Bias Vectors

Let $\mathbf{b} \in \mathbb{R}^\alpha$, with initialisation rule, $b_i = 0$ for all $i = 1, \dots, \alpha$ be a bias vector of the generator,

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_\alpha \end{bmatrix} = \begin{bmatrix} \text{bias for neuron 1 in the layer} \\ \text{bias for neuron 2 in the layer} \\ \vdots \\ \text{bias for neuron } \alpha \text{ in the layer} \end{bmatrix}.$$

Note that the bias terms are initialised to zero since assigning random values to them can increase variation and slow convergence. The reason this happens is because randomly assigning bias values is akin to shifting the activation boundary of a particular neuron arbitrarily. The variable of interest, $\mathbf{X}$, only interacts with the weights $\mathbf{W}$ via a product relation. As such, randomly assigning the w_{ij} is useful for breaking symmetry so that the neurons in a layer do not learn the same rule (Steinwart, 2019).

Symmetry specifically refers to when neurons receive the same gradient update and become indistinguishable in a layer. The consequence of this is redundancy of neurons in the layer and slowed learning via an inability to represent diverse features of the target function. The b_i do not influence this concept of symmetry hence do not require random assignment (Fok, An, & Wang, 2017).

Collecting $\mathbf{X}$, $\mathbf{W}$, and $\mathbf{b}$ into a linear expression is only possible if the bias vector is broadcasted row-wise n times for n inputs. This is to enable conformable operations. Let $\mathbf{B} \in \mathbb{R}^{\alpha \times n}$ be a broadcasted bias vector of the generator such that

$$\mathbf{B} = \begin{bmatrix} b_1 & b_1 & \cdots & b_1 \\ b_2 & b_2 & \cdots & b_2 \\ \vdots & & & \\ b_\alpha & b_\alpha & \cdots & b_\alpha \end{bmatrix} = \begin{bmatrix} \text{broadcasted bias for neuron 1 in the layer} \\ \text{broadcasted bias for neuron 2 in the layer} \\ \vdots \\ \text{broadcasted bias for neuron } \alpha \text{ in the layer} \end{bmatrix}.$$

3.2.5 Linear Pre-activation Function (Accumulator)

It is possible to compose a linear expression denoted $\mathbf{Z} \in \mathbb{R}^{\alpha \times n}$ for any arbitrary layer, k , of the network which relates the inputs to associated weights and biases of that layer. For the first layer, $k = 1$, and hence $\beta = p$ since β is the number of neurons in the previous layer, and we treat the input as a previous layer. The output, $\mathbf{Z}_1$, of this layer is

$$\begin{aligned} \mathbf{Z}_1 &= \mathbf{W}_1 \mathbf{X}' + \mathbf{B}_1 \\ &= \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1p} \\ w_{21} & w_{22} & \cdots & w_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ w_{\alpha 1} & w_{\alpha 2} & \cdots & w_{\alpha p} \end{bmatrix} \begin{bmatrix} x_{11} & x_{21} & \cdots & x_{n1} \\ x_{12} & x_{22} & \cdots & x_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ x_{1p} & x_{2p} & \cdots & x_{np} \end{bmatrix} + \begin{bmatrix} b_1 & b_1 & \cdots & b_1 \\ b_2 & b_2 & \cdots & b_2 \\ \vdots & & & \\ b_\alpha & b_\alpha & \cdots & b_\alpha \end{bmatrix} \\ &= \begin{bmatrix} z_{\text{neuron 1, sample 1}} & z_{\text{neuron 1, sample 2}} & \cdots & z_{\text{neuron 1, sample n}} \\ z_{\text{neuron 2, sample 1}} & z_{\text{neuron 2, sample 2}} & \cdots & z_{\text{neuron 2, sample n}} \\ \vdots & \vdots & \ddots & \vdots \\ z_{\text{neuron } \alpha, \text{ sample 1}} & z_{\text{neuron } \alpha, \text{ sample 2}} & \cdots & z_{\text{neuron } \alpha, \text{ sample n}} \end{bmatrix} \end{aligned}$$

The transpose of this matrix is

$$\mathbf{Z}'_1 = \begin{bmatrix} z_{\text{neuron 1, sample 1}} & z_{\text{neuron 2, sample 1}} & \cdots & z_{\text{neuron } \alpha, \text{ sample 1}} \\ z_{\text{neuron 1, sample 2}} & z_{\text{neuron 2, sample 2}} & \cdots & z_{\text{neuron } \alpha, \text{ sample 2}} \\ \vdots & \vdots & \ddots & \vdots \\ z_{\text{neuron 1, sample n}} & z_{\text{neuron 2, sample n}} & \cdots & z_{\text{neuron } \alpha, \text{ sample n}} \end{bmatrix}.$$

The transpose is returned because there is a single layer function which gets called recursively to compute each pre-activated linear combination $\mathbf{Z}$ of a particular layer. This results in the linear output being the same format as the input matrix,

$$\mathbf{Z}'_1 = \begin{bmatrix} \text{1st sample } (z_1, z_2, \cdots z_\alpha) \\ \text{2nd sample } (z_1, z_2, \cdots z_\alpha) \\ \vdots \\ \text{nth sample } (z_1, z_2, \cdots z_\alpha) \end{bmatrix}.$$

3.2.6 Post-activation Function (Layer Output)

The activation function, $f(\cdot)$, is applied element-wise to the linear output of each layer to provide the final result of that layer. In layer 1, $f(\mathbf{Z}'_1) = \hat{\mathbf{Y}}_1$ where

$$\begin{aligned}
\hat{\mathbf{Y}}_1 &= \begin{bmatrix} f(z_{\text{neuron } 1, \text{ sample } 1}) & f(z_{\text{neuron } 2, \text{ sample } 1}) & \cdots & f(z_{\text{neuron } \alpha, \text{ sample } 1}) \\ f(z_{\text{neuron } 1, \text{ sample } 2}) & f(z_{\text{neuron } 2, \text{ sample } 2}) & \cdots & f(z_{\text{neuron } \alpha, \text{ sample } 2}) \\ \vdots & \vdots & \ddots & \vdots \\ f(z_{\text{neuron } 1, \text{ sample } n}) & f(z_{\text{neuron } 2, \text{ sample } n}) & \cdots & f(z_{\text{neuron } \alpha, \text{ sample } n}) \end{bmatrix} \\
&= \begin{bmatrix} \text{1st sample } (\hat{y}_{11}, \hat{y}_{12}, \cdots \hat{y}_{1\alpha}) \\ \text{2nd sample } (\hat{y}_{21}, \hat{y}_{22}, \cdots \hat{y}_{2\alpha}) \\ \vdots \\ \text{nth sample } (\hat{y}_{n1}, \hat{y}_{n2}, \cdots \hat{y}_{n\alpha}) \end{bmatrix}.
\end{aligned}$$

$\hat{\mathbf{Y}}_1$ is now treated as the new input for the next layer ($k = 2$), hence the transpose in the accumulator step. The above process can be repeated an arbitrary amount of times where $\hat{\mathbf{Y}}_k$ is the new input for layer $k + 1$. Each layer can be expanded (adding neurons) or contracted (reducing neurons) by manipulating the dimensions of the weight matrix and bias vector accordingly.

Consider the following example of a generator of depth k

$$\begin{aligned}
\mathbf{Z}_1 &= \mathbf{W}_1 \mathbf{X}' + \mathbf{B}_1 \rightarrow f_1(\mathbf{Z}'_1) = \hat{\mathbf{Y}}_1, \\
\mathbf{Z}_2 &= \mathbf{W}_2 \hat{\mathbf{Y}}'_1 + \mathbf{B}_2 \rightarrow f_2(\mathbf{Z}'_2) = \hat{\mathbf{Y}}_2, \\
&\vdots \\
\mathbf{Z}_k &= \mathbf{W}_k \hat{\mathbf{Y}}'_{k-1} + \mathbf{B}_k \rightarrow f_k(\mathbf{Z}'_k) = \hat{\mathbf{Y}}_k.
\end{aligned}$$

Or equivalently

$$\hat{\mathbf{Y}}_k = f_k(f_{k-1}(\cdots f_1(\mathbf{X}\mathbf{W}'_1 + \mathbf{B}'_1) \cdots \mathbf{W}'_{k-1} + \mathbf{B}'_{k-1})\mathbf{W}'_k + \mathbf{B}'_k) \quad (3.2)$$

This is the logic of the generator's forward pass. Equation (3.2) highlights the fundamental structure of a neural network as being a single composite function with many degrees of freedom, where degrees of freedom refers to the large amount of effective, trainable parameters (weights and biases) that are free to adjust to the data. A large amount of parameters gives the network flexibility in approximating different types of functions. However more parameters do not necessarily imply a better model, since there is a risk of over fitting, generalisation error, and increased computation time the wider and deeper a network becomes (T. Gao & Jojic, 2016; James, Witten, Hastie, & Tibshirani, 2013; Salman & Liu, 2019).

The reason why the term effective is important in the context of network degrees of freedom is because it is a combination of the parameters which influences network outputs, not necessarily each and every weight and bias neuron. This means that the true degrees of freedom can often be much lower than the actual amount of network parameters (T. Gao & Jojic, 2016).

3.2.7 Empirical Characteristic Function Computation

The last step of the forward pass turns the network into a generator. Consider the constraint of dimensionality on the final layer of the network. In the paper by Brück (2025), the column dimension of the noise inputs $\mathbf{X}$ does not need to match that of the last layer output $\hat{\mathbf{Y}}_k$. As $\hat{\mathbf{Y}}_k$ a function of the network output $\hat{\phi}_{\hat{\mathbf{Y}}_k}(\mathbf{T})$ which is compared in the loss. Thus the only constraint is that the column dimension p of the estimates $\hat{\mathbf{Y}}_k$ match that of the frequencies $\mathbf{T}$. There is no column dimension specific condition on $\mathbf{X}$, what this means is that the column dimension d of the noise inputs $\mathbf{X}$ can be greater than that of $\hat{\mathbf{Y}}_k$ so long as it is transformed to size p by the end of the forward pass by reducing the number of neurons in each subsequent layer.

This would be done under the assumption that, up to a point, increasing the input dimension could enable the generator more flexibility to find patterns in the data (Hu et al., 2023). The point of this realisation is that it is now known that the output layer of the generator needs to have as many neurons as there are dimensions of $\mathbf{T}$ in $\mathbb{R}^p$. For the purposes of this paper, no transformation from dimension d to p takes place, the input is already set to be of dimension p . This is because the transformation will manifest into an extra dimension needing to be accounted for in the matrix operations within the back propagation algorithm which is avoided for simplicity. The condition still holds however, the only difference is that the column dimension of the input $\mathbf{X}$ has to be the same as the column dimension of $\mathbf{T}$.

The final layer of the generator produces the output matrix $\hat{\mathbf{Y}}_k \in \mathbb{R}^{n \times p}$, and substituting this into equation (2.14) yields

$$\hat{\phi}_{\hat{\mathbf{Y}}_k}(\mathbf{T}) := \hat{\phi}(\mathbf{T}) = \begin{bmatrix} \frac{1}{n} \cos(\mathbf{T} \hat{\mathbf{Y}}_k') \mathbf{1} & \frac{1}{n} \sin(\mathbf{T} \hat{\mathbf{Y}}_k') \mathbf{1} \end{bmatrix}_{m \times 2}.$$

This quantity is now compared with the target matrix $\phi(\mathbf{T})$ and the error is back-propagated through the network so that it can learn the characteristic function of the target distribution.

Example

The following graph depicts an FCFMG 3 layers deep which takes in noise of dimension p and outputs synthetic samples of dimension p . To avoid clutter, this graph depicts the structure for a single sample h of the $\mathbf{X}$ matrix, the h^{th} row of matrix $\mathbf{X}$. This graph applies for $h = 1, \dots, n$ samples.

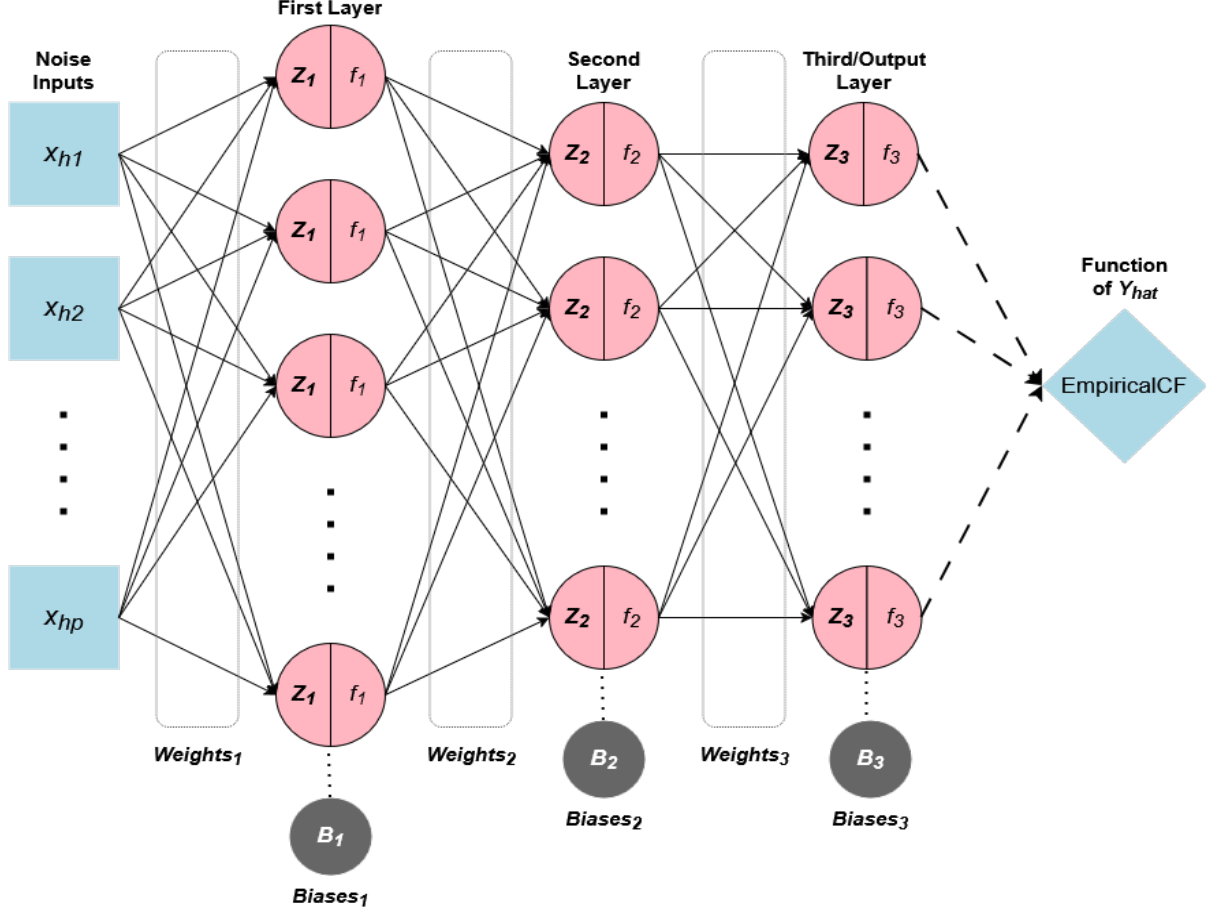


Figure 3.3: Example FCFMG Graph

The third/output layer of this example produces $\hat{\mathbf{Y}}_3$, the estimated samples which get used to compute the empirical characteristic function in the EmpiricalCF collector, that is $\hat{\phi}_{\hat{\mathbf{Y}}_3}$. Since the diagram is in two dimensions, the third/output layer is also passing a single sample of $\hat{\mathbf{Y}}_3$ along with the input, but mathematically the entire matrix is passed at once.

3.3 Loss Function

The generator of this paper uses a squared error loss function, given by

$$L = \sum_{j=1}^m \|\phi(\mathbf{t}_j) - \hat{\phi}(\mathbf{t}_j)\|_F^2 = \|\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T})\|_F^2.$$

or equivalently

$$\begin{aligned} L &= \text{tr}[(\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T}))'(\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T}))] \\ &= \text{tr}[(\phi(\mathbf{T})' - \hat{\phi}(\mathbf{T})')(\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T}))] \\ &= \text{tr}[\phi(\mathbf{T})'\phi(\mathbf{T}) - 2\hat{\phi}(\mathbf{T})'\phi(\mathbf{T}) + \hat{\phi}(\mathbf{T})'\hat{\phi}(\mathbf{T})] \end{aligned}$$

The subscript F on the norm expression explicitly states that this is the Frobenious norm, the euclidean norm but applied to matrices. If $\mathbf{A}$ is an arbitrary matrix, then $\|\mathbf{A}\|_F = \sqrt{\text{tr}[\mathbf{A}'\mathbf{A}]}$ (Magnus & Neudecker, 1999). The Frobenious norm is used because $\phi(T) - \hat{\phi}(T)$ is an $m \times 2$ matrix of real and imaginary parts. If both characteristic functions were complex valued, then the trace would not be necessary since $\phi(T) - \hat{\phi}(T)$ would be an $m \times 1$ vector. What this means is that the expression $(\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T}))'(\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T}))$ is a matrix of size 2×2 and not a scalar. The interpretation of this matrix is as follows

$$(\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T}))'(\phi(\mathbf{T}) - \hat{\phi}(\mathbf{T})) = \begin{bmatrix} \text{Total real error} & \text{Cor}[\text{Re}(\text{error}), \text{Im}(\text{error})] \\ \text{Cor}[\text{Re}(\text{error}), \text{Im}(\text{error})] & \text{Total imaginary error} \end{bmatrix}$$

Hence it is necessary to take the trace of this matrix to obtain the total error of both the real and imaginary components. The squared difference between the network estimates/output $\hat{\mathbf{Y}}_k$ and the target variable would normally be computed in most feed-forward neural networks. However, the FCFMG computes the squared difference between the empirical characteristic function $\hat{\phi}(\mathbf{T})$, which is a function of $\hat{\mathbf{Y}}_k$, and the aforementioned true characteristic function $\phi(\mathbf{T})$. This influences how the back propagation derivatives are derived.

3.4 The Optimising Algorithm

Once a forward pass has been completed and the loss function has been computed, it becomes necessary to adjust the weights and biases to reflect what was obtained in the output. Gradient descent is used to do this. The gradient descent algorithm seeks to find minimums by taking steps in the direction of quickest descent (Ruder, 2016). An analogy would be a person standing on a hill and seeking a way to get down. A reasonable approach would be to feel where the steepness is the greatest, and proceeding to move in that direction.

The algorithm can be presented by letting $\Omega = \{\omega_1, \omega_2, \dots, \omega_p\}$ denoting a set of parameters, $\eta \in \mathbb{R}$ denoting the step size, $L(\cdot)$ denoting the loss function that needs to be minimised, and $\nabla L(\Omega) = \left[\frac{\partial L}{\partial \omega_1}, \frac{\partial L}{\partial \omega_2}, \dots, \frac{\partial L}{\partial \omega_p} \right]'$ denoting the gradient of the loss function.

Update from Ω^{old} to Ω^{new} by:

$$\Omega^{new} = \Omega^{old} - \eta \nabla L(\Omega^{old}) \quad (3.3)$$

This is the classic or batch gradient descent method (J. D. Lee, Simchowitz, Jordan, & Recht, 2016; Ruder, 2016).

The loss L is a function which when evaluated at given points, provides information on how bad the estimates are compared to the target. Consider again the generator of length k . The goal is to find $\nabla L(\Omega)$ where $\Omega = \{\omega_1, \omega_2, \dots, \omega_p\} = \{\mathbf{W}_1, \dots, \mathbf{W}_k, \mathbf{B}_1, \dots, \mathbf{b}_k\}'$ are the parameters of the generator. Once a forward pass has been completed, the gradient of the loss function is sought with respect to these weights and biases.

3.5 Back Propagation

3.5.1 General Algorithm

Back propagation is the process which enables the parameters of the generator, namely the weights and biases, to be learnt, that is to be estimated based on the data (Kuninti & Rooban, 2021). After the forward pass, the total loss is computed using the loss function. The loss function is an indicator of how wrong the generator is by computing the error between the empirical characteristic function in equation (2.14) and the true characteristic function (2.1). The generator is able to update all of the parameters in the network by passing information about this error back through all of the parameters in each and every layer, using the gradient descent algorithm and the chain rule.

During back propagation, the partial derivative of the loss with respect to the last, that is the most recent layer, parameters is calculated. This quantity computes the amount by which the parameters in the current layer should adjust in order to minimise the loss (Cheng, 2021). This is analogous to stepping down the hill. In a multi-layer network in which this generator is a type, the partial derivative of the loss with respect to the second last layer must also be calculated, and so on until the first layer is reached. Every layer must have an opportunity to have its parameters adjusted so that the entire network can find the minimum, that is the bottom of the hill.

The chain rule arises when the form of the network in expression (3.2) is considered. In order to obtain the partial derivatives of previous layer results which are themselves functions of their own previous layers, it becomes necessary to invoke the chain rule. The following derivation of the back propagation algorithm is supplied by Haykin (2009).

Consider the weights:

$$\begin{aligned}
 & \text{Start at the last layer and work backwards} \\
 & \frac{\partial L}{\partial \mathbf{W}_k} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \mathbf{W}_k}, \\
 & \text{back propagate to the previous layer} \\
 & \frac{\partial L}{\partial \mathbf{W}_{k-1}} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{W}_{k-1}}, \\
 & \text{continue until the input as the last layer} \\
 & = \dots \\
 & \frac{\partial L}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \hat{\mathbf{Y}}_{k-2}} \dots \frac{\partial \mathbf{Z}_1}{\partial \mathbf{W}_1}.
 \end{aligned} \tag{3.4}$$

Consider the biases:

Start at the last layer and work backwards

$$\frac{\partial L}{\partial \mathbf{B}_k} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \mathbf{B}_k},$$

back propagate to the previous layer

$$\frac{\partial L}{\partial \mathbf{B}_{k-1}} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{B}_{k-1}}, \quad (3.5)$$

continue until the input as the last layer

$$\begin{aligned} &= \dots \\ \frac{\partial L}{\partial \mathbf{B}_1} &= \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \hat{\mathbf{Y}}_{k-2}} \dots \frac{\partial \mathbf{Z}_1}{\partial \mathbf{B}_1}. \end{aligned}$$

The derivatives in subsequent layers need the previous layer's derivative via the chain rule. To avoid having to define a unique derivative for each parameter, an absorbing term can be introduced to capture the information of the previous derivative. Call this absorbing term δ . Then the back propagating procedure can be re-written in terms of δ_k .

Let

$$\delta_k = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} = \frac{\partial L}{\partial \mathbf{Z}_k}, \quad (3.6)$$

then

$$\delta_{k-1} = \frac{\partial L}{\partial \mathbf{Z}_{k-1}} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} = \delta_k \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}}. \quad (3.7)$$

The equations in (3.4) become

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{W}_k} &= \delta_k \cdot \frac{\partial \mathbf{Z}_k}{\partial \mathbf{W}_k}, \\ \frac{\partial L}{\partial \mathbf{W}_{k-1}} &= \delta_{k-1} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{W}_{k-1}}, \\ &= \dots \\ \frac{\partial L}{\partial \mathbf{W}_1} &= \delta_1 \cdot \frac{\partial \mathbf{Z}_1}{\partial \mathbf{W}_1}, \end{aligned}$$

and the equations in (3.5) become

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{B}_k} &= \delta_k \cdot \frac{\partial \mathbf{Z}_k}{\partial \mathbf{B}_k}, \\ \frac{\partial L}{\partial \mathbf{B}_{k-1}} &= \delta_{k-1} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{B}_{k-1}}, \\ &= \dots \\ \frac{\partial L}{\partial \mathbf{B}_1} &= \delta_1 \cdot \frac{\partial \mathbf{Z}_1}{\partial \mathbf{B}_1}. \end{aligned}$$

This result is called the delta rule and make it possible to use the gradient descent equation (3.3) and the above derivatives to iteratively update the weights and biases of the generator (Haykin, 2009; Huk, 2012). This is how the FCFMG “learns”:

$$\begin{aligned}\mathbf{W}_k^{new} &:= \mathbf{W}_k^{old} - \eta \frac{\partial L}{\partial \mathbf{W}_k^{old}}, \\ \mathbf{B}_k^{new} &:= \mathbf{B}_k^{old} - \eta \frac{\partial L}{\partial \mathbf{B}_k^{old}}.\end{aligned}$$

3.5.2 Deriving the Generator Derivatives

Output Layer Derivatives

The goal is to find

$$\frac{\partial L}{\partial \mathbf{W}_k} \quad \text{and} \quad \frac{\partial L}{\partial \mathbf{B}_k}.$$

For ease of readability, define $\phi(\mathbf{T}) := \phi$ and $\hat{\phi}(\mathbf{T}) := \hat{\phi}$, then the partial derivative of the loss function with respect to the empirical characteristic function is

$$\begin{aligned}\frac{\partial L}{\partial \hat{\phi}} &= \frac{\partial}{\partial \hat{\phi}} \left(\text{tr}[(\phi - \hat{\phi})'(\phi - \hat{\phi})] \right) \\ &= \frac{\partial}{\partial \hat{\phi}} \left(\text{tr}[(\phi' \phi - \phi' \hat{\phi} - \hat{\phi}' \phi + \hat{\phi}' \hat{\phi})] \right) \\ &= \frac{\partial}{\partial \hat{\phi}} \left(\text{tr}[\phi' \phi] - \text{tr}[\phi' \hat{\phi}] - \text{tr}[\hat{\phi}' \phi] + \text{tr}[\hat{\phi}' \hat{\phi}] \right) \\ &= \frac{\partial}{\partial \hat{\phi}} \left(\text{tr}[\phi' \phi] - 2\text{tr}[\hat{\phi}' \phi] + \text{tr}[\hat{\phi}' \hat{\phi}] \right) \quad \text{since } \text{tr}[\phi' \hat{\phi}] = \text{tr}[\hat{\phi}' \phi] \\ &= -2\phi + 2\hat{\phi} \quad \text{since } \frac{\partial}{\partial \hat{\phi}} \left(\text{tr}[\hat{\phi}' \phi] \right) = \frac{\partial}{\partial \hat{\phi}} \left(\text{tr}[\phi' \hat{\phi}] \right) = \phi \quad \text{and} \quad \frac{\partial}{\partial \hat{\phi}} \left(\text{tr}[\hat{\phi}' \hat{\phi}] \right) = 2\hat{\phi} \\ &= 2(\hat{\phi} - \phi)_{m \times 2}.\end{aligned}$$

Recall from the definition of the empirical characteristic function in (2.14) that the dimension of $\hat{\phi}$ is an $m \times 2$ matrix since there are m values at which the characteristic function is evaluated, and two columns, one for the real part and the other for the imaginary part. From Magnus & Neudecker (1999) and Nguyen (2025), the derivative of a scalar function of a matrix variable has the same dimension as the matrix variable, that is $g : \mathbb{R}^{n \times p} \rightarrow \mathbb{R}$ and $\mathbf{X} \in \mathbb{R}^{n \times p}$, then $\frac{\partial g}{\partial \mathbf{X}} \in \mathbb{R}^{n \times p}$. This result has the form

$$\frac{\partial g}{\partial \mathbf{X}} = \begin{bmatrix} \frac{\partial g}{\partial x_{11}} & \cdots & \frac{\partial g}{\partial x_{1p}} \\ \vdots & & \vdots \\ \frac{\partial g}{\partial x_{n1}} & \cdots & \frac{\partial g}{\partial x_{np}} \end{bmatrix}, \quad (3.8)$$

and considering that the loss function L is a scalar function of a matrix variable $\hat{\phi}$, means that $L : \mathbb{R}^{m \times 2} \rightarrow \mathbb{R}$ and $\hat{\phi} \in \mathbb{R}^{m \times 2}$ which implies $\frac{\partial L}{\partial \hat{\phi}} \in \mathbb{R}^{m \times 2}$.

The next derivative is non-trivial to solve, and is given by

$$\frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k}.$$

This object is a 4^{th} order tensor of dimension $m \times 2 \times n \times p$, because $\hat{\phi}$ is an $m \times 2$ matrix and $\hat{\mathbf{Y}}_k$ is an $n \times p$ matrix. For each entry of $\hat{\phi}$, the derivative with respect to $\hat{\mathbf{Y}}_k$ is computed, that is for each $\hat{\phi}_{ij}$, the definition in (3.8) applies, but this should not be seen as a block matrix (Magnus & Neudecker, 1999). Since the tensor exists in high dimensional space, it cannot be written down algebraically like a matrix without manipulating it in a way so as to take advantage of the normal matrix and vector relations. It is necessary to invoke the $\text{vec}()$ operator (Magnus & Neudecker, 1999). The $\text{vec}()$ operator takes all of the columns of a matrix and stacks them on top of each other. In so doing, this ‘flattens’ a high dimensional object. Consider an arbitrary matrix $\mathbf{A} \in \mathbb{R}^{n \times p}$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{np} \end{bmatrix}_{n \times p}.$$

Then

$$\text{vec}(\mathbf{A}) = \begin{bmatrix} a_{11} \\ a_{12} \\ \vdots \\ a_{1p} \\ \vdots \\ a_{n1} \\ a_{n2} \\ \vdots \\ a_{np} \end{bmatrix}_{np \times 1}.$$

For the purposes of clean notation, denote $\hat{\phi} \in \mathbb{R}^{m \times 2}$ where $\hat{\phi}_{11}$ is the first element of $\hat{\phi}$, $\hat{\mathbf{Y}}_k := \hat{\mathbf{Y}} \in \mathbb{R}^{n \times p}$ where $\hat{y}_{11} \in \mathbb{R}$ is the first element of $\hat{\mathbf{Y}}$, $\mathbf{1} \in \mathbb{R}^n$ is a vector of 1’s, and $\mathbf{T} \in \mathbb{R}^{m \times p}$ is a matrix of frequencies $t_{jl} \in \mathbb{R}$ for $j = 1, \dots, m$ and $l = 1, \dots, p$.

Then consider

$$\frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}} \in \mathbb{R}^{m \times 2 \times n \times p}.$$

Using the $\text{vec}()$ operator, this tensor can be flattened into the matrix

$$\frac{\partial \text{vec}(\hat{\phi})}{\partial \text{vec}(\hat{\mathbf{Y}})} \in \mathbb{R}^{2m \times np}.$$

Writing out the full matrix of this expression and making use of $\hat{\mathbf{y}}_l = [\hat{y}_{l1}, \dots, \hat{y}_{lp}]' \in \mathbb{R}^p$ as the l^{th} row of $\hat{\mathbf{Y}}$ yields

$$\frac{\partial \text{vec}(\hat{\phi})}{\partial \text{vec}(\hat{\mathbf{Y}})} = \begin{bmatrix} \frac{\partial \hat{\phi}_{11}}{\partial \hat{y}_{11}} & \frac{\partial \hat{\phi}_{11}}{\partial \hat{y}_{12}} & \dots & \frac{\partial \hat{\phi}_{11}}{\partial \hat{y}_{1p}} \\ \frac{\partial \hat{\phi}_{12}}{\partial \hat{y}_{11}} & \frac{\partial \hat{\phi}_{12}}{\partial \hat{y}_{12}} & \dots & \frac{\partial \hat{\phi}_{12}}{\partial \hat{y}_{1p}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \hat{\phi}_{m1}}{\partial \hat{y}_{11}} & \frac{\partial \hat{\phi}_{m1}}{\partial \hat{y}_{12}} & \dots & \frac{\partial \hat{\phi}_{m1}}{\partial \hat{y}_{1p}} \\ \frac{\partial \hat{\phi}_{m2}}{\partial \hat{y}_{11}} & \frac{\partial \hat{\phi}_{m2}}{\partial \hat{y}_{12}} & \dots & \frac{\partial \hat{\phi}_{m2}}{\partial \hat{y}_{1p}} \end{bmatrix} = \begin{bmatrix} \frac{\partial \hat{\phi}_{11}}{\partial \hat{\mathbf{y}}_1} & \dots & \frac{\partial \hat{\phi}_{11}}{\partial \hat{\mathbf{y}}_n} \\ \frac{\partial \hat{\phi}_{12}}{\partial \hat{\mathbf{y}}_1} & \dots & \frac{\partial \hat{\phi}_{12}}{\partial \hat{\mathbf{y}}_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial \hat{\phi}_{m1}}{\partial \hat{\mathbf{y}}_1} & \dots & \frac{\partial \hat{\phi}_{m1}}{\partial \hat{\mathbf{y}}_n} \\ \frac{\partial \hat{\phi}_{m2}}{\partial \hat{\mathbf{y}}_1} & \dots & \frac{\partial \hat{\phi}_{m2}}{\partial \hat{\mathbf{y}}_n} \end{bmatrix}_{2m \times np}.$$

This matrix can be interpreted as finding every derivative of each element of $\hat{\phi}$ with respect to every element of $\hat{\mathbf{Y}}$. The form of this matrix becomes clear when treating $\text{vec}(\hat{\phi})$ and $\text{vec}(\hat{\mathbf{Y}})$ as two generic vectors $\mathbf{v} \in \mathbb{R}^m$ and $\mathbf{u} \in \mathbb{R}^n$, where the derivative of $\mathbf{v}$ with respect to $\mathbf{u}$ produces a matrix of the form

$$\frac{\partial \mathbf{v}}{\partial \mathbf{u}} = \begin{bmatrix} \frac{\partial v_1}{\partial u_1} & \frac{\partial v_1}{\partial u_2} & \dots & \frac{\partial v_1}{\partial u_n} \\ \frac{\partial v_2}{\partial u_1} & \frac{\partial v_2}{\partial u_2} & \dots & \frac{\partial v_2}{\partial u_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial v_m}{\partial u_1} & \frac{\partial v_m}{\partial u_2} & \dots & \frac{\partial v_m}{\partial u_n} \end{bmatrix}.$$

The only subtlety which makes the first matrix look strange is due to a preference to keep the indexing of the original matrices for $\hat{\phi}$ and $\hat{\mathbf{Y}}$ intact since these affect the element wise applied functions of sine and cosine.

Recall from equation (2.14)

$$\hat{\phi} = f(\hat{\mathbf{Y}}) = \hat{\phi}_{\hat{\mathbf{Y}}}(\mathbf{T}) = \left[\frac{1}{n} \cos(\mathbf{T}\hat{\mathbf{Y}}') \mathbf{1} \quad \frac{1}{n} \sin(\mathbf{T}\hat{\mathbf{Y}}') \mathbf{1} \right]_{m \times 2},$$

where

$$\mathbf{T}\hat{\mathbf{Y}}' = \begin{bmatrix} t_{11} & t_{12} & \dots & t_{1p} \\ t_{21} & t_{22} & \dots & t_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ t_{m1} & t_{m2} & \dots & t_{mp} \end{bmatrix}_{m \times p} \begin{bmatrix} \hat{y}_{11} & \hat{y}_{21} & \dots & \hat{y}_{n1} \\ \hat{y}_{12} & \hat{y}_{22} & \dots & \hat{y}_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{y}_{1p} & \hat{y}_{2p} & \dots & \hat{y}_{np} \end{bmatrix}_{p \times n} = \begin{bmatrix} \mathbf{t}'_1 \hat{\mathbf{y}}_1 & \dots & \mathbf{t}'_1 \hat{\mathbf{y}}_n \\ \vdots & \ddots & \vdots \\ \mathbf{t}'_m \hat{\mathbf{y}}_1 & \dots & \mathbf{t}'_m \hat{\mathbf{y}}_n \end{bmatrix}_{m \times n}.$$

Then in a similar fashion to the derivation of the multivariate empirical characteristic function, let f be a generic element-wise applied function such that

$$\frac{1}{n} f(\mathbf{T}\hat{\mathbf{Y}}') \mathbf{1} = \frac{1}{n} f \begin{bmatrix} \mathbf{t}'_1 \hat{\mathbf{y}}_1 & \dots & \mathbf{t}'_1 \hat{\mathbf{y}}_n \\ \vdots & \ddots & \vdots \\ \mathbf{t}'_m \hat{\mathbf{y}}_1 & \dots & \mathbf{t}'_m \hat{\mathbf{y}}_n \end{bmatrix} \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} = \frac{1}{n} \begin{bmatrix} f(\mathbf{t}'_1 \hat{\mathbf{y}}_1) + \dots + f(\mathbf{t}'_1 \hat{\mathbf{y}}_n) \\ \vdots \\ f(\mathbf{t}'_m \hat{\mathbf{y}}_1) + \dots + f(\mathbf{t}'_m \hat{\mathbf{y}}_n) \end{bmatrix} = \begin{bmatrix} \frac{1}{n} \sum_{h=1}^n f(\mathbf{t}'_1 \hat{\mathbf{y}}_h) \\ \vdots \\ \frac{1}{n} \sum_{h=1}^n f(\mathbf{t}'_m \hat{\mathbf{y}}_h) \end{bmatrix}_{m \times 1}.$$

The derivative of this quantity with respect to a particular $\hat{\mathbf{y}}_l$ is given by

$$\frac{\partial \frac{1}{n} f(\mathbf{T}\hat{\mathbf{Y}}') \mathbf{1}}{\partial \hat{\mathbf{y}}_l} = \begin{bmatrix} \frac{1}{n} \sum_{h=1}^n \frac{\partial}{\partial \hat{\mathbf{y}}_l} f(\mathbf{t}'_1 \hat{\mathbf{y}}_h) \\ \vdots \\ \frac{1}{n} \sum_{h=1}^n \frac{\partial}{\partial \hat{\mathbf{y}}_l} f(\mathbf{t}'_m \hat{\mathbf{y}}_h) \end{bmatrix} = \begin{bmatrix} \frac{1}{n} f'(\mathbf{t}'_1 \hat{\mathbf{y}}_l) \mathbf{t}_1 \\ \vdots \\ \frac{1}{n} f'(\mathbf{t}'_m \hat{\mathbf{y}}_l) \mathbf{t}_m \end{bmatrix}_{m \times 1}.$$

Using these results and considering that each column of $\hat{\phi}$ is either a sine or cosine element wise applied function, it is possible to denote

$$\hat{\phi}_{ij} = \begin{cases} \frac{1}{n} \sum_{h=1}^n \cos(\mathbf{t}'_i \hat{\mathbf{y}}_h) & \text{if } j = 1, \\ \frac{1}{n} \sum_{h=1}^n \sin(\mathbf{t}'_i \hat{\mathbf{y}}_h) & \text{if } j = 2. \end{cases}$$

The derivative of $\hat{\phi}_{ij}$ with respect to a particular $\hat{\mathbf{y}}_l$ being given by

$$\frac{\partial \hat{\phi}_{ij}}{\partial \hat{\mathbf{y}}_l} = \begin{cases} -\frac{1}{n} \sin(\mathbf{t}'_i \hat{\mathbf{y}}_l) \mathbf{t}_i & \text{if } j = 1, \\ \frac{1}{n} \cos(\mathbf{t}'_i \hat{\mathbf{y}}_l) \mathbf{t}_i & \text{if } j = 2. \end{cases}$$

As a result

$$\frac{\partial \text{vec}(\hat{\phi})}{\partial \text{vec}(\hat{\mathbf{Y}})} = \begin{bmatrix} -\frac{1}{n} \sin(\mathbf{t}'_1 \hat{\mathbf{y}}_1) \mathbf{t}_1 & \cdots & -\frac{1}{n} \sin(\mathbf{t}'_1 \hat{\mathbf{y}}_n) \mathbf{t}_1 \\ \frac{1}{n} \cos(\mathbf{t}'_1 \hat{\mathbf{y}}_1) \mathbf{t}_1 & \cdots & \frac{1}{n} \cos(\mathbf{t}'_1 \hat{\mathbf{y}}_n) \mathbf{t}_1 \\ \vdots & \ddots & \vdots \\ -\frac{1}{n} \sin(\mathbf{t}'_m \hat{\mathbf{y}}_1) \mathbf{t}_m & \cdots & -\frac{1}{n} \sin(\mathbf{t}'_m \hat{\mathbf{y}}_n) \mathbf{t}_m \\ \frac{1}{n} \cos(\mathbf{t}'_m \hat{\mathbf{y}}_1) \mathbf{t}_m & \cdots & \frac{1}{n} \cos(\mathbf{t}'_m \hat{\mathbf{y}}_n) \mathbf{t}_m \end{bmatrix}_{2m \times np}.$$

Each entry as depicted in this matrix is a $1 \times p$ row vector, totaling to a dimension of $2m \times np$. In code, this would be unpacked into a tensor of dimension $m \times 2 \times n \times p$. This would also require the other matrices to be broadcasted to be conformable in a similar way as was done with the bias vector.

Reverting back to using subscripts k , $k-1$, etc to indicate layer position, consider the next derivative of the chain and recall the form of the last layer output from section 3.2.6

$$\mathbf{Z}_k = \mathbf{W}_k \hat{\mathbf{Y}}'_{k-1} + \mathbf{B}_k \text{ which is transformed into } f_k(\mathbf{Z}'_k) = \hat{\mathbf{Y}}_k. \quad (3.9)$$

Then

$$\frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} = (f'_k(\mathbf{Z}'_k))' = f'_k(\mathbf{Z}_k),$$

where f'_k is the k^{th} activation function's derivative applied element wise to $\mathbf{Z}'_k$, which is the transposed linear combination, that is the accumulator function, of the weights and broadcasted biases. The $\text{vec}()$ operator is not needed in this case because the form of the expression is simple enough, just the element wise application of a function, to apply normal derivative rules.

The transpose is removed because it is the derivative with respect to $\mathbf{Z}_k$ not $\mathbf{Z}'_k$. Derivatives with respect to a matrix must be in the same format as that matrix, and to have

the derivative with respect to $\mathbf{Z}_k$, the result should be in the same format as $\mathbf{Z}_k$ (Nguyen, 2025). The only way to achieve this is to transpose the result. In the context of back propagation, this could be seen as needing the resulting derivatives to have the same shape corresponding to the previous layer's dimensions to be conformable. This idea can be seen when travelling backwards along the example diagram in 3.3, the weight matrices have to be conformable from right to left.

The last derivatives follow as

$$\frac{\partial \mathbf{Z}_k}{\partial \mathbf{W}_k} = \hat{\mathbf{Y}}_{k-1},$$

and

$$\frac{\partial \mathbf{Z}_k}{\partial \mathbf{B}_k} = \mathbf{I} \quad \text{where } \mathbf{I} \text{ is broadcasted.}$$

It is important to note that computing $\frac{\partial L}{\partial \mathbf{W}_k}$ and $\frac{\partial L}{\partial \mathbf{B}_k}$ is not possible as is because the dimensions of their components are not conformable. That is, $\frac{\partial L}{\partial \hat{\phi}}$ is $m \times 2$, $\frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k}$ is $m \times 2 \times n \times p$, and $\frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k}$ is $p \times n$. To solve this problem, broadcasting, transposition and a tensor contraction are used. The goal is to transform all of the objects into the same dimension, compute an element wise multiplication operation on them, then shrink the result down, an idea present in literature dealing with high dimensional data structures (X. Gao, Lu, & Liu, 2024). Tensor contraction works by summing over the shared dimensions of two or more different tensors and can be thought of as combining the information shared between them with the goal being to drop a dimension (Evenbly, 2022).

Denote the following:

- $2(\hat{\phi} - \phi)_{m \times 2}$ is broadcasted along the n (number of samples) and p (number of sample dimensions) dimensions to become $2(\hat{\phi} - \phi)_{m \times 2 \times n \times p}^*$.
- $\frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \in \mathbb{R}^{m \times 2 \times n \times p}$ is the 4th order tensor.
- $f'_k(\mathbf{Z}_k)_{p \times n}$ is the derivative of the layer output with respect to the accumulator function which is broadcasted along m (number of frequencies) and 2 (real and imaginary parts) and transposed to become $f'_k(\mathbf{Z}_k)_{m \times 2 \times n \times p}^*$.

Then define

$$Q := \left(2(\hat{\phi} - \phi)^* \circ \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \circ f'_k(\mathbf{Z}_k)^* \right) \in \mathbb{R}^{m \times 2 \times n \times p}. \quad (3.10)$$

Applying a contraction along the first two dimensions of Q by summing across the $i = 1, \dots, m$ frequency sample dimension and the real and imaginary part dimension $j = 1, 2$ transforms it from being a $m \times 2 \times n \times p$ tensor to a $n \times p$ matrix. That is

$$Q \in \mathbb{R}^{m \times 2 \times n \times p} \text{ via contraction becomes } Q_C \in \mathbb{R}^{n \times p}.$$

Putting everything together and letting $Q_C := \delta_k$ yields the last layer derivatives

$$\frac{\partial L}{\partial \mathbf{W}_k} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \mathbf{W}_k} = \left(2(\hat{\phi} - \phi)^* \circ \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \circ f'_k(\mathbf{Z}_k)^* \right)_C \cdot \hat{\mathbf{Y}}_{k-1} = \delta'_k \hat{\mathbf{Y}}_{k-1},$$

and

$$\frac{\partial L}{\partial \mathbf{B}_k} = \frac{\partial L}{\partial \hat{\phi}} \cdot \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \cdot \frac{\partial \hat{\mathbf{Y}}_k}{\partial \mathbf{Z}_k} \cdot \frac{\partial \mathbf{Z}_k}{\partial \mathbf{B}_k} = \left(2(\hat{\phi} - \phi)^* \circ \frac{\partial \hat{\phi}}{\partial \hat{\mathbf{Y}}_k} \circ f'_k(\mathbf{Z}_k)^* \right)_C \cdot \mathbf{I} = \delta'_k \mathbf{I}.$$

The δ_k is transposed to be conformable with the with the derivative of the accumulator function with respect to the network parameter, either the weights or biases $\frac{\partial \mathbf{Z}_k}{\partial \mathbf{W}_k}$ and $\frac{\partial \mathbf{Z}_k}{\partial \mathbf{B}_k}$. This notation uses the transpose symbol instead of the dot product to make the dimensions more clear, that is

$$\delta'_k \hat{\mathbf{Y}}_{k-1} \text{ is the same as } \delta_k \cdot \hat{\mathbf{Y}}_{k-1}.$$

Second Last Layer Derivatives

The goal is to find

$$\frac{\partial L}{\partial \mathbf{W}_{k-1}} \quad \text{and} \quad \frac{\partial L}{\partial \mathbf{B}_{k-1}}.$$

Note that the first three terms in both objects have been derived and come from the previous layer's result. Using equation (3.6) yields

$$\frac{\partial L}{\partial \mathbf{W}_{k-1}} = \delta_k \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{W}_{k-1}},$$

and

$$\frac{\partial L}{\partial \mathbf{B}_{k-1}} = \delta_k \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \cdot \frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{B}_{k-1}}.$$

Recall from equation (3.9) that

$$\frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} = \mathbf{W}_k,$$

and hence

$$\frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} = f'_{k-1}(\mathbf{Z}_{k-1}).$$

But these objects are not of the same shape orientation. This is because they are separated by a layer. $\mathbf{W}_k$ is in $\mathbf{Z}_k$ shape format while $\mathbf{Z}_{k-1}$ is in the shape format of the previous

layer. These differ by a transpose. The reason this happens comes from section 3.2.6 where the accumulator is transposed to produce the layer output. Therefore in order to go backwards, the transpose must be taken again and hence applied to the $\frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}}$ term to make it conformable to $\mathbf{W}_k$.

These results yield

$$\frac{\partial L}{\partial \mathbf{W}_{k-1}} = \delta_k \cdot \mathbf{W}_k \circ \left(f'_{k-1}(\mathbf{Z}_{k-1}) \right)' \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{W}_{k-1}},$$

and

$$\frac{\partial L}{\partial \mathbf{B}_{k-1}} = \delta_k \cdot \mathbf{W}_k \circ \left(f'_{k-1}(\mathbf{Z}_{k-1}) \right)' \cdot \frac{\partial \mathbf{Z}_{k-1}}{\partial \mathbf{B}_{k-1}}.$$

The use of dot notation is deliberate to avoid having to be explicit about transposes in the definition. Since the element wise operator is being used however, it becomes necessary to be explicit that it is the $\frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}}$ term which gets transposed in the multiplication process.

Using the delta in (3.7) produces the recursive formula used in the back propagation algorithm by letting

$$\delta_{k-1} = \delta_k \cdot \frac{\partial \mathbf{Z}_k}{\partial \hat{\mathbf{Y}}_{k-1}} \circ \left(\frac{\partial \hat{\mathbf{Y}}_{k-1}}{\partial \mathbf{Z}_{k-1}} \right)',$$

which finally yields the following results, note the inclusion of the transpose on the delta,

$$\frac{\partial L}{\partial \mathbf{W}_{k-1}} = \delta_{k-1} \cdot \hat{\mathbf{Y}}_{k-2} = \delta'_{k-1} \hat{\mathbf{Y}}_{k-2},$$

and

$$\frac{\partial L}{\partial \mathbf{B}_{k-1}} = \delta_{k-1} \cdot \mathbf{I} = \delta'_{k-1} \mathbf{I}.$$

The same procedure follows for the next $k - 2$ layers until layer 1 is reached. The optimising procedure proceeds as

$$\begin{aligned} \mathbf{W}_k^{new} &:= \mathbf{W}_k^{old} - \eta \frac{\partial L}{\partial \mathbf{W}_k^{old}} = \mathbf{W}_k^{old} - \eta \delta_k'^{old} \hat{\mathbf{Y}}_{k-1}^{old} \quad \text{for } k = \text{last layer}, \dots, 1, \\ \mathbf{B}_k^{new} &:= \mathbf{B}_k^{old} - \eta \frac{\partial L}{\partial \mathbf{B}_k^{old}} = \mathbf{B}_k^{old} - \eta \delta_k'^{old} \mathbf{I} \quad \text{for } k = \text{last layer}, \dots, 1. \end{aligned}$$

3.6 Post Convergence Form

Once the error stabilises, that is the calculated loss stops changing, it could be said that the network has converged. Convergence and or achieving a specified number of passes

through the network is a sufficient stopping condition which means back-propagation no longer occurs and the loss function is no longer needed (Haykin, 2009).

After convergence, the generator becomes a single forward push through the network which fulfills the mapping from random standard normal samples to target samples which can be expressed as

$$\begin{aligned}\hat{\mathbf{Y}}_k &= G_\Omega(\mathbf{X}) \\ &= G_{\mathbf{W}_1, \dots, \mathbf{W}_k, \mathbf{B}_1, \dots, \mathbf{B}_k}(\mathbf{X}) \\ &= f_k(f_{k-1}(\dots f_1(\mathbf{X}\mathbf{W}'_1 + \mathbf{B}'_1) \dots \mathbf{W}'_{k-1} + \mathbf{B}'_{k-1})\mathbf{W}'_k + \mathbf{B}'_k) \\ &\text{where rows of } \hat{\mathbf{Y}}_k \sim \mathcal{A}(\theta)\end{aligned}$$

3.7 Choice of Activation Function

The ReLU (Rectified Linear Unit) is chosen as a fixed hyper-parameter for the analysis of this paper. Although the generator allows for a variety of activation functions, choosing a fixed activation function streamlines the analysis. The ReLU is defined as

$$\text{ReLU}(x) = \begin{cases} x, & \text{if } x > 0, \\ 0, & x \leq 0. \end{cases}$$

The primary trait sought from an activation function is non-linearity. If a linear activation such as the identity, that is returning the accumulator sum $\mathbf{Z}'_k$, then equation (3.2) would collapse into another linear function

$$\hat{\mathbf{Y}}_k = \mathbf{Z}'_k = ((\mathbf{X}\mathbf{W}'_1 + \mathbf{B}'_1)\mathbf{W}'_2 + \mathbf{B}'_2) \dots \mathbf{W}'_k + \mathbf{B}'_k = \mathbf{X}\mathbf{W}'_\star + \mathbf{B}'_\star.$$

This would prevent the generator from being able to model any complex patterns that may exist in the data (Fernandez & Mali, 2025). The ReLU function in particular works by constructing piece-wise linear segments of otherwise nonlinear functions (Cai, Choi, & Liu, 2024).

3.7.1 Advantages of using ReLU functions:

The ReLU is efficient, that is they are computationally cheap since they return either what it is given or 0 (Fernandez & Mali, 2025). Inside each neuron where this function is present, only an addition via the accumulator, that is a linear combination $\mathbf{Z}_k$, and then a comparison, that is returning $\max(x, 0)$, takes place as opposed to other activation functions which incorporate multiplicative operations such as the sigmoid function.

The ReLU promotes sparsity. Sparse networks tend to be more efficient in the sense that they lead to lower memory requirements and can still generalise well if not better than dense networks (Hoeffler, Alistarh, Ben-Nun, Dryden, & Peste, 2021). This is done by zeroing out parameters, the neurons, and reducing the computational intensity of the network overall. In (symmetric) randomly initialised networks such as the generator in

this paper, up to half of the neurons in a layer can be inactive since all negative inputs result in zero weights (Lu, Shin, Su, & Karniadakis, 2020).

The ReLU is less prone to vanishing gradient. Recall that when gradients become small during back propagation, the network experiences the vanishing gradient phenomenon (H. Lee et al., 2024). Since the derivative of the ReLU function is 1 for positive inputs, and 0 otherwise, the back propagation of terms through the network is relatively stable. For a sigmoid function, these squish input values between 1 and 0. When back propagation occurs, terms get multiplied by values which get progressively smaller in scale (Sonoda & Murata, 2017).

The ReLU has fast convergence (Fernandez & Mali, 2025). Fast convergence in this context refers to the reduction in training time, or how quickly the loss is reduced. Fast reduction of the loss can be attributed down to the ReLU function’s desirable gradient descent properties as mentioned above, allowing the generator to compute with fewer passes as opposed to using a sigmoid activation function.

The ReLU is Positively scale invariant (Terada & Hirose, 2020). Scaling weights in the positive direction only changes the magnitude of the output, not the direction. This is a useful property for back propagation since the generator can learn faster with large (not unbounded) activations since the gradient descent algorithm becomes stable over time.

$$\text{ReLU}(cx) = c\text{ReLU}(x) \text{ for } c > 0.$$

3.7.2 Disadvantages of using ReLU functions:

The ReLU is unbounded. Since ReLU functions have no upper bound, and because of the scale invariance property mentioned above, they can explode (values tend to infinity) for very large inputs and cause numerical instability which can however still be controlled through careful weight initialisation (Hanin, 2018). Exploding inputs can lead to unstable gradients, sensitivity to outliers, and overfitting.

$$\text{as } x \rightarrow \infty, \quad \text{ReLU}(x) \rightarrow \infty.$$

ReLU activation functions can suffer from parameterisation redundancy and the dying ReLU phenomenon (@ Lu et al., 2020). Dying ReLU refers to when neurons in a layer continuously output 0 after some time, and as such, become inactive for the rest of the learning duration. These neurons no longer contribute to the learning procedure and become dead weight to the network, in other words, they become redundant. Lu et al. (2020) prove that as the depth of a ReLU network becomes infinitely large, it will ‘die’ in probability. They propose initialising the weights and biases from asymmetric distributions in order to alleviate this phenomenon.

Chapter 4

Data Analysis and Selected Results

4.1 Hyperparameters

- $\mathbf{W}^0 \sim \text{Uni}(-1, 1)$, the initialisation of the weights
- $\mathbf{b}^0 = 0$, the initialisation of the biases
- $\mathbf{X} \sim N_1(0, 1)$
- $\mathbf{T} \sim N_1(0, 1)$
- $n = 100$ Noise samples
- $m = 100$ Frequency samples
- Architecture: 25 ReLU, 10 ReLU, 1 purelin
- $\text{lr} = 0.001$
- passes = 5000

All simulations were conducted with a set seed of 1 for reproducibility.

These hyper-parameters were chosen primarily to enjoy a mix of both performance and convergence of the generator. Uniform initialisation of the generator weights resulted in large variation in convergence rates, with some training periods struggling to converge within a tolerable number of passes (< 10000 passes) whilst others converging almost instantly. The result of this behaviour was that re-initialisation of the training cycle would often times result in completely different performance outcomes without setting predetermined random seeds despite no adjustments being made to the hyper-parameters. In essence, the generator is overly sensitive to random weight initialisation. This phenomenon can be seen when comparing the last examples of the normal, gamma, chi-square, and exponential distributions in figures 4.5, 4.10, 4.15, and 4.20 with different seeded equivalents in figures 4.41, 4.42, 4.43, and 4.44 respectively.

The biases being set to 0 is common practice as per the literature in Steinwart (2019) so as to not arbitrarily shift the decision boundary of each neuron in a layer. The initial samples and frequencies were each drawn from standard univariate normal distributions of length 100. The value of 100 was arbitrary but found to offer a decent balance between computational efficiency and accuracy. Lower values struggled to produce good quality samples and higher values tended to slow down the generator considerably.

The choice to draw the frequencies from a standard normal distribution comes from the weighting scheme used by Brück (2025). In their paper, the frequencies were weighted

using a Gaussian kernel which would give preference to values centered near 0 in the frequency domain. The method in this paper however, simulates this weighting by drawing the frequencies directly from a standard normal distribution, effectively computing a Monte-Carlo estimate of the Gaussian kernel.

The architecture of a 2 hidden layer network with 25 and 10 neurons each with ReLU activation functions applied is inspired by the network used by Brück (2025). The 300 and 50 neuron structure they used proved to be computationally slow for the generator in this paper and hence through experimentation (iteratively reducing the number of neurons in each layer), the values of 25 and 10 were settled upon for allowing the generator to compute quickly and still converging most of the time.

The learn rate was chosen only by dividing or multiplying by 100 and running the generator through multiple training cycles to find the largest value at which the approximations would converge. For example, a learn rate of 1 would be too large, so would 0.1, and 0.01, but the generator would become stable at 0.001 hence this value would become the learn rate. The learn rate is desired to be ‘big’ to aid in faster convergence, but to also avoid causing the generator to diverge (You, Long, Wang, & Jordan, 2019). The number of passes was chosen to be 5000 arbitrarily to avoid lengthy simulation times. Most of the distributions studied in this paper were successfully mimicked at the 3000th pass mark if not following a particular seed. From repeated testing, if the generator had not yet converged by the 5000th mark, it would become unlikely that it would do so in the next 5000.

4.2 Univariate Distribution Matching

4.2.1 Description of the Data

Eight univariate distributions were mimicked with various parameter values using a set seed of 1, with the purposes of testing whether the generator could find a successful mapping to each. Six of the Eight distributions are commonly used in the field of statistics with the remaining two being a mixture and mixture of mixtures to determine whether tail-heaviness and multi-modality had a negative effect on the generator’s ability to reproduce sample estimates or not.

Testing was done by comparing the theoretical mean and variance of the target distribution with the actual mean and variance of the estimated samples produced by the generator. The justification behind this method of comparison rests on the idea that matching mean and variance would provide the same information as comparing the first and second moments as well as the parameters of the target and simulated distributions. 10 000 samples were produced each time.

Tables 4.1 to 4.8 depict the results of comparing the mean and variance of the target distribution with the generator-produced counterpart, with accompanying errors for each statistic and the final value of the loss function at the 5000th pass. The distribution tab provides information on the particular type of distribution being mimicked as well as the parameters used to create it. Due to space constraints, the equally weighted NIG distribution’s parameter names have been cut but remain in the order of $\alpha_1, \beta_1, \delta_1, \mu_1, \alpha_2, \beta_2, \delta_2, \mu_2$ in table 4.8.

Figures 4.1 to 4.44 depict the generator’s reconstruction of the target distribution. The left side of each figure depicts the generator’s attempt at reconstructing both the real and imaginary parts of the target distribution’s characteristic function. A tighter fit (not seeing the dashed line) is better. The right side of each figure depicts a histogram of the generated sample estimates overlaid with the density line of true samples drawn from the target distribution. Large overlap, if it looks like the histogram produced the density line, between the histogram and density line is better.

4.2.2 Reporting

As per tables 4.1 to 4.8, the generator was able to find mappings to all but the Cauchy distribution, depending on the nature of the parameters. Distributions with nice (small mean, small variance) parameters were able to be reconstructed relatively easily, with features revolving around heavy tails and large means being the most difficult to reproduce and mostly resulting in divergent reconstructions. Figures 4.10, 4.15, 4.21, 4.22, and all of the Cauchy distributions represent what would be termed as having not nice parameter values and show divergent or unsuccessful reconstructions. These distributions generally have large probability density in the tails as well as a large mean such as the chi-square distribution in 4.15 with 50 degrees of freedom.

Oscillatory behaviour in the real and imaginary parts of the fitted characteristic function, indicating lack of fit, and poor overlap of the density line and estimated sample histogram are signs that the generator struggled to find a good quality mapping to that particular target distribution. When observing the chi-square distribution in figure 4.15, it is possible to see this oscillatory behaviour of the fitted characteristic function, with the lower portion of the true real part of the characteristic function not being fitted at all. It is also possible to observe that the estimated samples histogram includes values below 0 which is impossible for a chi-square distribution, which only exists in the positive real numbers.

The Cauchy distributions in figures 4.26 to 4.30 depict an unidentifiable estimated samples histogram with a density line that is essentially vertical, a spike at the location parameter. The reason this occurs is because the Cauchy distribution has extremely heavy tails and infinite variance. This means that in large sample batches, in this case 10 000, it is possible to observe large values in the extremes of the real number line which dwarf those produced by the generator. It is for this reason too why the results in table 4.6 uses the location and scale parameters as measures of true versus simulated fit. However, it can also be seen that these measures do not provide a good idea on the quality of generated samples considering the state of the histogram and density line overlap.

The equally weighted NIG mixtures were reconstructed successfully except for the highly skewed and shifted bimodal distribution in figure 4.40 which had large variance error of -69.2162. This case is an example of why it is necessary to look at both the moment matching table as well as the histogram. From the histogram, the overlap looks like a good fit, however the variance of the estimated samples is way off from the true theoretical value. This is the opposite to what was seen in cases for the Cauchy distribution, where the parameter table showed decent results but the histogram revealed severe mismatch. Apart from the moment mismatch, the generator did not struggle to capture the bimodal features of each mixture distribution.

Noticeably, reconstruction of the same distributions with different random seeds produces large variation in performance, sometimes being the difference between a successful reconstruction and a divergent one. Reconstruction the 20 mean normal distribution in figure 4.5 with a seed of 1 for example produces a simulated distribution with substantial mismatch in the left tail. Using a different seed without changing any of the hyperparameters results in figure 4.41 which is much closer to the true distribution. The large mean normal approximating chi-square distribution provides a more severe example of divergence and the difference a random seed can make. Although the initial reconstruction in figure 4.15 is poor, changing the seed results in figure 4.43 which essentially has no overlap at all to the true distribution and has shifted into the negative values.

4.2.3 Normal Distributions

Table 4.1: Normal Distribution True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
0 Mean	Mean	0	-0.3749	0.3749	0.0116
(mu = 0, var = 10)	Variance	10	11.6454	-1.6454	
5 Mean	Mean	5	4.6278	0.3722	0.0117
(mu = 5, var = 10)	Variance	10	11.6158	-1.6158	
10 Mean	Mean	10	9.7400	0.2600	0.0688
(mu = 10, var = 10)	Variance	10	12.7881	-2.7881	
15 Mean	Mean	15	14.7651	0.2349	0.0663
(mu = 15, var = 10)	Variance	10	12.6720	-2.6720	
20 Mean	Mean	20	17.1245	2.8755	0.4506
(mu = 20, var = 10)	Variance	10	62.8881	-52.8881	

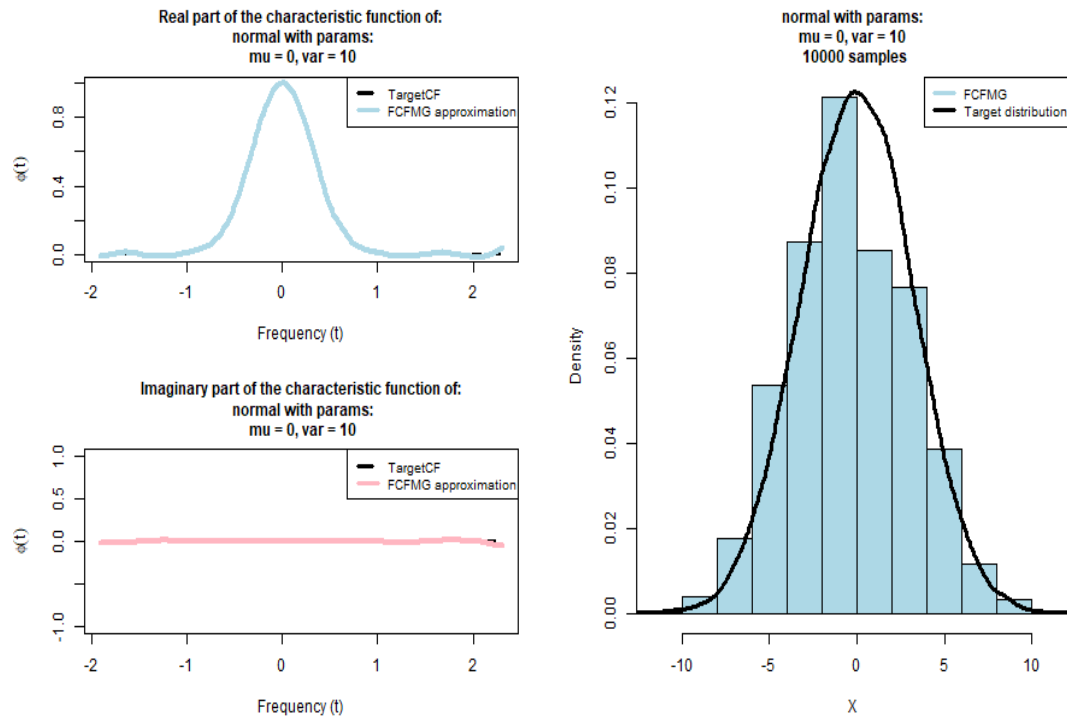


Figure 4.1: 0 Mean Normal Distribution

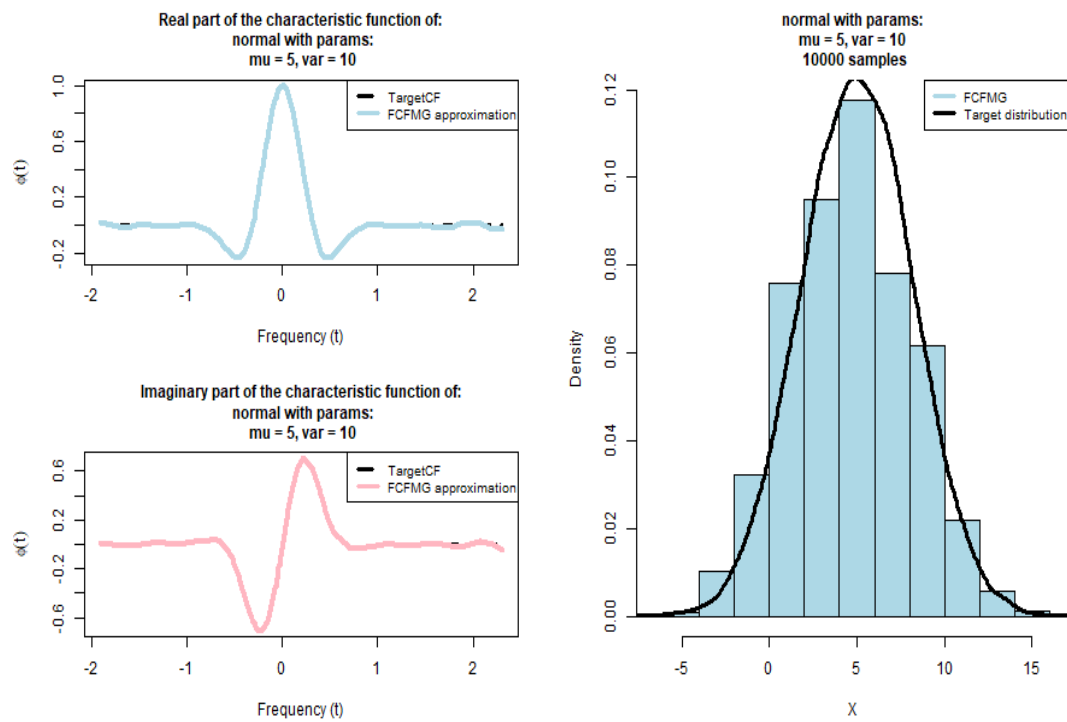


Figure 4.2: 5 Mean Normal Distribution

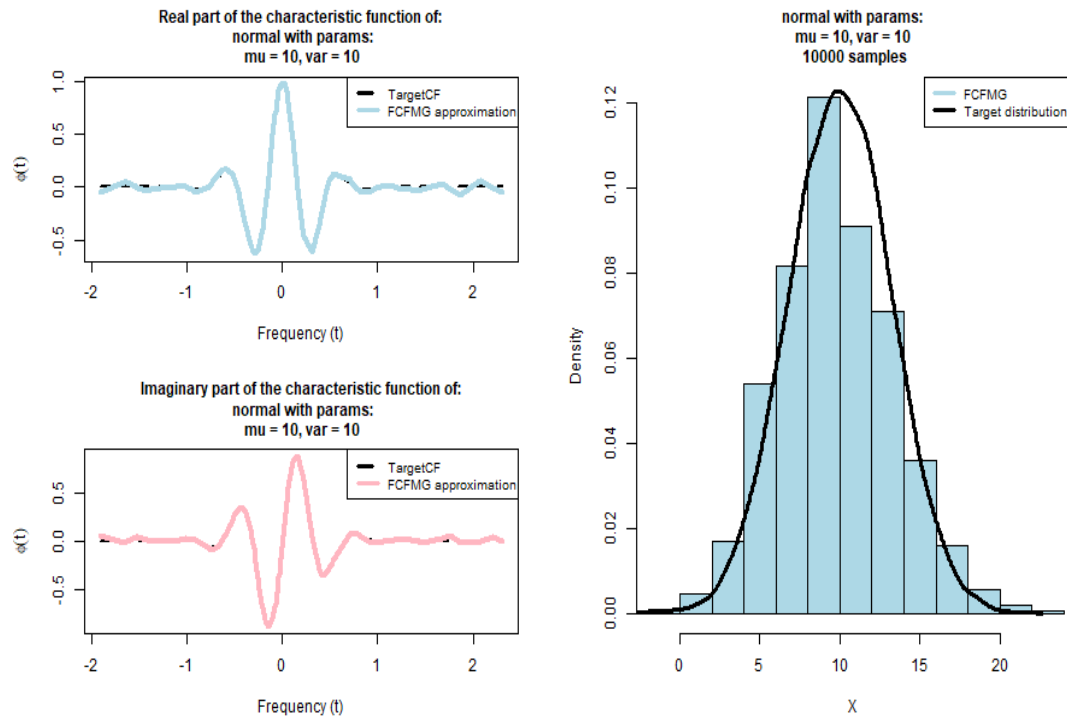


Figure 4.3: 10 Mean Normal Distribution

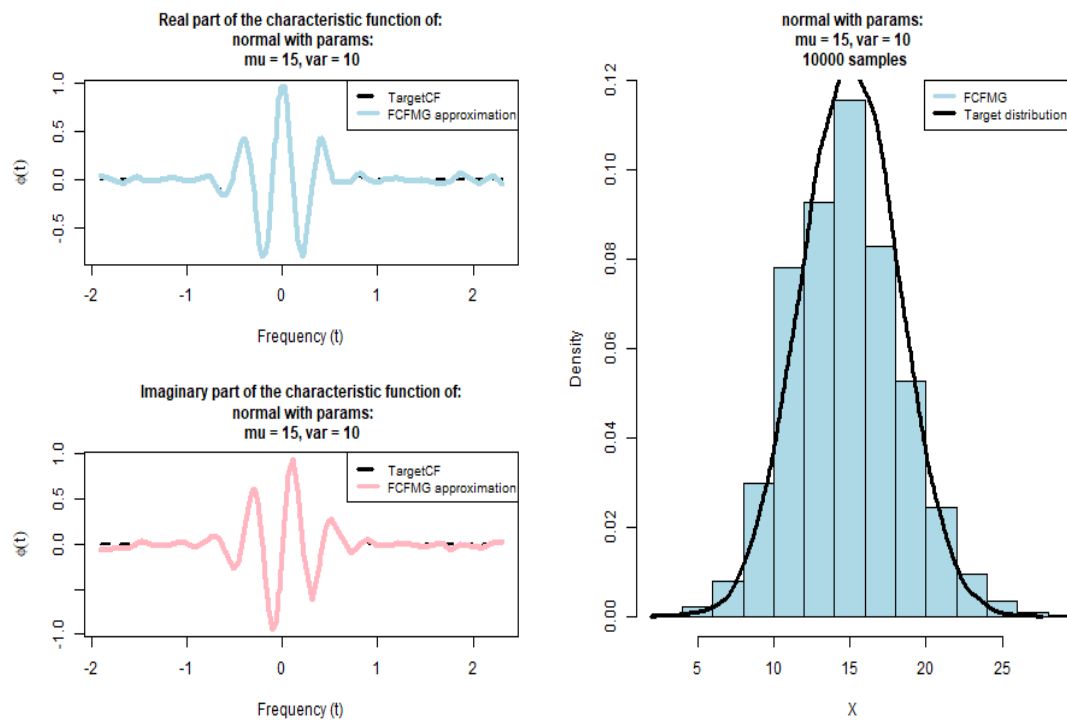


Figure 4.4: 15 Mean Normal Distribution

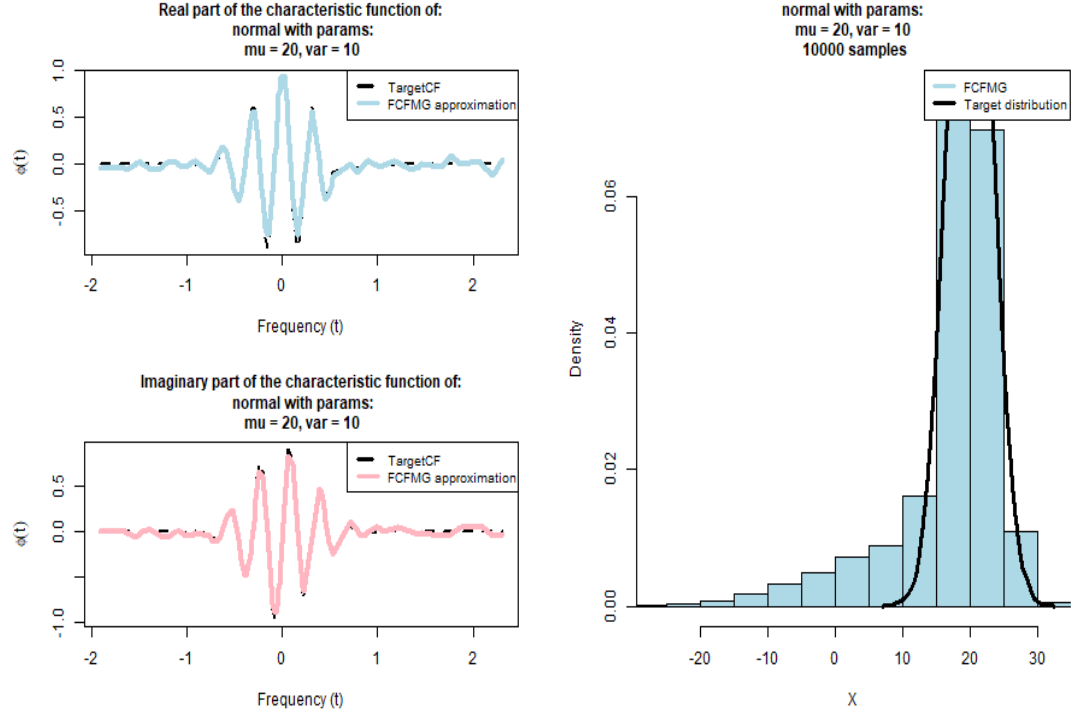


Figure 4.5: 20 Mean Normal Distribution

4.2.4 Gamma Distributions

Table 4.2: Gamma Distribution True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Right Skew Heavy Tailed	Mean	0.5	0.5118	-0.0118	0.0032
(alpha = 0.5, lambda = 1)	Variance	0.5	0.4911	0.0089	
Moderate Right Skew	Mean	2.0	1.8678	0.1322	0.0026
(alpha = 2, lambda = 1)	Variance	2.0	2.1631	-0.1631	
Moderate Symmetric	Mean	10.0	9.7169	0.2831	0.048
(alpha = 10, lambda = 1)	Variance	10.0	12.0629	-2.0629	
Moderate Symmetric Heavy Tailed	Mean	10.0	9.6074	0.3926	0.0749
(alpha = 5, lambda = 0.5)	Variance	20.0	22.5259	-2.5259	
Large Mean	Mean	50.0	36.7375	13.2625	2.3043
(alpha = 10, lambda = 0.2)	Variance	250.0	1119.9150	-869.9150	

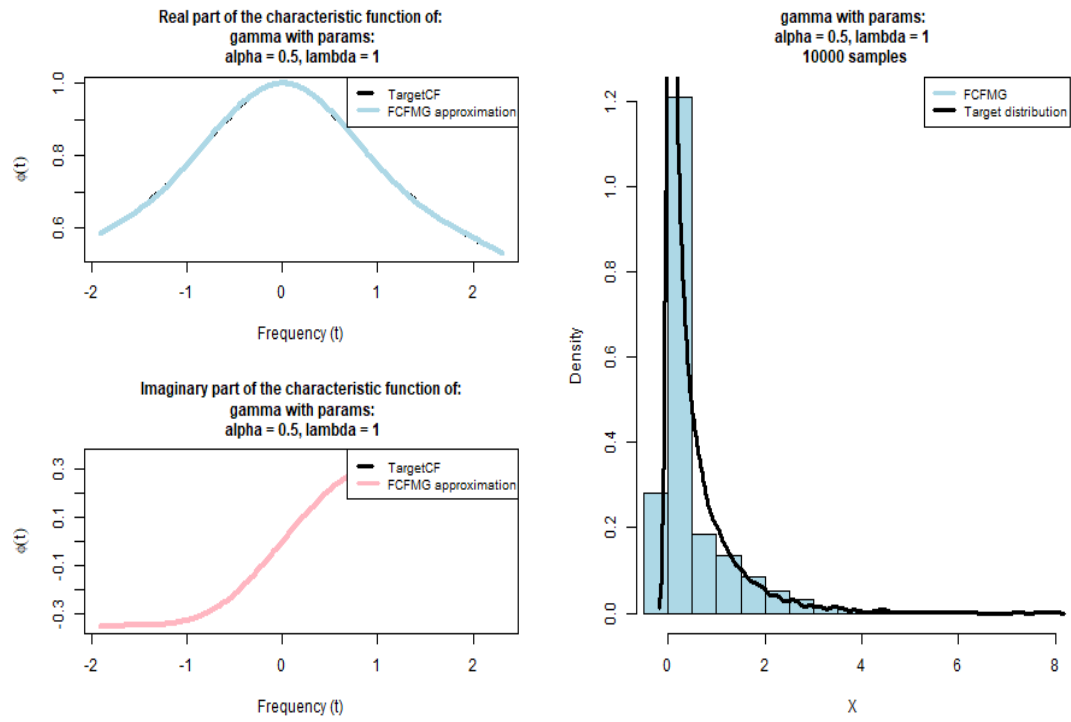


Figure 4.6: Right Skew Heavy Tailed Gamma Distribution

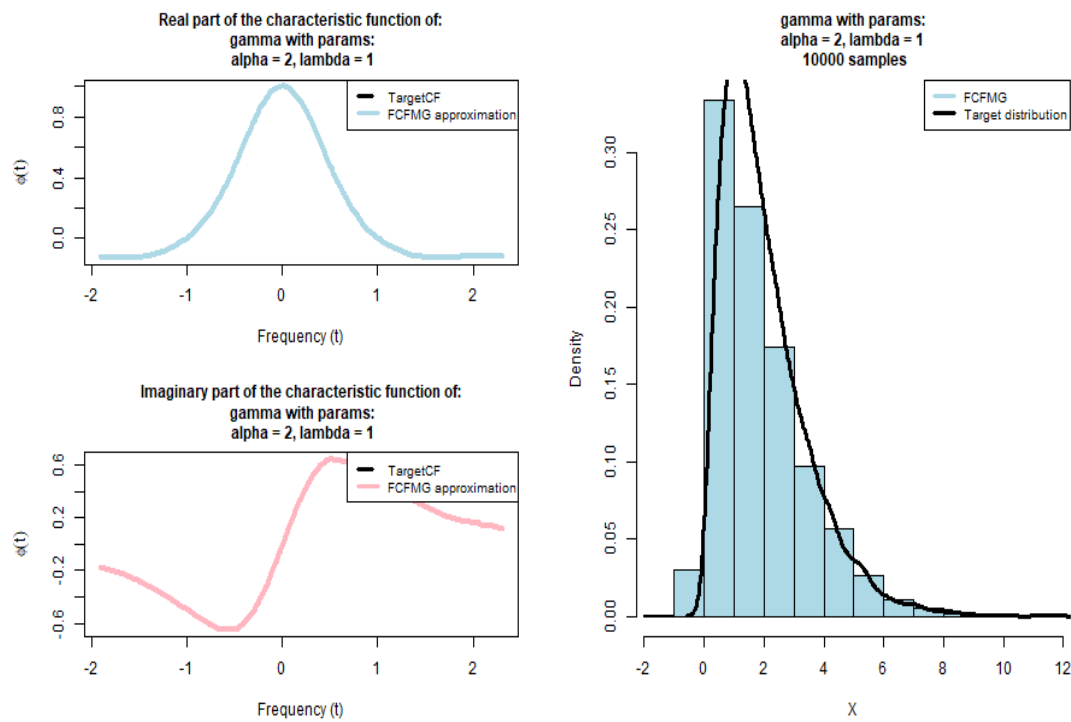


Figure 4.7: Moderate Right Skew Gamma Distribution

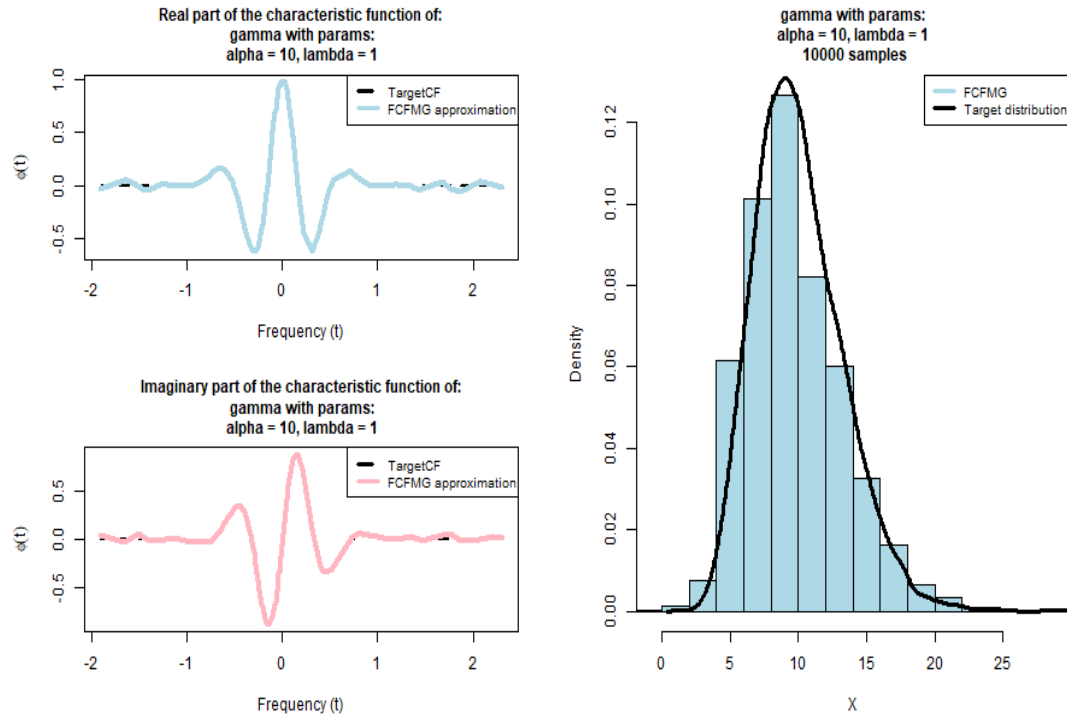


Figure 4.8: Moderate Symmetric Gamma Distribution

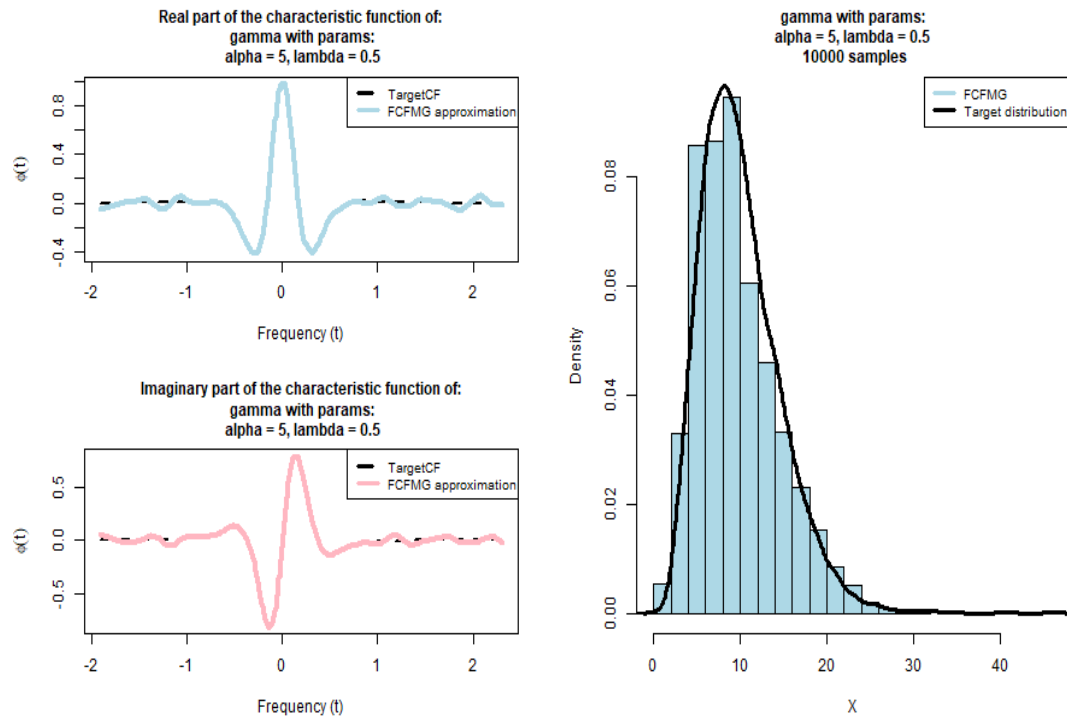


Figure 4.9: Moderate Symmetric Heavy Tailed Gamma Distribution

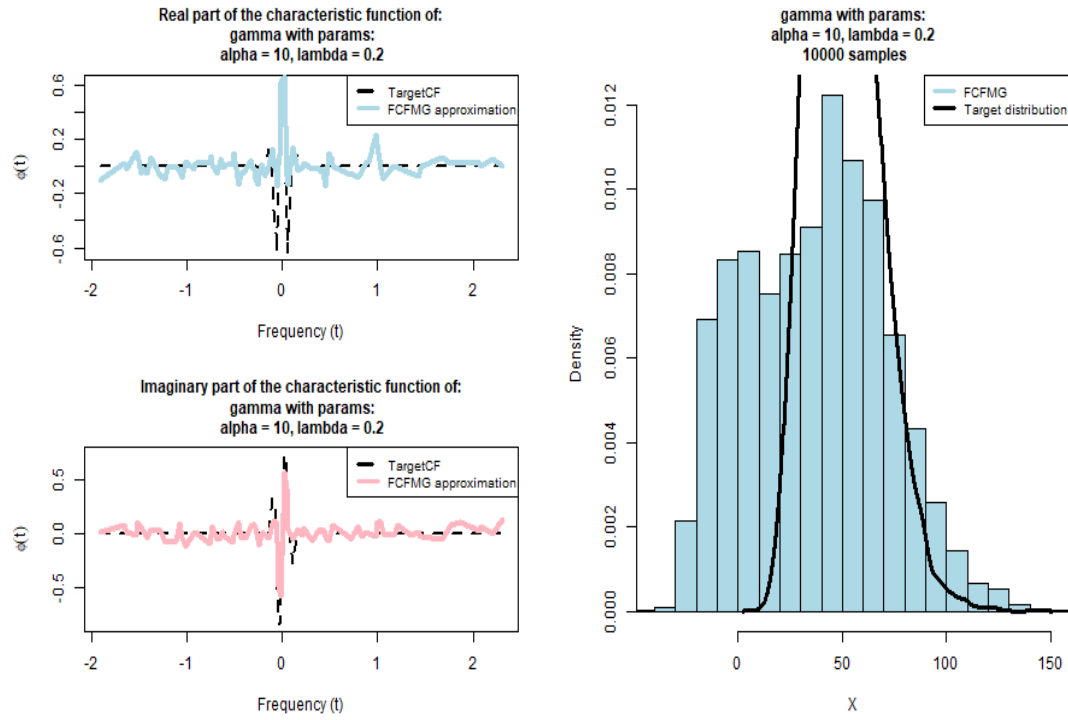


Figure 4.10: Large Mean Gamma Distribution

4.2.5 Chi-Square Distributions

Table 4.3: Chi-Square Distribution True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Right Skew	Mean	1	0.8890	0.1110	0.0148
(k = 1)	Variance	2	1.7543	0.2457	
Moderate Skew	Mean	5	4.6333	0.3667	0.0308
(k = 5)	Variance	10	10.2238	-0.2238	
Moderate Symmetric	Mean	10	9.6074	0.3926	0.0749
(k = 10)	Variance	20	22.5259	-2.5259	
Large Mean Moderate Symmetric	Mean	30	25.1618	4.8382	0.6385
(k = 30)	Variance	60	140.3593	-80.3593	
Large Mean Normal Approximating	Mean	50	34.2606	15.7394	1.888
(k = 50)	Variance	100	769.4549	-669.4549	

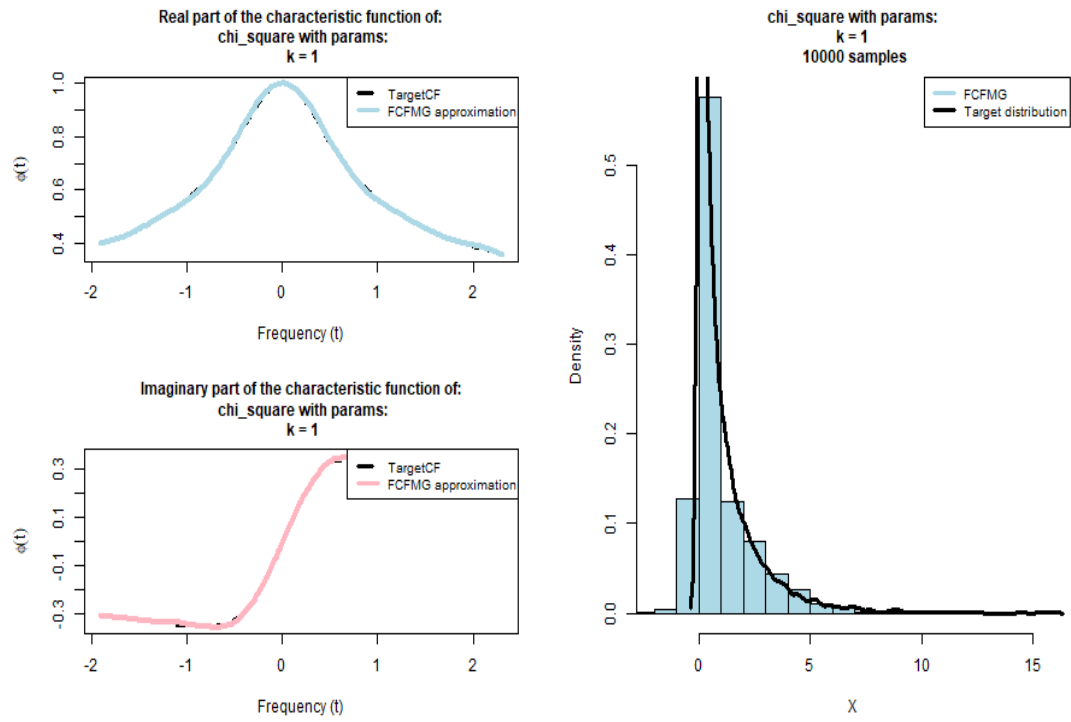


Figure 4.11: Right Skew Chi-Square Distribution

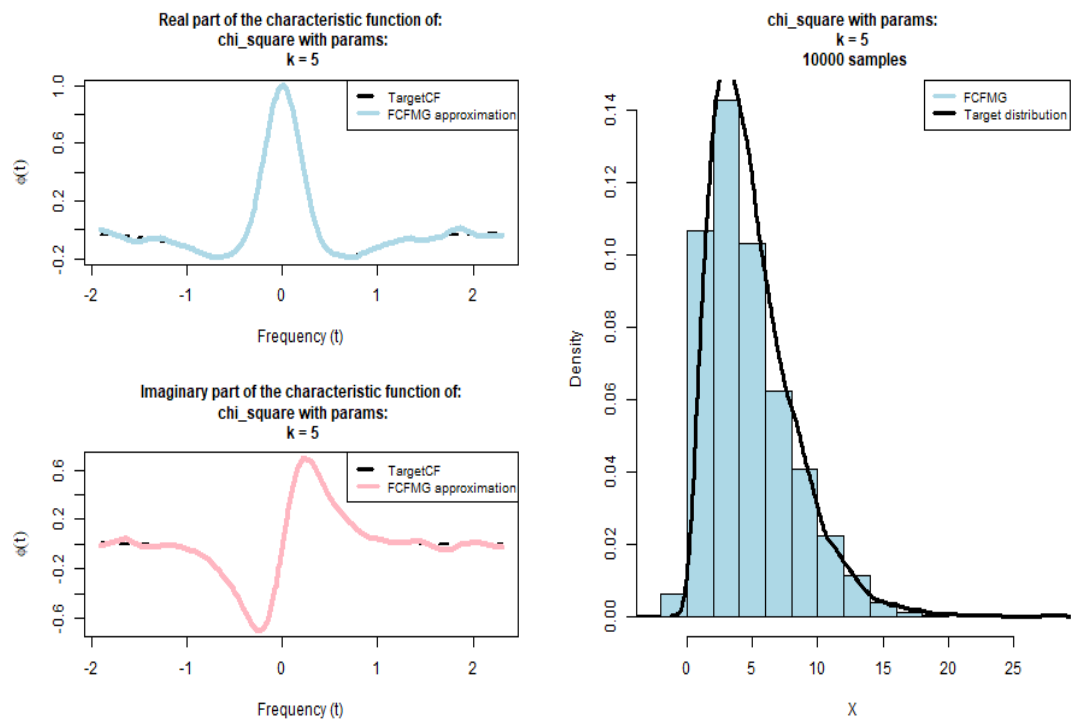


Figure 4.12: Moderate Skew Chi-Square Distribution

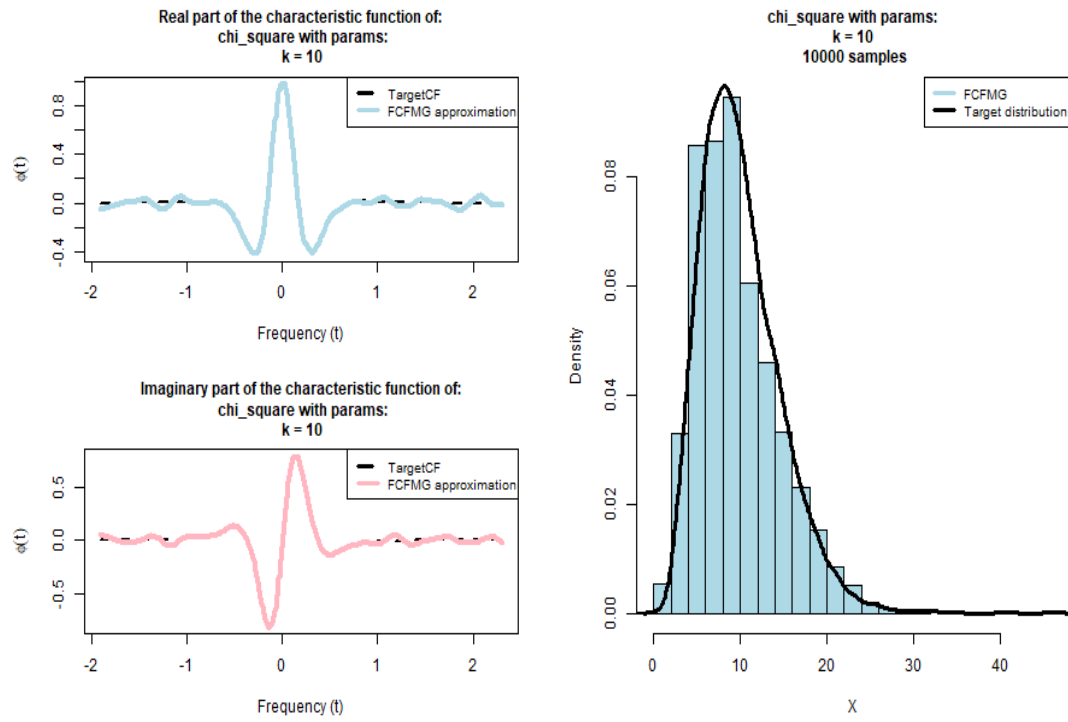


Figure 4.13: Moderate Symmetric Chi-Square Distribution

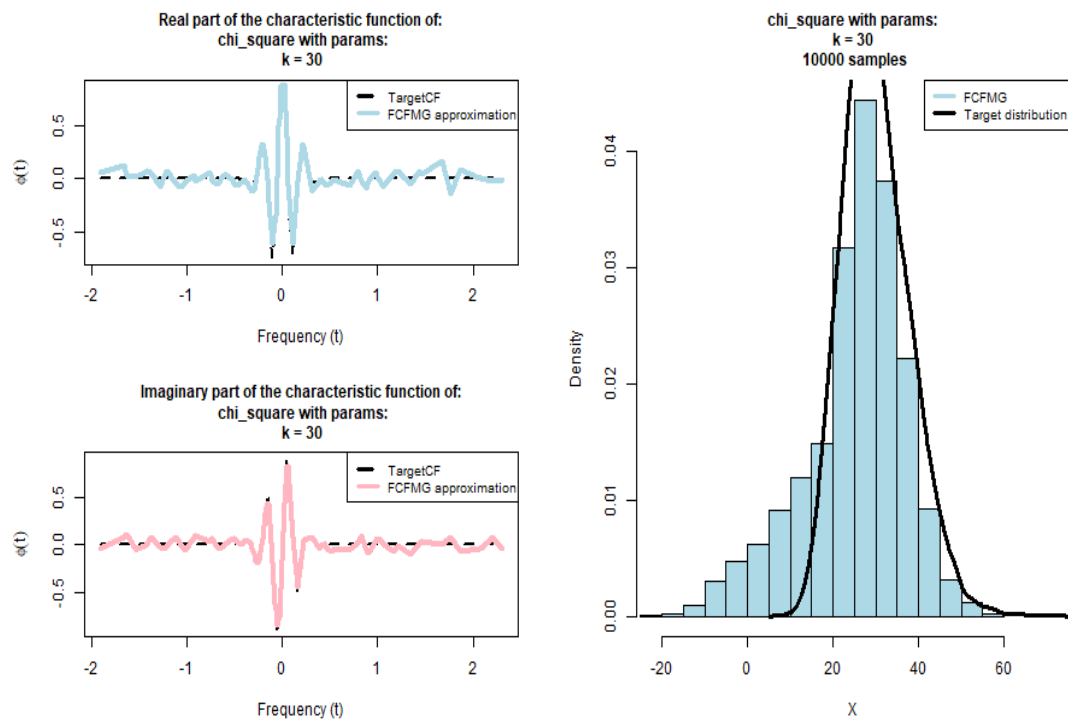


Figure 4.14: Large Mean Moderate Symmetric Chi-Square Distribution

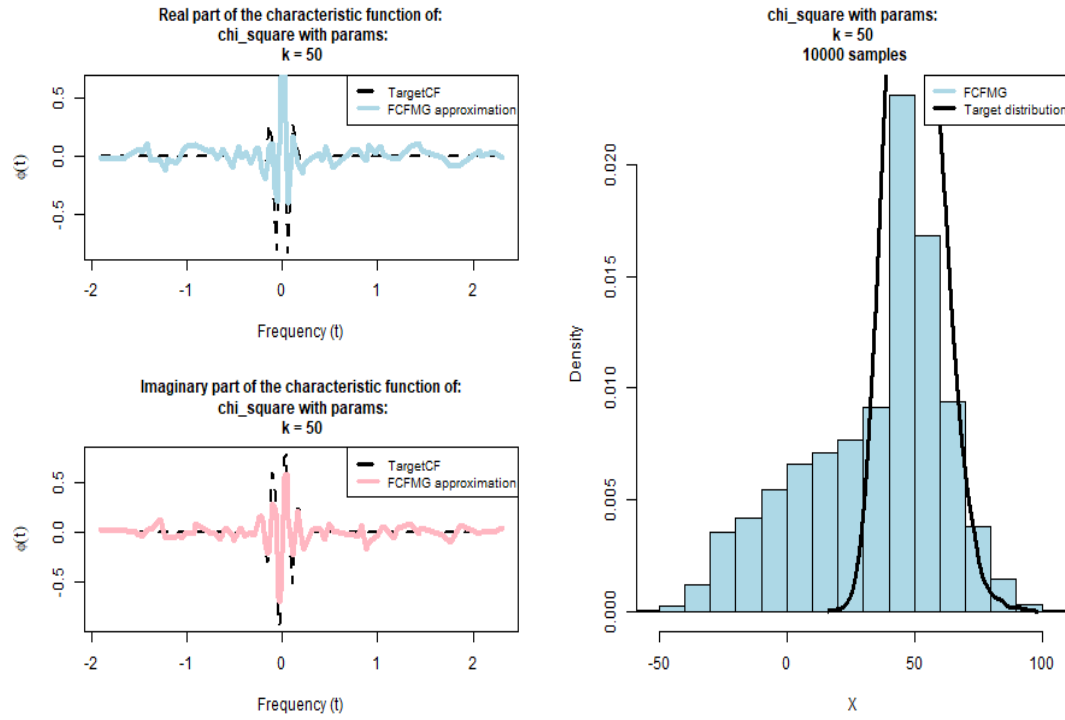


Figure 4.15: Large Mean Normal Approximating Chi-Square Distribution

4.2.6 Exponential Distributions

Table 4.4: Exponential Distribution True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Heavy tailed	Mean	10.00	8.2310	1.7690	0.2154
(lambda = 0.1)	Variance	100.00	63.5205	36.4795	
Moderate Spread	Mean	2.00	1.8159	0.1841	0.0125
(lambda = 0.5)	Variance	4.00	3.6954	0.3046	
Standard	Mean	1.00	0.9309	0.0691	0.0049
(lambda = 1)	Variance	1.00	1.0108	-0.0108	
Narrow	Mean	0.50	0.4815	0.0185	0.0011
(lambda = 2)	Variance	0.25	0.2631	-0.0131	
Short Tailed	Mean	0.20	0.1916	0.0084	1e-04
(lambda = 5)	Variance	0.04	0.0477	-0.0077	

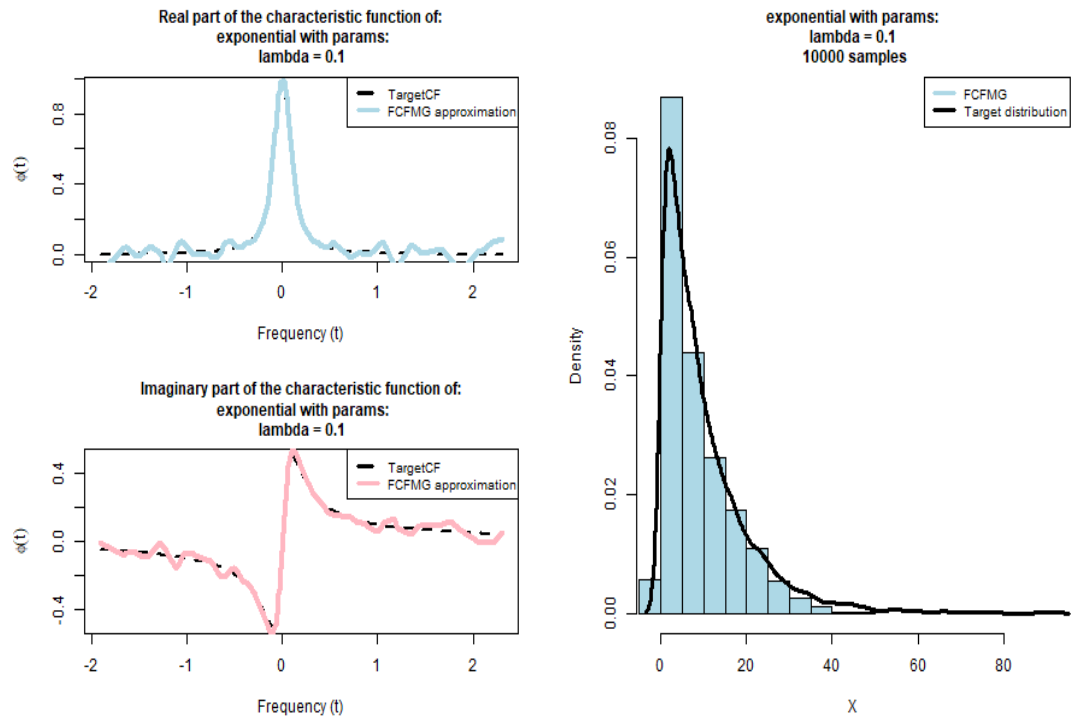


Figure 4.16: Heavy tailed Exponential Distribution

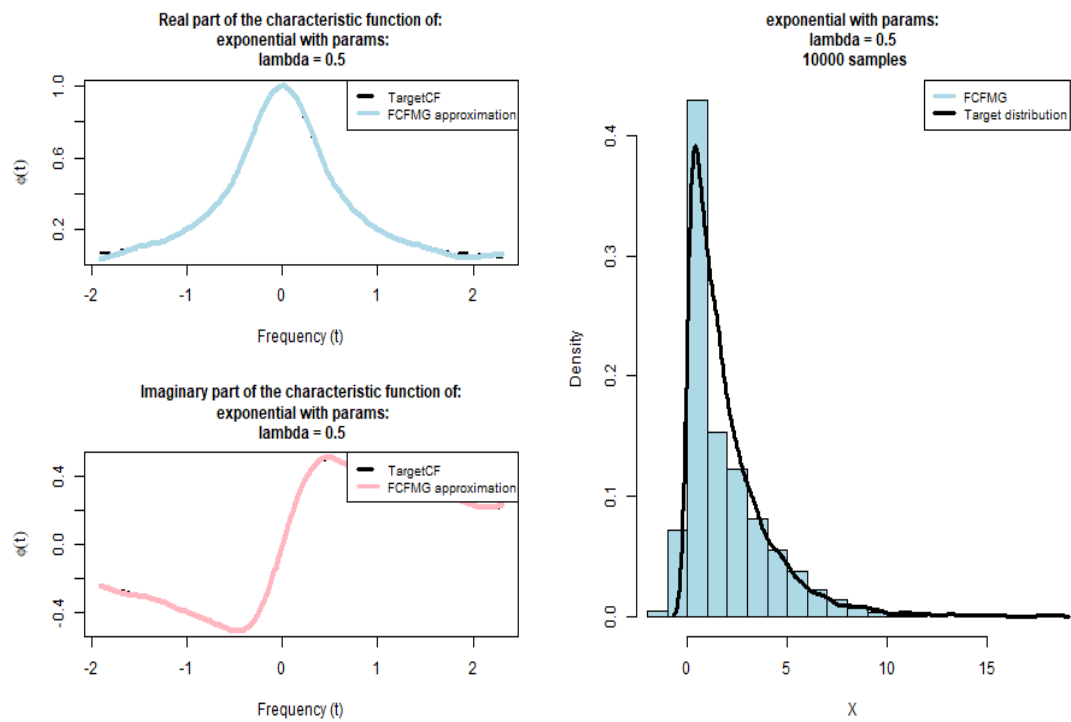


Figure 4.17: Moderate Spread Exponential Distribution

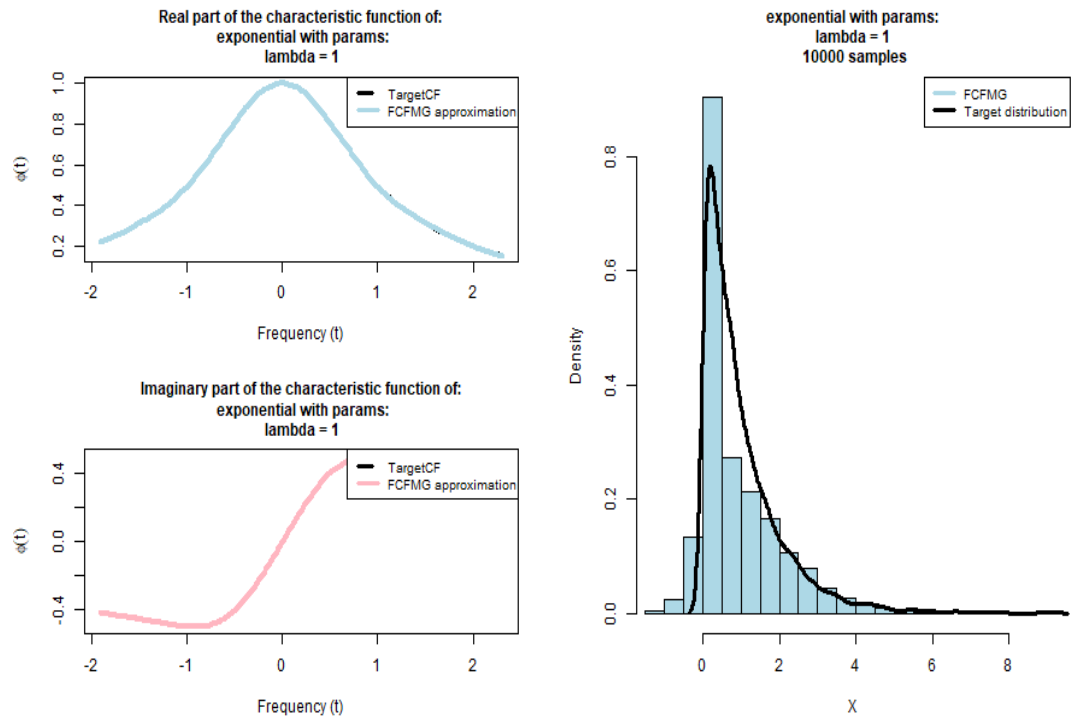


Figure 4.18: Standard Exponential Distribution

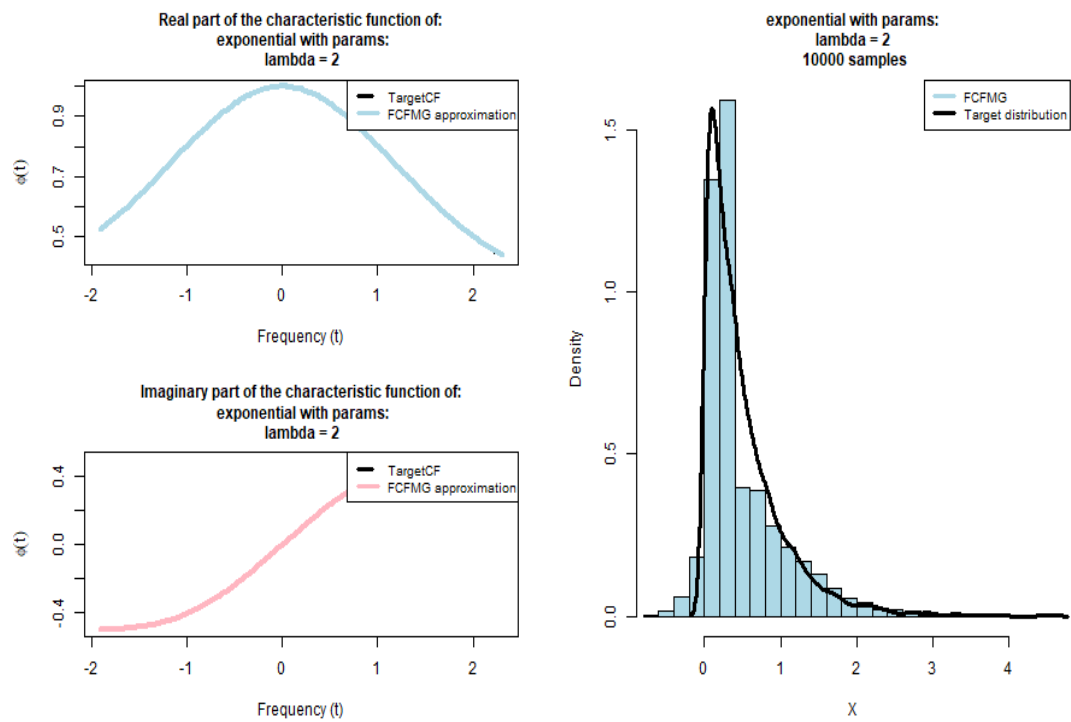


Figure 4.19: Narrow Exponential Distribution

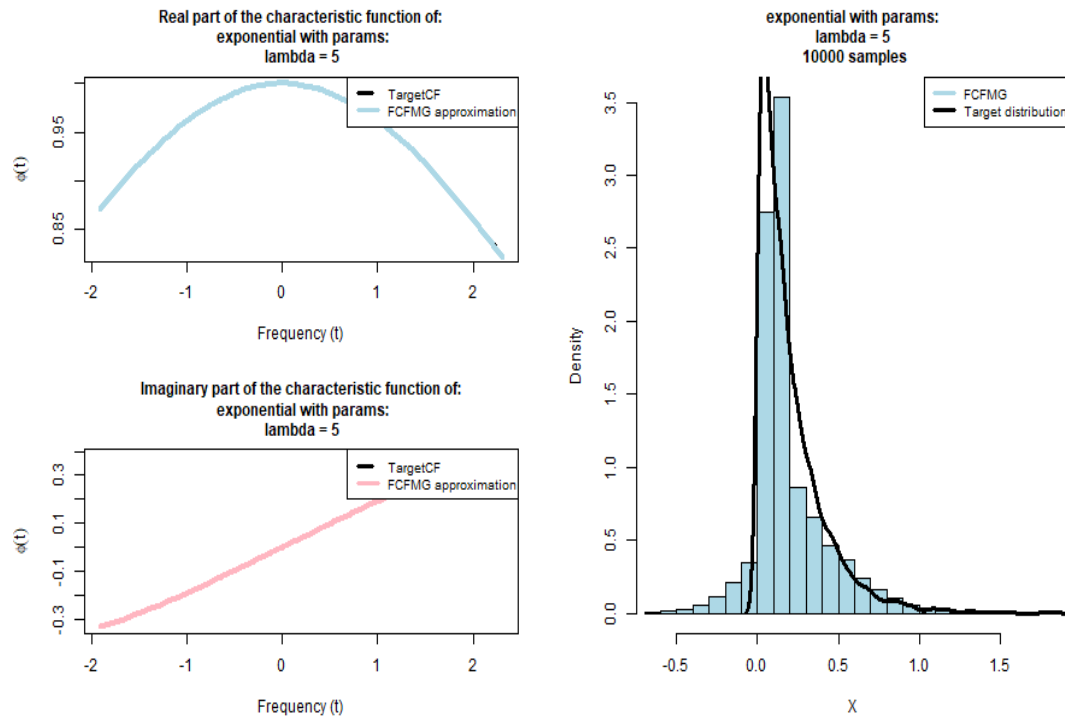


Figure 4.20: Short Tailed Exponential Distribution

4.2.7 T Distributions

Table 4.5: T Distribution True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Standard Cauchy	Mean	0	-0.5651	0.5651	0.2202
(nu = 1)	Variance	undefined	7.4208	undefined	
Infinite Variance	Mean	0	-0.2011	0.2011	0.085
(nu = 2)	Variance	undefined	2.9989	undefined	
Finite Variance Heavy Tailed	Mean	0	-0.1473	0.1473	0.0371
(nu = 3)	Variance	3	2.2625	0.7375	
Moderate Tailed	Mean	0	-0.1353	0.1353	0.0119
(nu = 5)	Variance	1.6667	1.7733	-0.1067	
Approximately Normal	Mean	0	-0.1157	0.1157	0.0027
(nu = 10)	Variance	1.25	1.5040	-0.254	

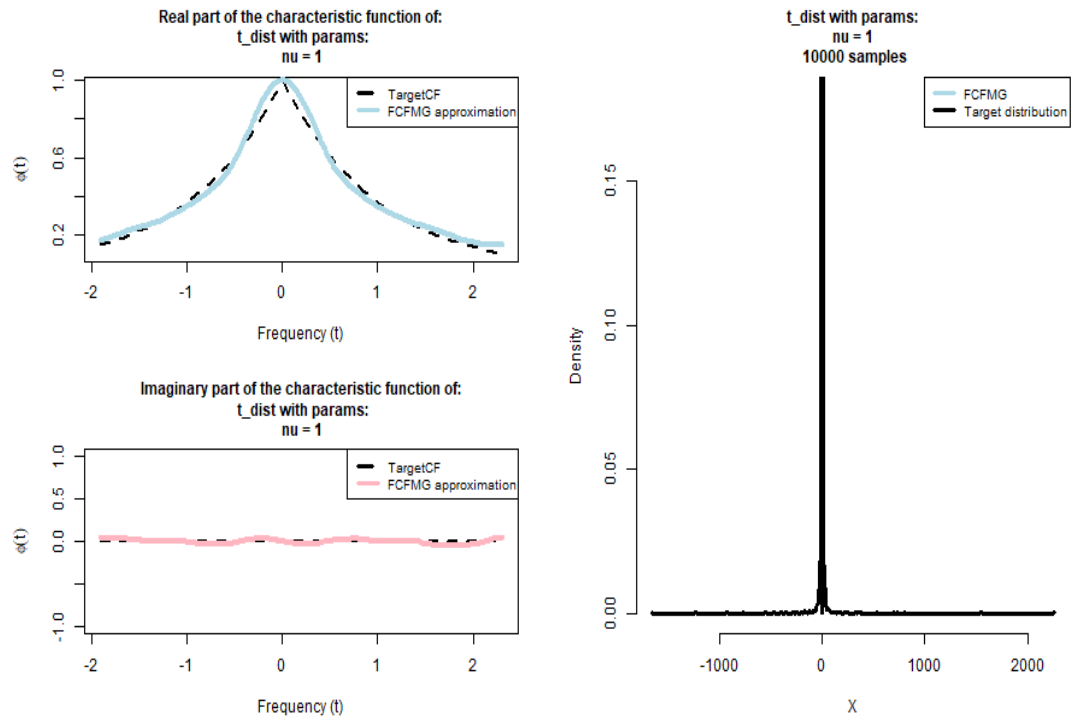


Figure 4.21: Standard Cauchy (T) Distribution

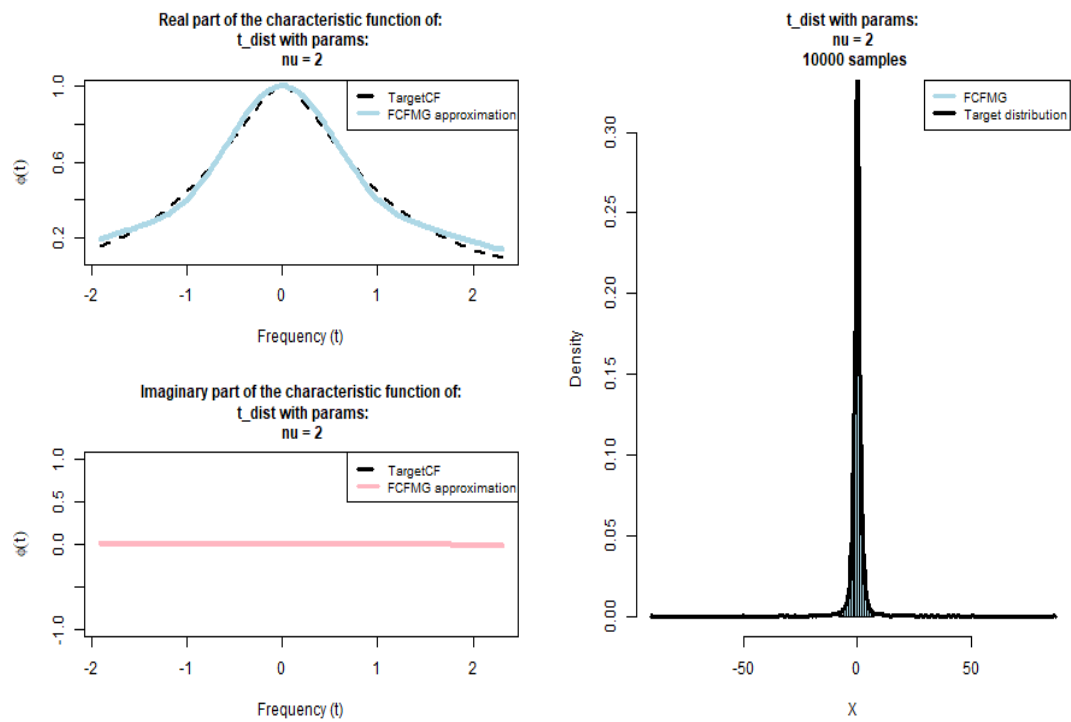


Figure 4.22: Infinite Variance T Distribution

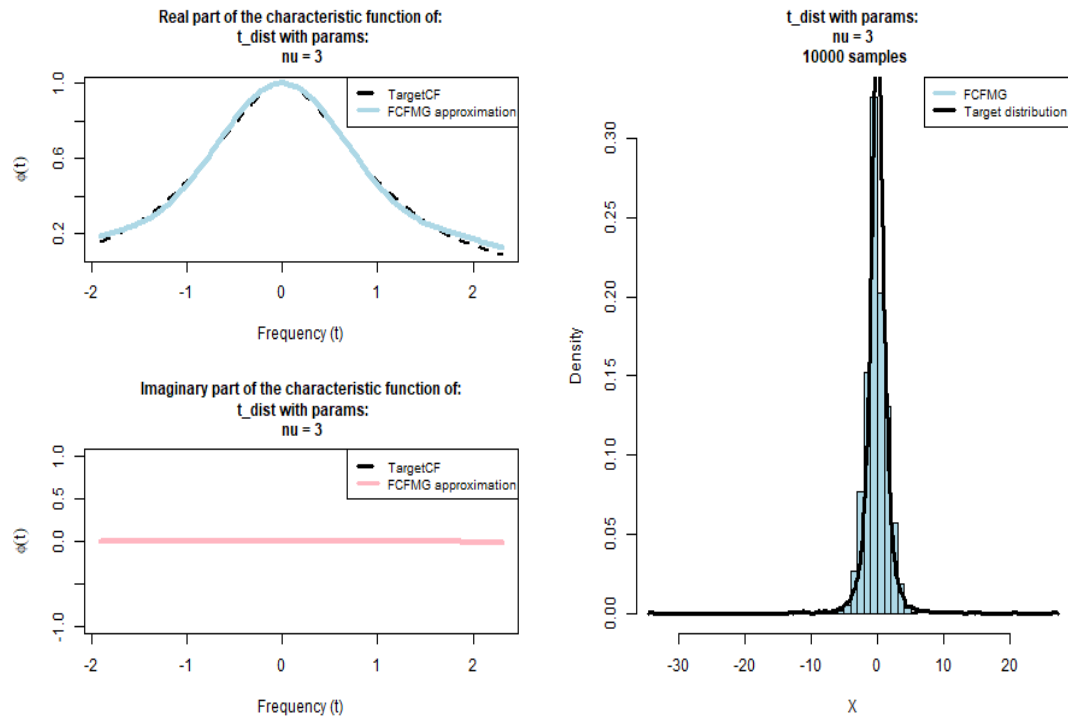


Figure 4.23: Finite Variance Heavy Tailed T Distribution

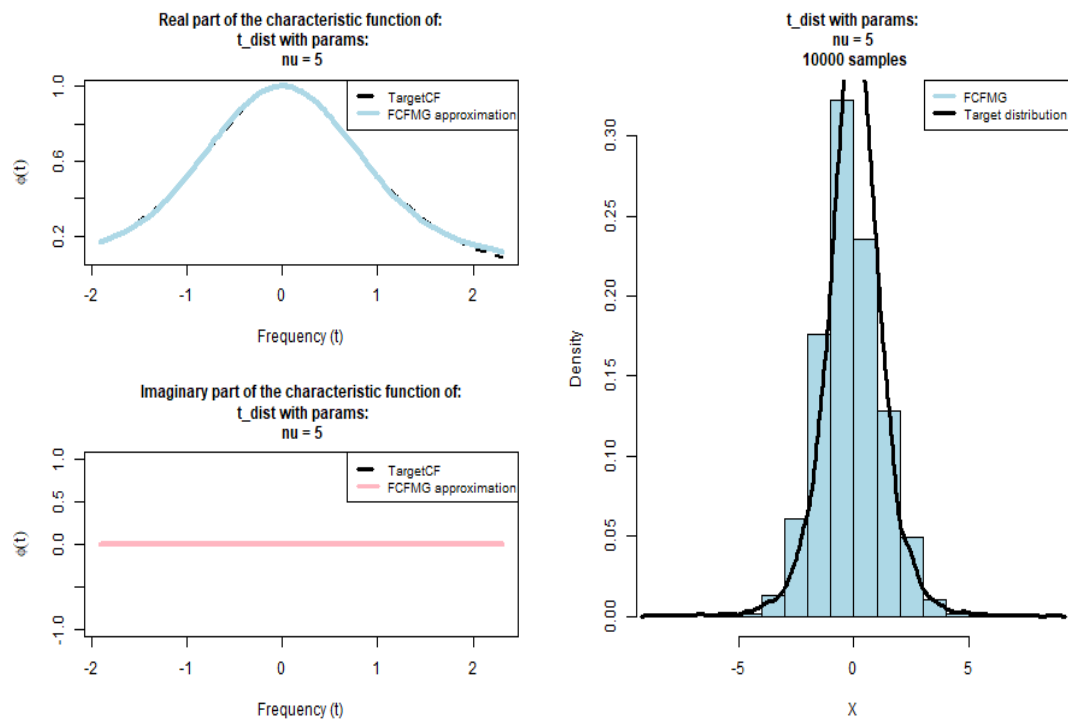


Figure 4.24: Moderate Tailed T Distribution

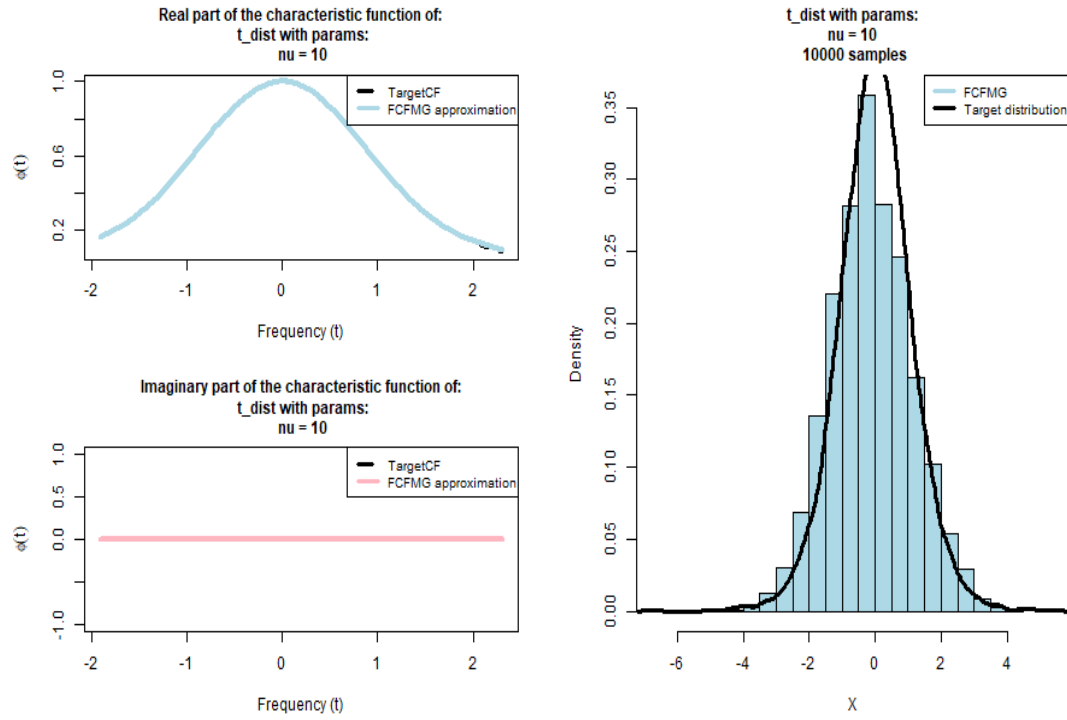


Figure 4.25: Approximately Normal T Distribution

4.2.8 Cauchy Distributions

Table 4.6: Cauchy Distribution True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Standard	Mean	0.0	-0.2382	0.2382	0.2202
(location = 0, scale = 1)	Variance	1.0	1.3361	-0.3361	
Narrow Peak	Mean	0.0	0.0093	-0.0093	0.091
(location = 0, scale = 0.5)	Variance	0.5	0.5755	-0.0755	
Wide Peak	Mean	0.0	-0.2966	0.2966	0.2485
(location = 0, scale = 2)	Variance	2.0	2.4308	-0.4308	
Shifted	Mean	10.0	25.1114	-15.1114	43.6228
(location = 10, scale = 1)	Variance	1.0	9.3513	-8.3513	
Shifted Heavy Tailed	Mean	10.0	9.3046	0.6954	0.2
(location = 10, scale = 2)	Variance	2.0	2.9099	-0.9099	

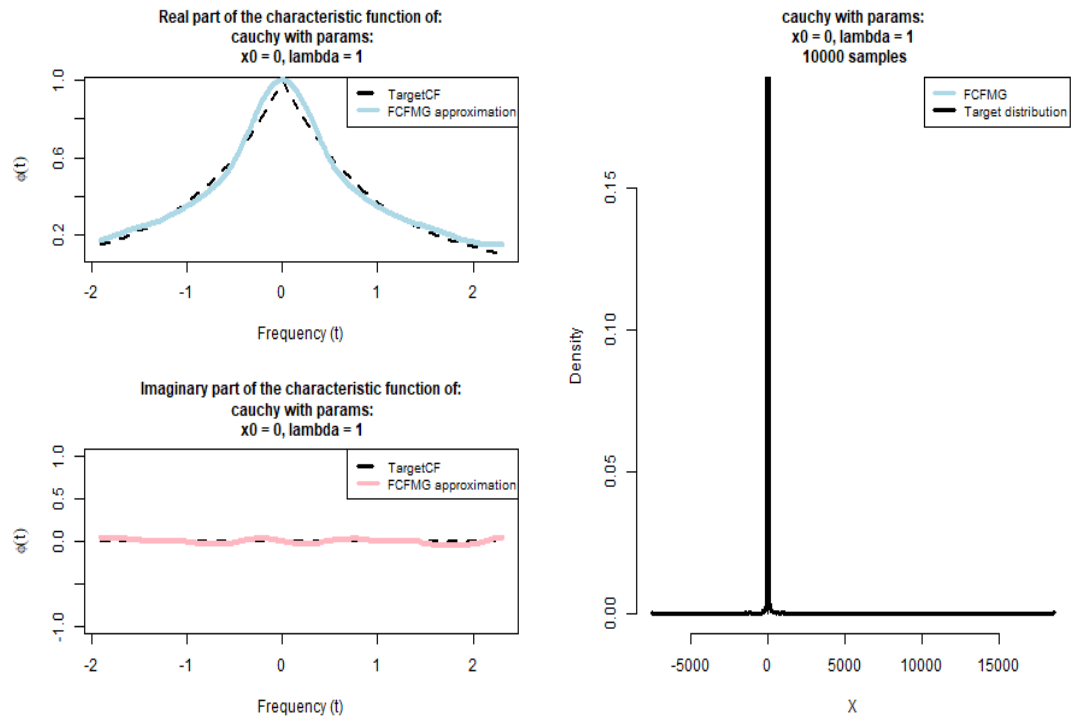


Figure 4.26: Standard Cauchy Distribution

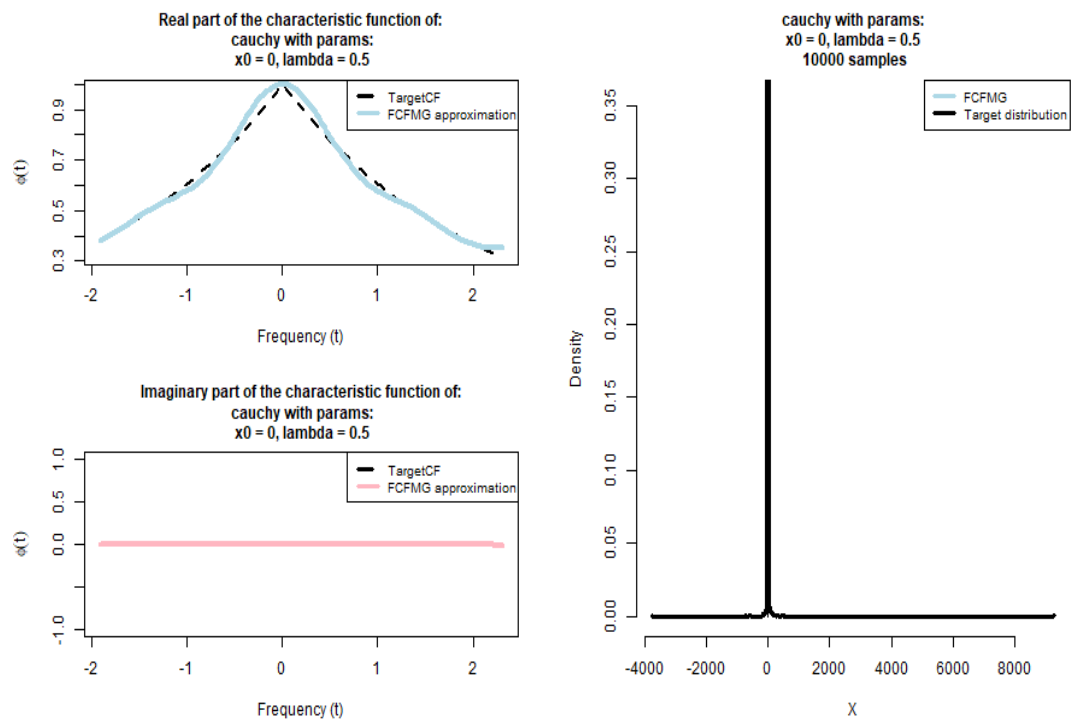


Figure 4.27: Narrow Peak Cauchy Distribution

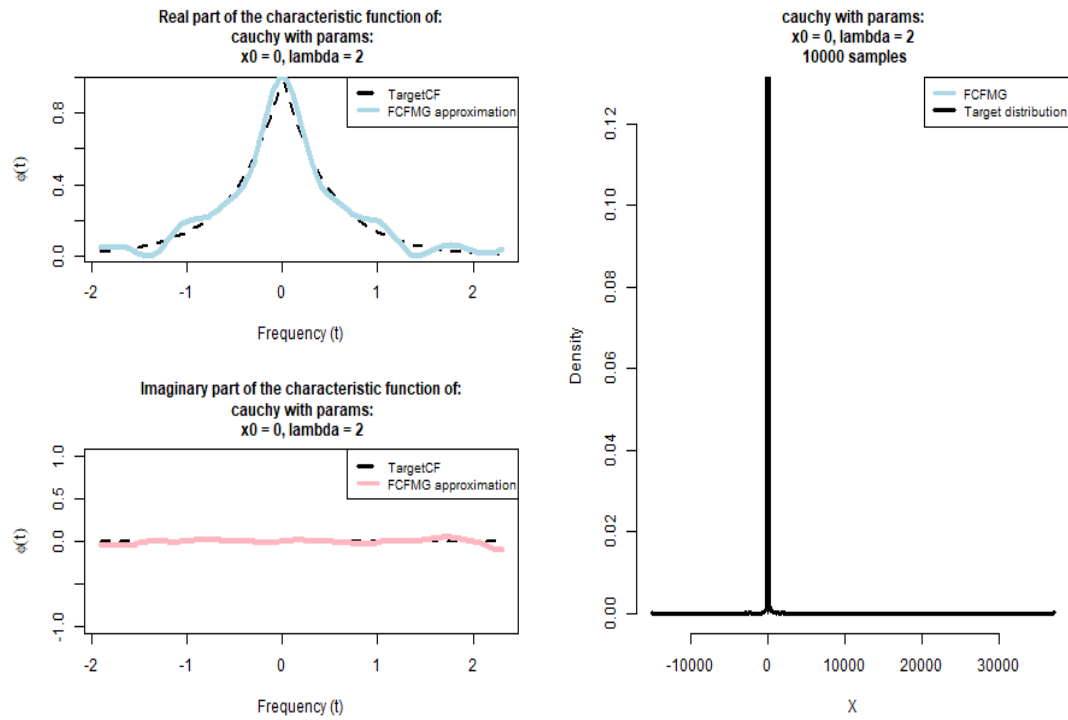


Figure 4.28: Wide Peak Cauchy Distribution

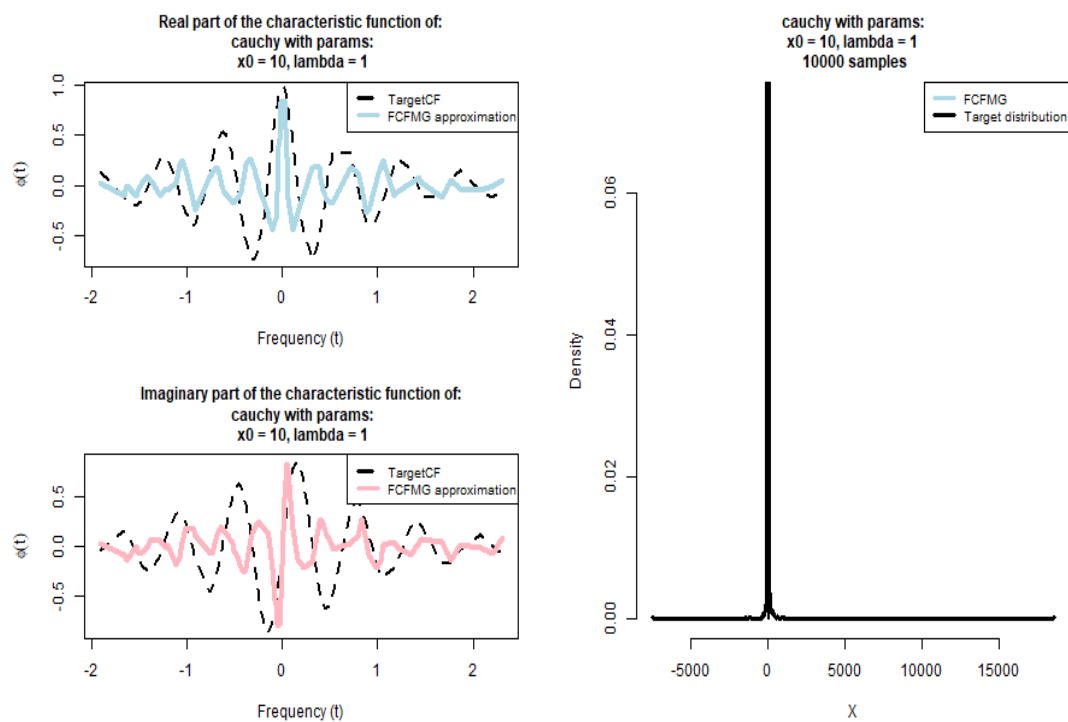


Figure 4.29: Shifted Cauchy Distribution

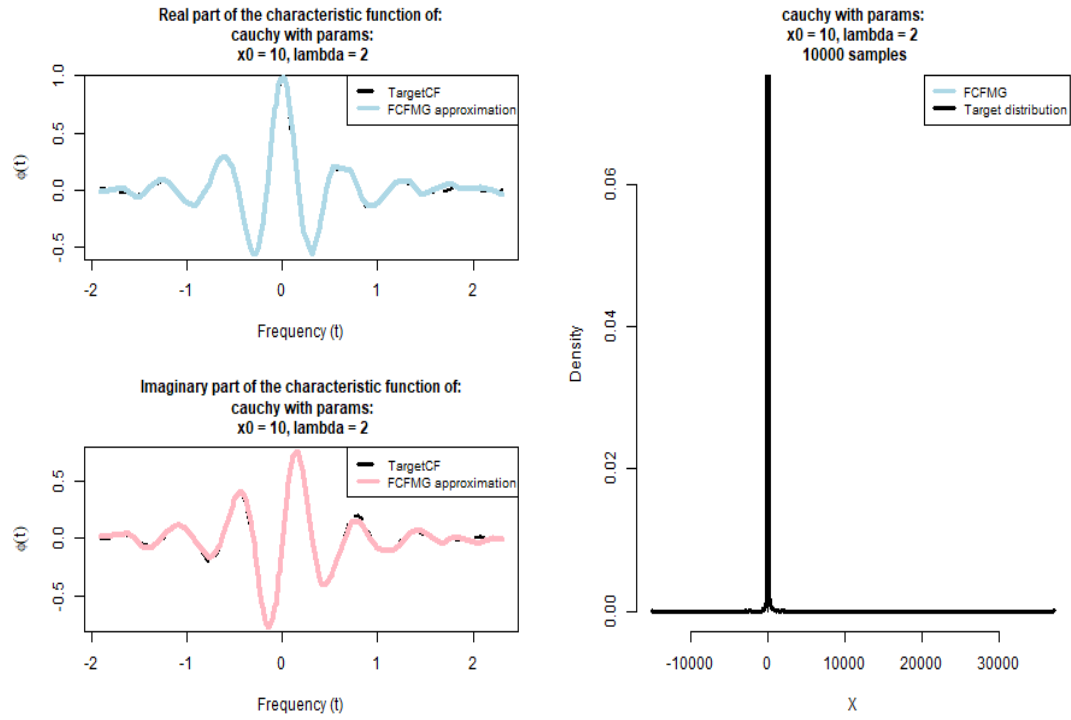


Figure 4.30: Shifted Heavy Tailed Cauchy Distribution

4.2.9 Normal Inverse Gaussian Distributions

Table 4.7: NIG True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Heavy Tailed Symmetric	Mean	0.0000	-0.1080	0.1080	0.0074
(a = 1, b = 0, d = 1, m = 0)	Variance	1.0000	1.1299	-0.1299	
Right Skew	Mean	0.2582	0.1800	0.0782	0.0012
(a = 2, b = 0.5, d = 1, m = 0)	Variance	0.5508	0.6671	-0.1163	
Left Skew Heavy Tailed	Mean	-1.1633	-1.2976	0.1343	0.0029
(a = 1.5, b = -0.3, d = 0.8, m = -1)	Variance	0.5670	0.6549	-0.0879	
Large Spread	Mean	-15.0000	-14.9300	-0.0700	0.052
(a = 5, b = -3, d = 20, m = 0)	Variance	7.8125	13.3610	-5.5485	
Highly Right Skew	Mean	2.0647	1.9861	0.0786	0.002
(a = 10, b = 9, d = 1, m = 0)	Variance	1.2075	1.3581	-0.1506	

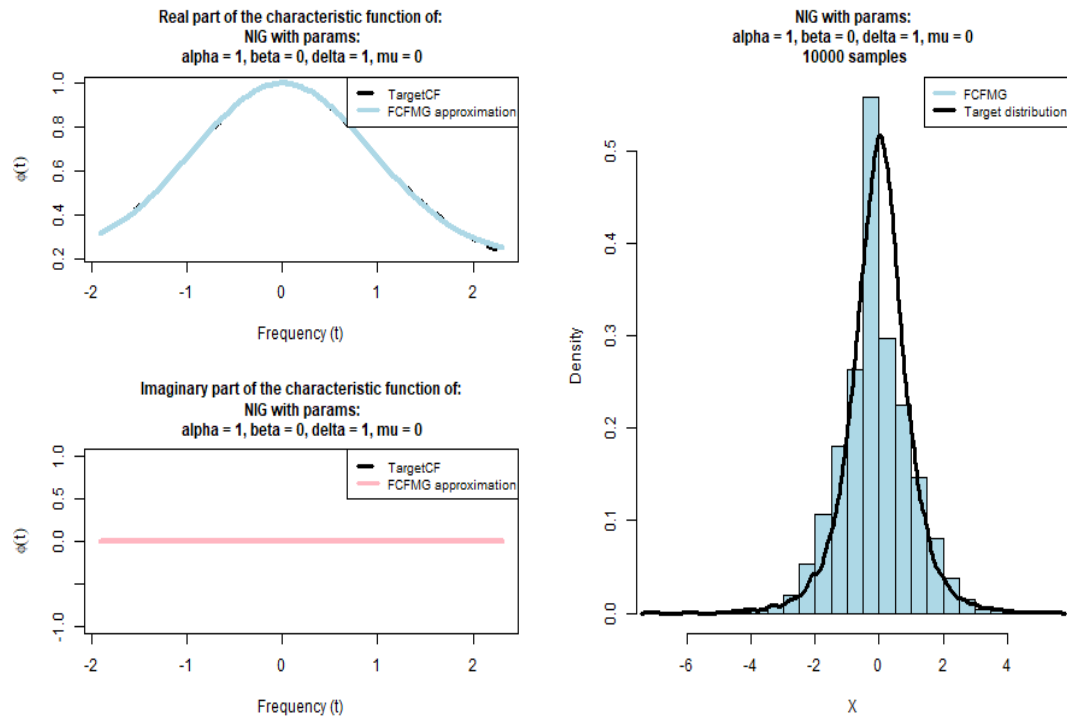


Figure 4.31: Heavy Tailed Symmetric NIG Distribution

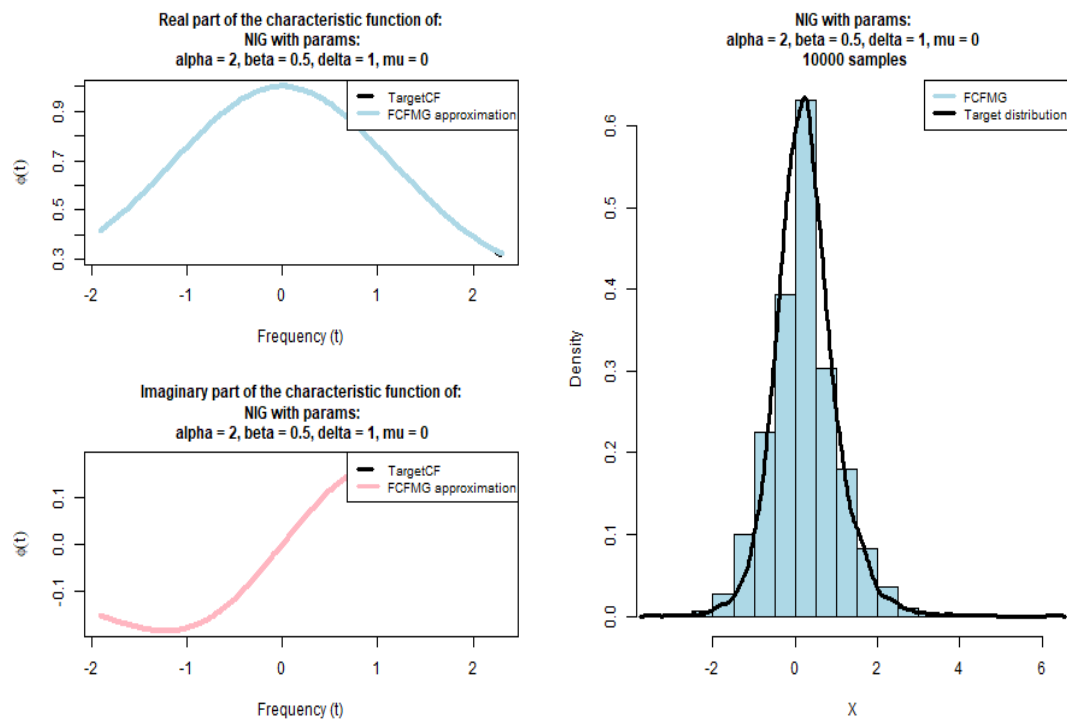


Figure 4.32: Right Skew NIG Distribution

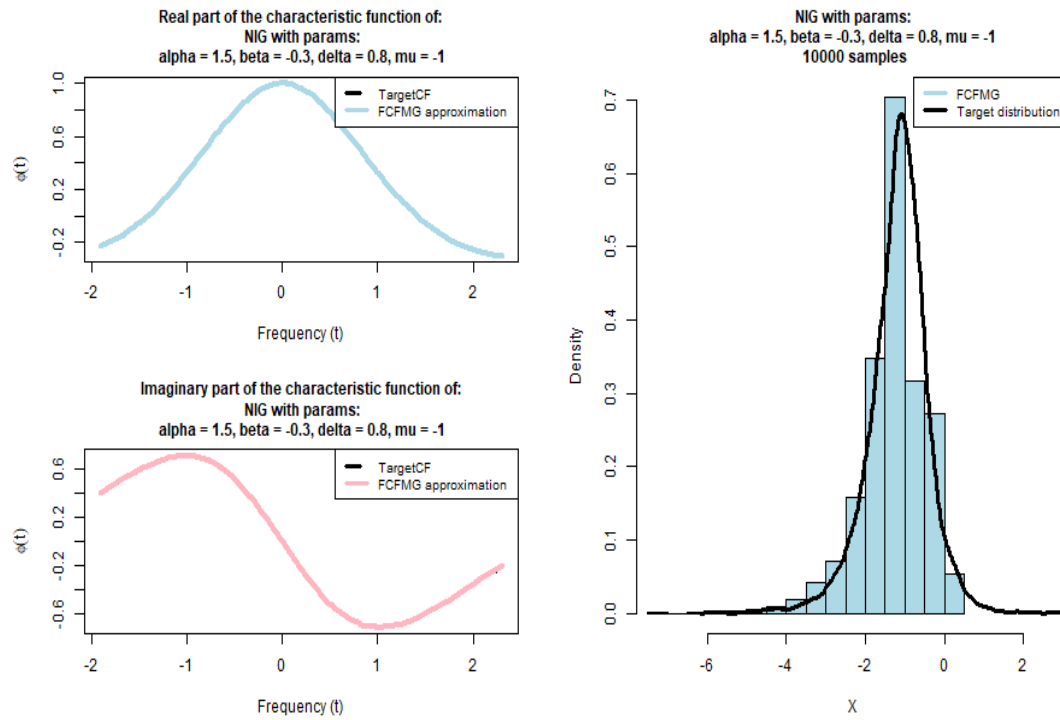


Figure 4.33: Left Skew Heavy Tailed NIG Distribution

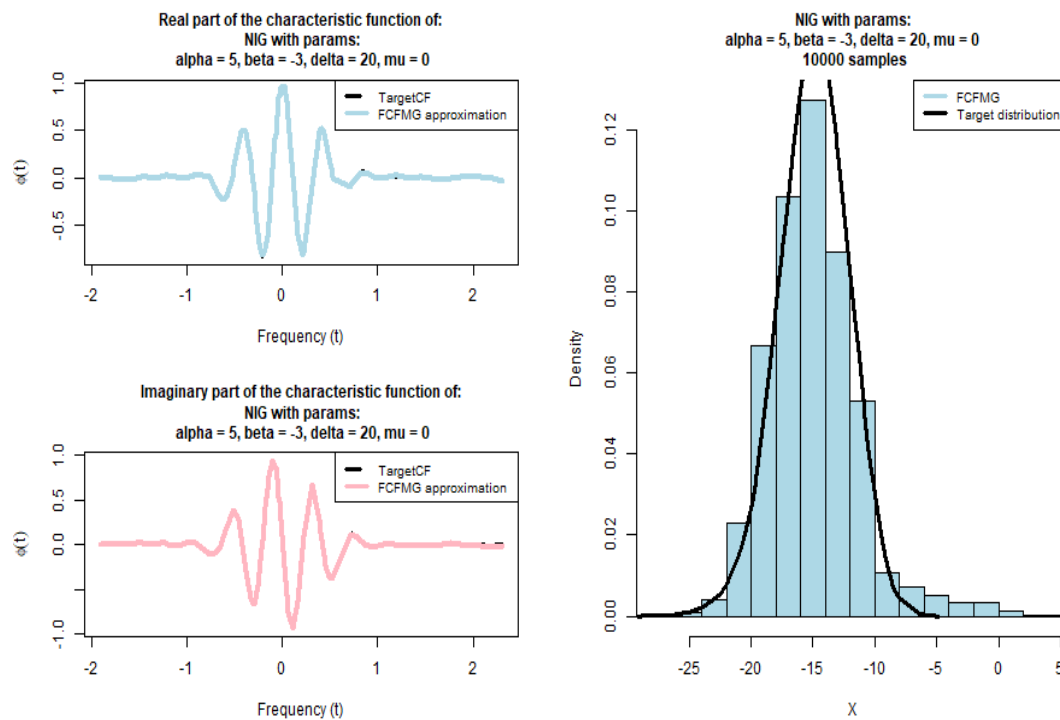


Figure 4.34: Large Spread NIG Distribution

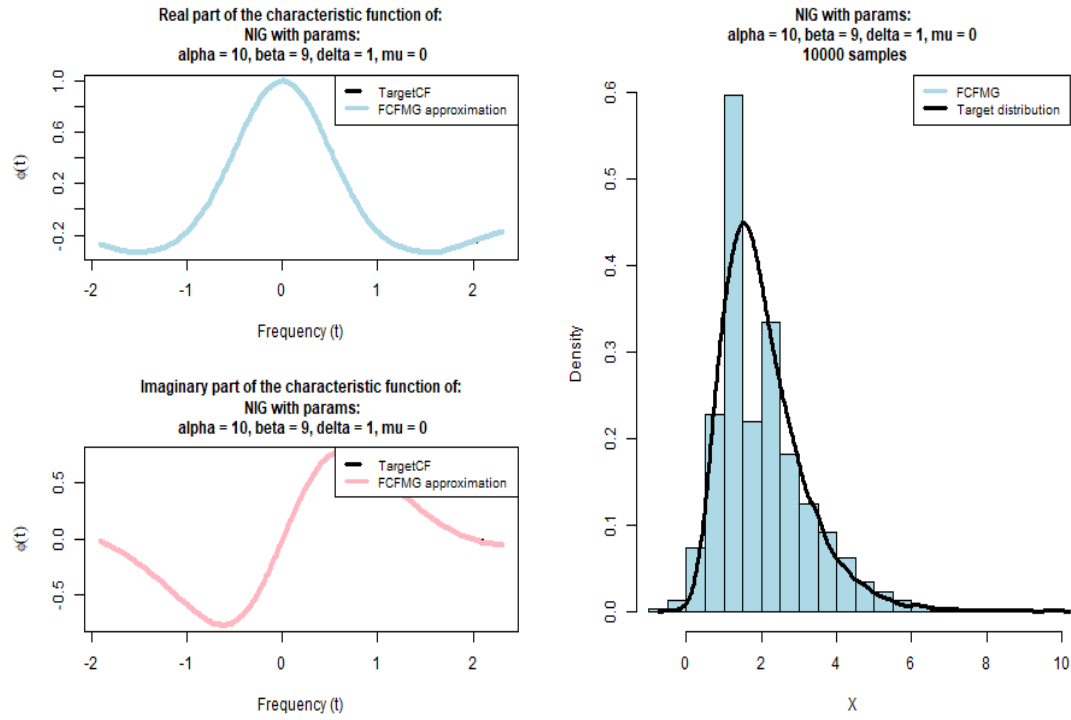


Figure 4.35: Highly Right Skew NIG Distribution

4.2.10 Equally Weighted Normal Inverse Gaussian Distribution Mixtures

Table 4.8: Equally Weighted NIG True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Moderate Symmetric Bimodal	Mean	-1.0911	-1.6124	0.5213	0.0067
(5, -2, 10, 0, 5, 2, 5, 0)	Variance	12.6627	13.9572	-1.2946	
Highly Skewed Bimodal	Mean	1.2113	0.7186	0.4928	0.0196
(1, 0.8, 1.5, -3, 3, -1.5, 1, 4)	Variance	8.6188	8.8750	-0.2562	
Highly Skewed Heavy Tailed Bimodal	Mean	-0.0589	-0.9307	0.8718	0.0679
(0.7, 0.6, 2, -5, 4, -3.5, 0.8, 3)	Variance	13.9369	9.9395	3.9973	
Moderate Symmetric	Mean	-0.6006	-0.7825	0.1819	0.0014
(1.5, 0.3, 1, -2, 2.5, -0.8, 1.2, 1)	Variance	2.0653	2.3786	-0.3133	
Highly Skewed Shifted Bimodal	Mean	1.5410	-9.0448	10.5858	0.1587
(0.8, 0.7, 2.5, -6, 5, -2, 1, 5)	Variance	23.0363	92.2524	-69.2162	

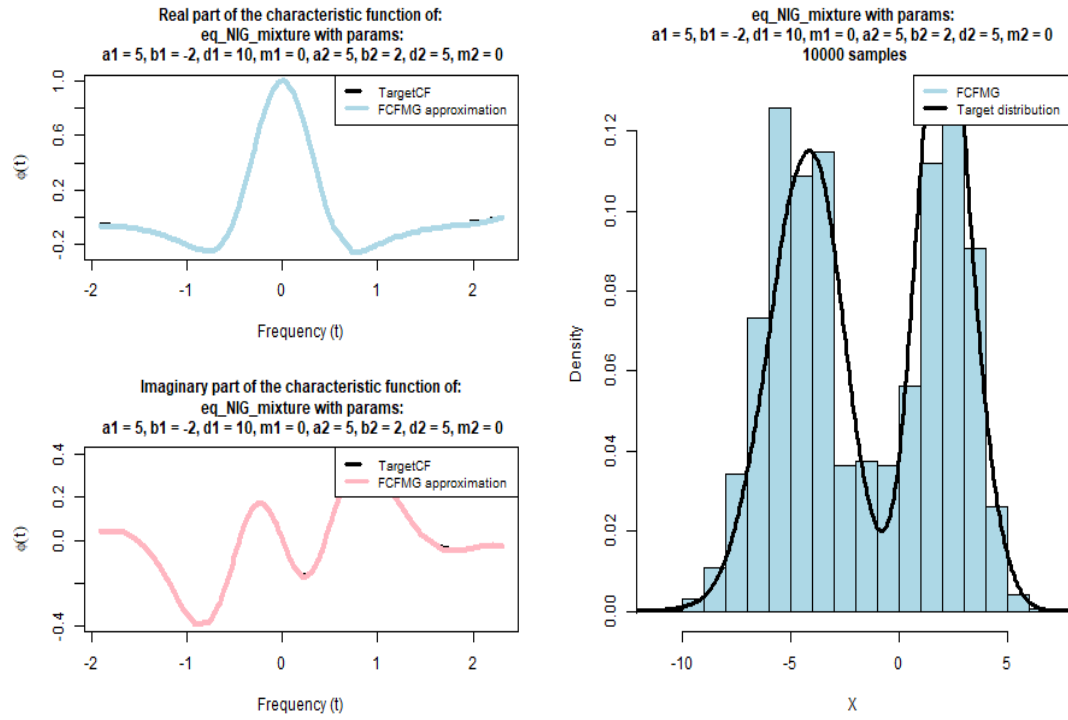


Figure 4.36: Moderate Symmetric Bimodal Equally Weighted NIG Distribution

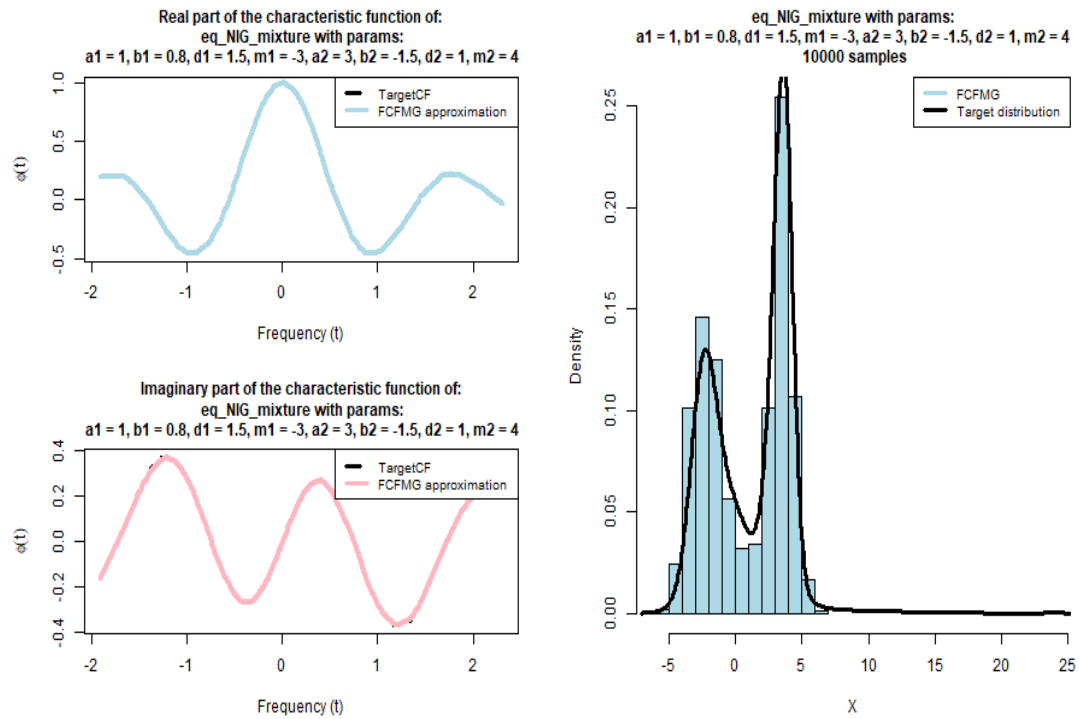


Figure 4.37: Highly Skewed Bimodal Equally Weighted NIG Distribution

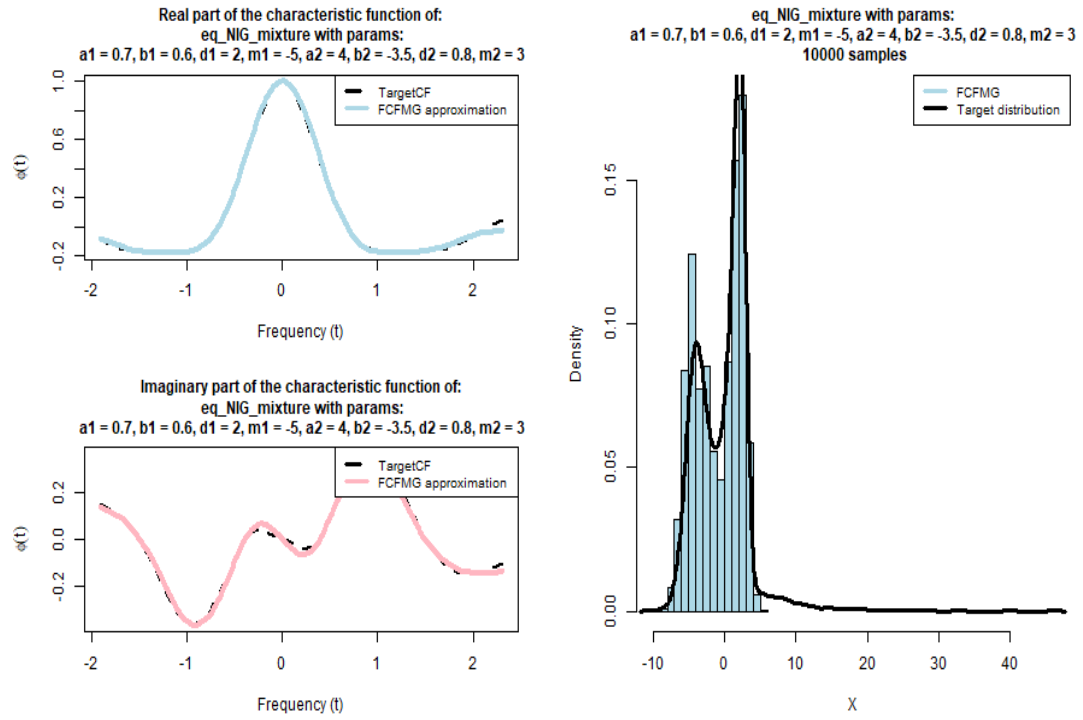


Figure 4.38: Highly Skewed Heavy Tailed Bimodal Equally Weighted NIG Distribution

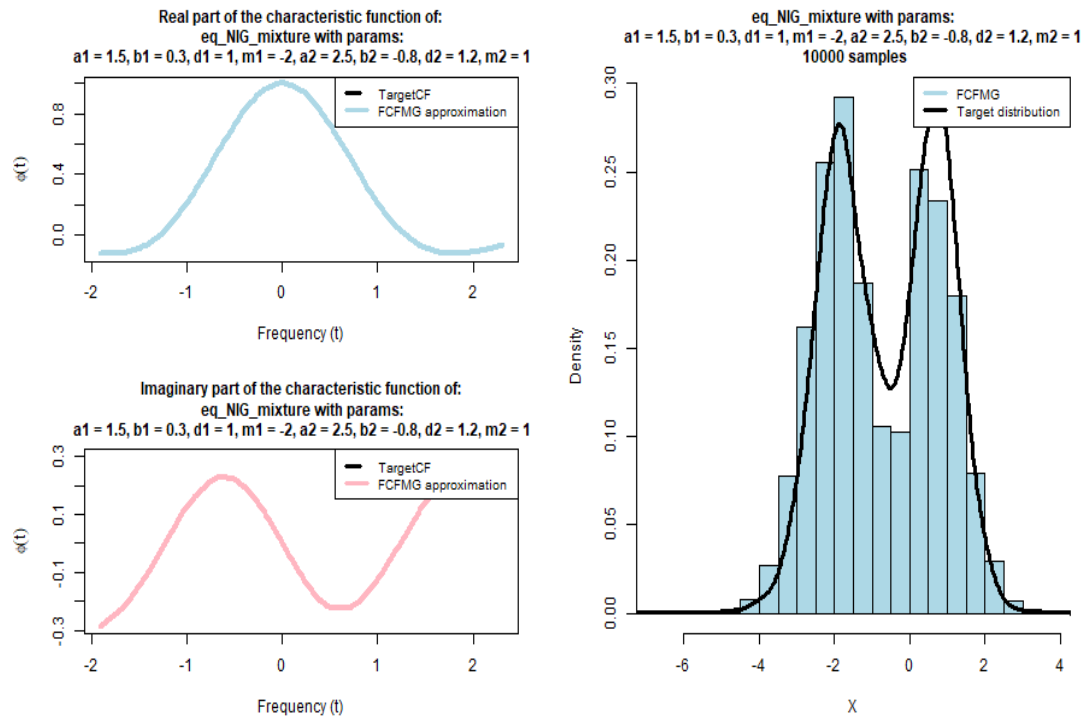


Figure 4.39: Moderate Symmetric Equally Weighted NIG Distribution

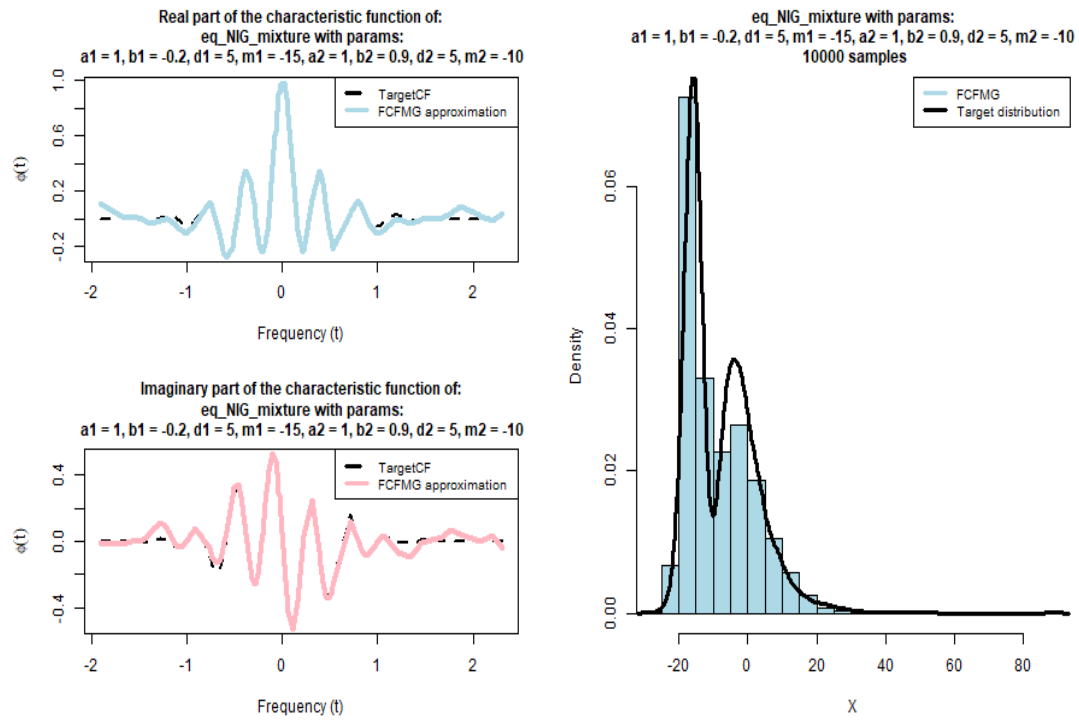


Figure 4.40: Highly Skewed Shifted Bimodal Equally Weighted NIG Distribution

4.2.11 Differently Seeded Simulations

Table 4.9: Selected Different Seeded True vs Simulated Results

Distribution	Statistic	True	Simulated	Error	Loss
Normal (seed = 4)	Mean	20.00	19.5971	0.4029	0.0061
(mu = 20, var = 10)	Variance	10.00	13.0288	-3.0288	
Gamma (seed = 2)	Mean	50.00	43.9858	6.0142	1.291
(alpha = 10, lambda = 0.2)	Variance	250.00	382.7541	-132.7541	
Chi (seed = 4)	Mean	50.00	-25.3050	75.3050	7.0222
(k = 50)	Variance	100.00	389.6430	-289.6430	
Exponential (seed = 2)	Mean	0.20	0.1569	0.0431	2e-04
(lambda = 5)	Variance	0.04	0.0333	0.0067	

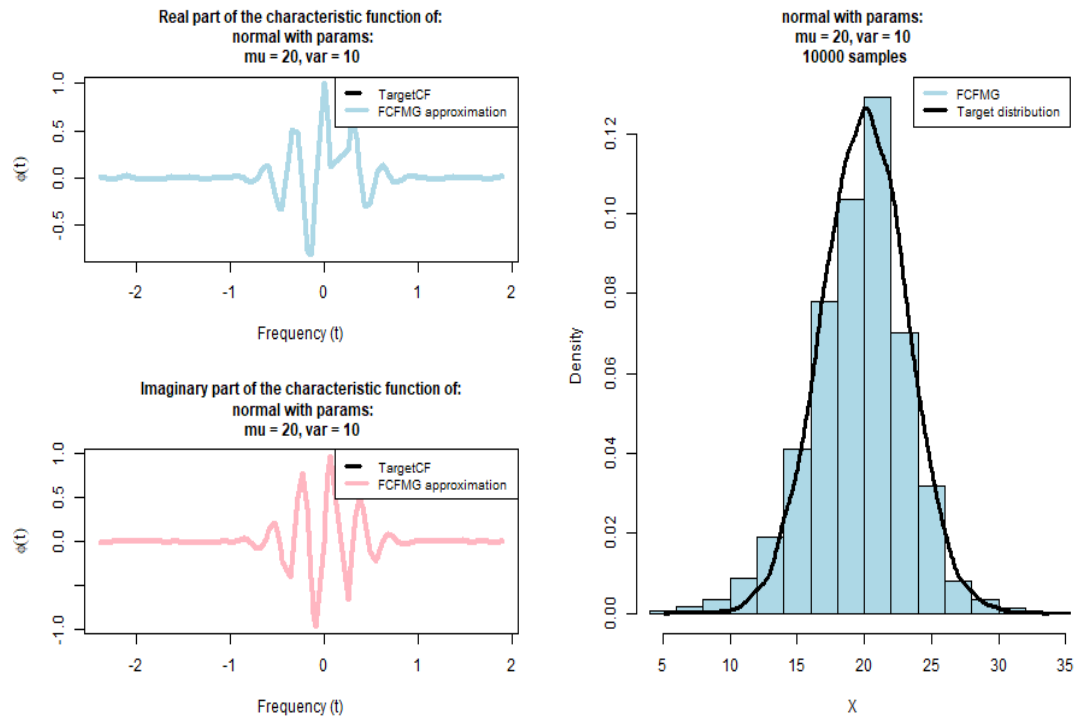


Figure 4.41: 20 Mean Normal Distribution (seed = 4)

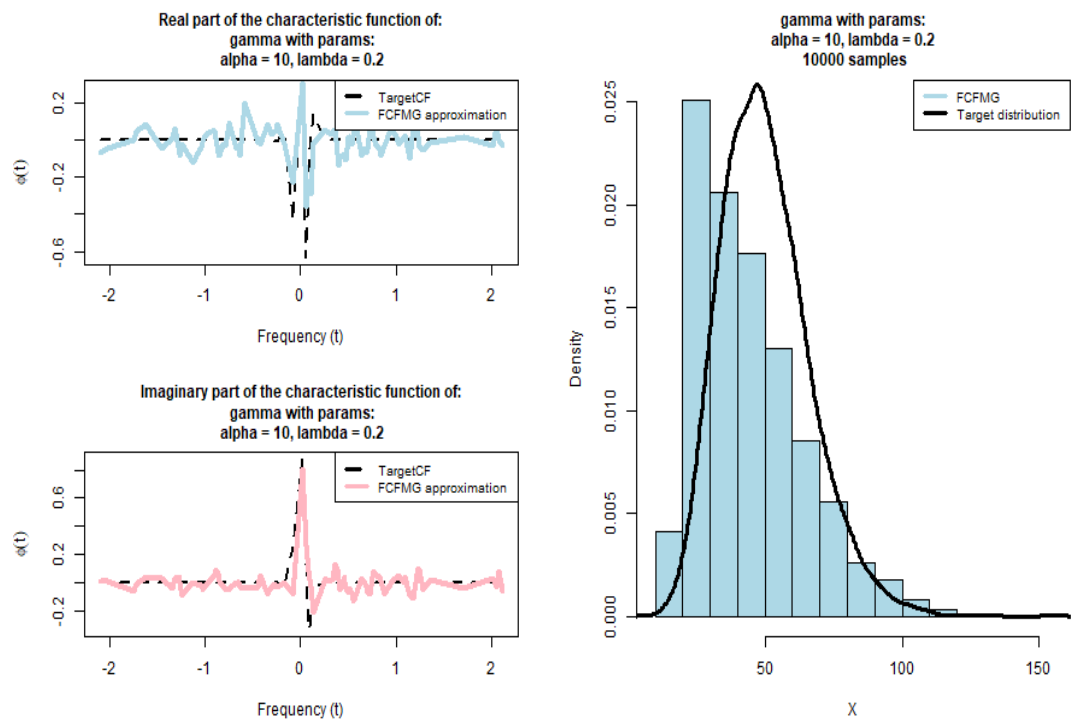


Figure 4.42: Large Mean Gamma Distribution (seed = 2)

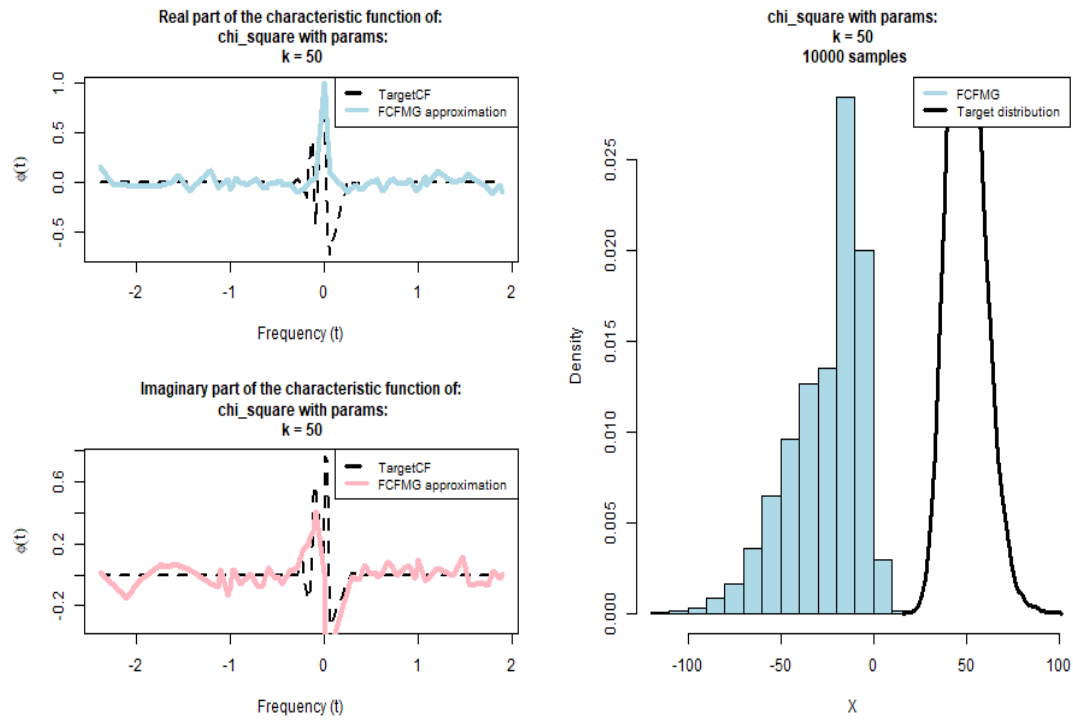


Figure 4.43: Large Mean Normal Approximating Chi-Square Distribution (seed = 4)

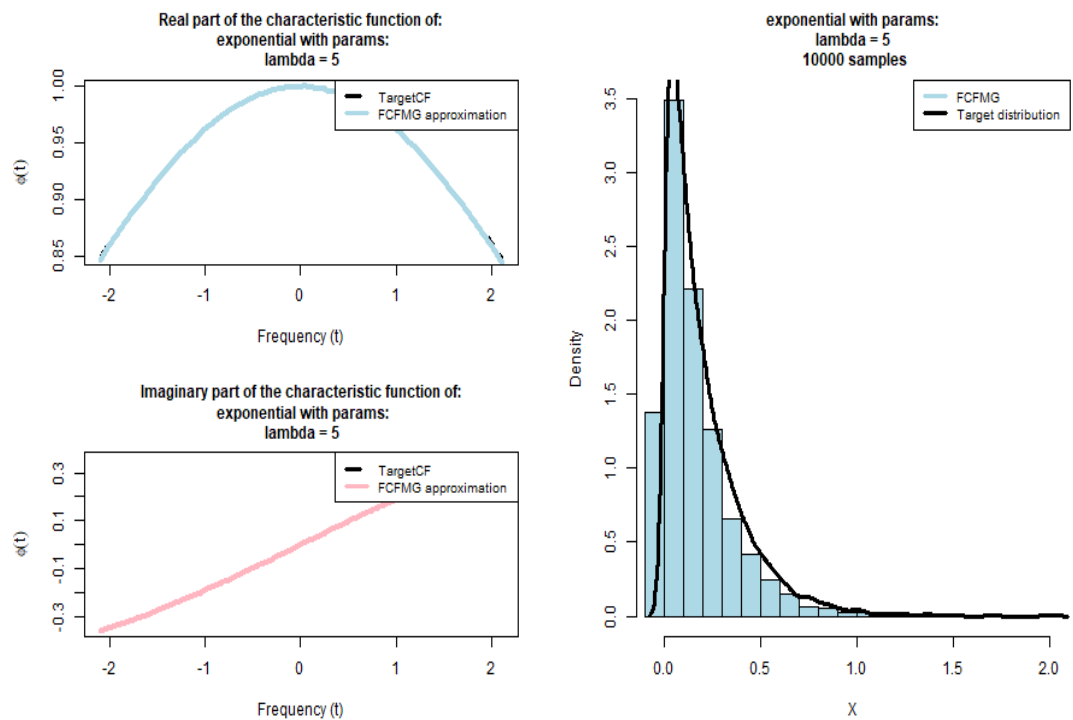


Figure 4.44: Short Tailed Exponential Distribution (seed = 2)

Chapter 5

Discussion and Conclusion

5.1 The Research Objective

The purpose of this paper was to construct a neural network sampler called a generator, that would be able to generate samples from a univariate target distribution by incorporating the characteristic function into the loss equation. Depending on the nature of the parameters used in the target distribution, the generator was able to find mappings in all except the Cauchy distribution. Importantly, figures 4.36 to 4.40 demonstrate that the generator is able to find mappings to mixture distributions with more than one mode present.

5.2 Evaluation of Results

The Cauchy Distribution and the “Transportation” Effect

As per table 4.6, the generator was able to match the parameters of the Cauchy distribution decently well, reproducing the location and scale parameters with small error (< 1) in four of the five cases. The issue however is with the aforementioned range of values simulated which can be seen in figures 4.26 to 4.30, represented by a complete absence of an identifiable estimated samples histogram.

The generator’s difficulty in reproducing the density is not surprising however as the Cauchy distribution has undefined moments, they are infinite (H. Y. Lee, Park, & Kim, 2014). Intuitively, it is difficult for the generator to take small numbers centered at 0 and a standard deviation of 1 and transform them into numbers as large as those seen in the extreme ends of the tails of the Cauchy distribution.

The issue of transforming small numbers into large numbers arises again when trying to generate samples from distributions with large means. Figures 4.5, 4.10, and 4.15 demonstrate a sort of *transportation* effect, where it can be seen how the generator is trying to transport the starting sample values from around 0 to the mean of the target at 50. It should be noted that given enough passes, the generator can converge, but it will take hundreds of thousands of passes to do this.

The transportation effect appears in almost every case where the mean of a target distribution is large. This issue is not severe however since, according to Soch et al. (2024),

the mean is a location parameter, meaning it can be shifted without changing the fundamental structure of a particular distribution such as the skewness or tail-heaviness which is what is ideally sought after in distribution reconstruction. What this means is that if a distribution has a large mean, the generator can reconstruct it around 0 and then shift all estimated samples by the mean to the desired location. This is the same as subtracting the mean from the target and adding it later to the estimated samples, thereby solving the problem.

The difficulty in reconstructing large-mean distributions comes from the form in which the characteristic function takes. As the mean gets large and the distribution shifts along the real line, the characteristic function becomes increasingly oscillatory around 0 in frequency space. This behaviour makes it increasingly difficult for the generator to approximate the target distribution, and a deeper network (more layers) might be necessary to compute its corresponding characteristic function. Increasing the network depth will come at increased computational cost and hence the idea of subtracting the mean.

The process of subtracting the mean can be extended by standardising the target distribution by subtracting the mean and dividing by the standard deviation. Once the generator has produced standardised sample estimates, the transformation is reversed on those generated samples. The way this method would be implemented in the generator is by replacing the true characteristic function with a standardised counter part. According to Heckert & Filliben (2003), this process is commonly used in statistical software and would hence fit conceptually with the use case of the generator of this paper.

5.3 Implications and Applications of Results

From a practical perspective, using a generator as described in this paper would be overkill for univariate distribution estimation/reconstruction. It would generally be easier to draw random samples from already existing algorithms that do not require a training cycle. Although the benefits of batch sample generation exist in the univariate case, it is debatable whether the trade off between accuracy and speed is worth considering given that traditional sampling algorithms produce true samples near instantly even if in a one-at-a-time scheme.

The only practical univariate use case for a generator would be if the characteristic function were simple enough, and the probability density and distribution functions were complex enough, or unavailable. This would warrant introducing a training cycle into the sample generation process, assuming that the true characteristic function is known and being used in the loss.

Two empirical characteristic functions could also have been used in the loss equation instead of a empirical/true combination. This would require a change in the setup of the generation process but would enable a sort of non-parametric distribution reconstruction algorithm akin to that of kernel density estimation. For example, suppose 25 samples are collected and nothing is known about the population distribution they came from. Theoretically, it would be possible to use those 25 samples to reconstruct a empirical characteristic function of the population distribution, and use this as an estimate of the

target characteristic function. The generator would then proceed as usual and produce estimated samples from an estimated distribution.

The proper application of a characteristic function matching generator however still theoretically revolves around multivariate sample generation. The price paid in terms of having a training cycle is quickly accounted for if the curse of dimensionality as mentioned by Li & Bickel (2005) can be adequately addressed. This is because one-at-a-time samplers would face exponential cost in producing samples in higher dimensions, especially if those higher dimensional spaces have complex “terrain”, or multimodality. The issue however is that high dimensional sample generation even via a neural network sampler would require intensive computing power in order to deal with large matrix operations (Gabbay, Lev Aharoni, & Schweitzer, 2022).

5.4 Strengths and Weaknesses of Analysis

5.4.1 Strengths

The common univariate distributions were all tested to demonstrate the application of the generator with first and second moment comparisons. Additionally, mixture models were also considered to test the generator’s ability to extend to more complex distributional shapes such as having more than one mode.

A fundamental aspect to this paper however was a thorough derivation of each mathematical object of the generator as well as investigating the statistical and mathematical properties of the characteristic function. These provide justification to entertain the idea of the generator as a whole instead of doing simple hyper-parameter tuning on pre-made models. Furthermore, since the generator was built from the ground up, every component can be accessed and modified as needs be, from the initialisation scheme of the parameters, to the optimisation algorithm itself.

Although not computed in this paper, all derivations for the generator had been conducted in the multivariate context, meaning that the theoretical foundations for an extended project into multivariate sample generation can be implemented by following already established theory.

5.4.2 Weaknesses

There is a host of different combinations of hyper-parameters that could have been investigated and tested to determine a better performing model. For example, the generator in this paper made use of weights initialised from a uniform distribution with bounds at -1 and 1, which caused issues of inconsistent performance in the form of random seed sensitivity. Other weight initialisation procedures such as drawing from a normal distribution could have been tested as mentioned in Dodge et al. (2020).

Another example of the effect of hyper-parameter choice on the performance of the generator could be seen in the range and selection method of the random frequencies at which to evaluate the characteristic function. As seen in 4.10, the oscillatory behaviour of the

approximation could be due to an aliasing issue. Aliasing would mean that the characteristic function is not being sampled enough, or not generating enough frequency values $\mathbf{T}$ at which to evaluate it. This is because a continuous function is being reconstructed over a discrete window of select values $\mathbf{T}$ (Olshausen, 2000).

The choice of how to draw $\mathbf{T}$ would also have been a valid investigation point, since a Gaussian kernel would have given preference to values of the characteristic function near 0 since this is where the $\mathbf{T}$ are clustered, providing more weighting to the global nature of the distribution but sacrificing the details of the higher order moments. Different sampling schemes or larger domains for $\mathbf{T}$ could have been tested.

The quality of the reconstructed distributions could also have been tested using more than just the first and second moments, however these were chosen out of ease and practicality. The third and fourth moments, representing skewness (asymmetry) and kurtosis (tail heaviness) respectively, could have been used as additional data points to check the quality of the generated sample estimates.

Lastly the optimisation of the generator code itself could be seen as a weakness of this approach, that is no optimisation was considered. The code for the generator was written in R using verbose language for the purposes of learning. It is thus possible to consider that the computational performance of the generator is held back in this regard. Examples of this can be seen in the use of for loops and large list structures in the code. The generator also employs batch gradient descent which according to Ruder (2016) can become computationally expensive with large quantities of data, possibly explaining the model's inability to handle multivariate data effectively even when supported by correct algebra.

Additionally and most importantly is the fact that the code is computed on the CPU only, that is no GPU accelerating has been used. This means that the generator is incredibly slow at training time and is too slow to use for higher dimensional objects since matrix multiplication tasks will not compute in tolerable time. The use of batch gradient descent potentially inflates this issue.

5.5 Conclusion

Univariate matching results show that it is possible to create a generator which maps standard normal samples to sample estimates of a target distribution via the characteristic function. The generator makes use of a relatively small architecture comprising of only 25 and 10 neurons in each hidden layer. Target distributions with large means and heavy tails proved to be the most difficult for the generator to reproduce as is discussed in the transportation effect. Using a generator for univariate sample generation does not provide much practical benefit when compared to traditional sampling methods, mainly because of the sensitivity to parameter initialisation and the necessity of a training cycle.

A primary limiting factor of the generator in this paper stems from its code structure. The use of the batch gradient optimisation rule in the back propagation algorithm, as well as being computed on the CPU instead of using GPU acceleration, meant only relatively small matrix operations could be computed. This meant that large input and frequency samples that would be used to produce more accurate approximations of the characteristic

function could not be computed. This fact also meant that multivariate testing was out of reach since the extra dimensions would require too much computational power.

Further research could be done on the generator's application to non-parametric methods. Instead of incorporating the empirical/true combination of characteristic functions in the loss equation, the empirical/empirical combination could be used to reconstruct univariate distributions without knowing anything about the random variable.

Appendix A

The R Code Structure

A standard feed-forward fully connected neural network design is used to create the generator, with an architecture that recursively calls a single layer function in order to build each subsequent layer of the network. Using this approach, it is possible to construct a network arbitrarily deep, layers in the network, and which has layers arbitrarily wide, neurons in a single layer, using the same layer function. The code for this network has been written in R. The generator contains five main functions with four auxiliary, quality of life functions.

An example function call is as follows:

```
FCFMGout <- FCFMG(samples = noise, # The X matrix.
                  the_t = the_t, # The T matrix.
                  characteristic_function = "normal",
                  params = list(mu = 20, var = 5),
                  architecture = list(25, 10, 1), # Three layer network.
                  functions = list("ReLU", "ReLU", "purelin"),
                  loss = "SEL", # Squared error loss.
                  lr = 0.001, # Learn rate.
                  passes = 5000, # Iterations of the network.
                  printLoss = TRUE, # Prints the value of the loss.
                  printGraph = FALSE, # Prints the topology.
                  printPlot = TRUE, # Plots the real and imaginary CFs.
                  printHisto = TRUE) # Plots synthetic vs true samples.
```

A.1 The ‘define_architecture’ Function

The ‘define_architecture’ function establishes the topology of the generator. It receives an instruction which stipulates how many layers and how many neurons in each layer the generator will have. This is done by creating a list of matrices whose dimensions correspond to the instruction and populates them with random numbers drawn from a uniform $[-1, 1]$ distribution as discussed in section 3.2.3. The bias vectors are handled in a similar fashion except that they are initially populated with zeros instead as discussed in 3.2.4. It returns a list with starting parameters, weights and biases for the generator.

Because the generator is built using recursive logic, the input or noise matrix $\mathbf{X}$ is treated like any other layer, hence it is appended to the architecture list within this function. This function is given by:

```
define_architecture <- function(samples, architecture){
  # Extracting the input dimension of the sample matrix
  architecture <- append(architecture, ncol(samples), after = 0)
  weights <- list()
  biases <- list()
  # List has all layers plus input, hence we go up to length - 1
  for (i in 1:(length(architecture)-1)){
    # Weights ~ neurons in next layer * neurons in current layer
    weights[[i]] <- matrix(runif(architecture[[i+1]]*architecture[[i]],
                                -1, 1),
                          nrow = architecture[[i+1]],
                          ncol = architecture[[i]],
                          byrow = TRUE)
    # Biases ~ neurons in next layer
    biases[[i]] <- matrix(0,
                          nrow = architecture[[i+1]],
                          ncol = 1,
                          byrow = TRUE)
  }
  return(list(weights = weights,
              biases = biases,
              architecture = architecture))
}
```

A.2 The ‘nn_layer’ Function

The ‘nn_layer’ is the recursively called layer function. It computes a linear combination of an input matrix $\mathbf{X}$, a weight matrix $\mathbf{W}$, and a broadcasted bias vector $\mathbf{B}$. After computing the linear combination $\mathbf{Z} = \mathbf{WX}' + \mathbf{B}$, it applies an activation function element-wise and transposes the result to obtain $\hat{\mathbf{Y}} = f(\mathbf{Z}')$. The reason why the result is transposed is because the result will be used as an input for another call of the same layer function, that is $\mathbf{X}_{\text{new}} := \hat{\mathbf{Y}}$. This function is given by:

```
nn_layer <- function(x, w, b, activation){
  # Broadcast dimensions
  B <- matrix(b, nrow = dim(w%*%t(x))[1], ncol = dim(w%*%t(x))[2])
  z <- w%*%t(x) + B # Pre-activation
  activation_function <- switch(activation,
                                sigmoid = function(x){
                                  return(1/(1+exp(-x)))
                                },
                                ReLU = function(x){
```

```

        return(ifelse((x > 0), x, 0))
    },
    purelin = function(x){
        return(x)
    },
    sin = function(x){
        return(sin(x))
    })
derivative_function <- switch(activation,
    sigmoid = function(x){
        f <- 1/(1+exp(-x))
        return(f*(1-f))
    },
    ReLU = function(x){
        return(ifelse((x > 0), 1, 0))
    },
    purelin = function(x){
        return(x/x)
    },
    sin = function(x){
        return(cos(x))
    })
a <- t(activation_function(z)) # Post-activation
return(list(estimates = a,
    samples = x,
    pre_activation = z,
    weights = w,
    biases = b,
    neuron_type = activation,
    derivative_function = derivative_function))
}

```

A.3 The ‘execute’ Function

The ‘execute’ function handles each forward pass and computes back propagation. For every iteration in the stipulated number of passes, the execute function pushes the input matrix $\mathbf{X}$ through the layers of the network by recursively calling the `nn_layer` function, and then calculates the derivatives necessary for back propagation thereafter. The derivatives are handled by first calculating a delta term which captures information of the previous derivatives since it is necessary to use the chain rule to push updates through each layer. These deltas are then used in computing the update rule which in this case is the gradient descent algorithm. For example:

$$\mathbf{W}_{\text{new}} := \mathbf{W}_{\text{old}} - \eta \delta \frac{\partial \mathbf{Z}}{\partial \mathbf{W}_{\text{old}}}$$

```

execute <- function(architecture,
                    samples,
                    weights,
                    biases,
                    targets,
                    the_t,
                    functions,
                    loss,
                    lr,
                    passes,
                    printGraph, printLoss, printPlot){
  loss_vector <- c()
  for (i in 1:passes){
    # Forward pass
    layers <- list()
    input <- samples
    for (j in 1:length(weights)){
      layers[[j]] <- nn_layer(input,
                              weights[[j]],
                              biases[[j]],
                              functions[[j]])
      input <- layers[[j]]$estimates
    }

    # Back propagation
    delta <- list()
    last_layer <- layers[[length(weights)]]
    # Most recent output from the last forward pass
    X <- last_layer$estimates

    TX_tr <- the_t%%t(X) # (m * n)
    col_of_ones <- matrix(1, nrow = n, ncol = 1)

    empCF_real <- (1/n)*cos(TX_tr)%%col_of_ones
    empCF_imaginary <- (1/n)*sin(TX_tr)%%col_of_ones
    empCF <- cbind(empCF_real, empCF_imaginary)

    ##### First derivative term
    dL_dempCF <- 2*(empCF - targets)
    dL_dempCF_bd <- array(dL_dempCF,
                          dim = c(m, 2, n, p))

    ##### Second derivative term
    sins <- sin(TX_tr)
    coses <- cos(TX_tr)
    sins_bd <- array(rep(sins, each = p), dim = c(m, n, p))
    coses_bd <- array(rep(coses, each = p), dim = c(m, n, p))
  }
}

```

```

the_t_bd <- array(rep(the_t, times = n), dim = c(m, n, p))

real_part <- -(1/n)*sins_bd*the_t_bd
imaginary_part <- (1/n)*coses_bd*the_t_bd

dempCF_dX <- array(0, dim = c(m, 2, n, p))
dempCF_dX[ , 1, , ] <- real_part
dempCF_dX[ , 2, , ] <- imaginary_part

dL_dX <- apply(dL_dempCF_bd*dempCF_dX, c(3, 4), sum)

#### Third derivative term
Z <- last_layer$pre_activation
dX_dZ <- last_layer$derivative_function(t(Z))

dL_dZ <- dL_dX*dX_dZ

delta[[length(weights)]] <- dL_dZ

for (j in length(weights):1){
  if (j != length(weights)){
    Zj <- layers[[j]]$pre_activation
    Zprime <- t(layers[[j]]$derivative_function(Zj))
    delta[[j]] <- (delta[[j+1]]**weights[[j+1]])*Zprime
  }
  if (j == 1){
    grad <- t(delta[[j]])**samples
    weights[[j]] <- weights[[j]] - lr*grad
  }else{
    grad <- t(delta[[j]])**layers[[j-1]]$estimates
    weights[[j]] <- weights[[j]] - lr*grad
  }
  grad <- t(delta[[j]])**matrix(1, nrow = ncol(t(delta[[j]])))
  biases[[j]] <- biases[[j]] - lr*grad
}

currentloss <- switch(loss,
                      "SEL" = sum((targets - empCF)^2))
loss_vector <- append(loss_vector, currentloss)

# Printing passes
if ((printLoss == TRUE) & (i %% 100 == 0)){
  cat("Pass:", i, "Loss:", currentloss, "\n")
}

# Calling graph
if ((printGraph == TRUE) & (i %% 1000 == 0)){
  graph(weights, biases, architecture)
}

```



```

    }
    # Plotter
    if((printPlot == TRUE) & (i %% 100 == 0)){
        plotter(the_t, targets, characteristic_function, params, empCF)
    }
}

# Extract final weights and biases
list_of_weights <- list()
list_of_biases <- list()
for (i in 1:length(layers)){
    list_of_weights[[i]] <- layers[[i]]$weights
    list_of_biases[[i]] <- layers[[i]]$biases
}

return(list(targets = targets,
            empCF = empCF,
            the_t = the_t,
            final_estimates = layers[[length(weights)]]$estimates,
            final_weights = list_of_weights,
            final_biases = list_of_biases,
            architecture = architecture,
            functions = functions,
            layers = layers,
            lossvals = loss_vector))
}

```

A.4 The ‘push’ and ‘generate_samples’ Functions

The ‘push’ and ‘generate_samples’ functions work in tandem. The push function takes an input $\mathbf{X}$ and pushes it through the generator, transforming it into $\hat{\mathbf{Y}}$ as seen in section 3.2.6 in equation 3.2. The generate_samples function randomly draws standard normal noise and uses the push function to feed this noise through the generator. These two functions are what is used to produce the finished synthetic samples. These functions are given by:

```

push <- function(samples, weights, biases, functions){
    pushedlayers <- list()
    input <- samples
    for (j in 1:length(weights)){
        pushedlayers[[j]] <- nn_layer(input,
                                     weights[[j]],
                                     biases[[j]],
                                     functions[[j]])
        input <- pushedlayers[[j]]$estimates
    }
}

```

```

    return(pushlayers[[length(weights)]]$estimates)
}

generate_samples <- function(N, n, p, weights, biases, functions){
  samples_tensor <- array(NA, dim = c(n, p, N))
  for (i in 1:N){
    new_noise <- matrix(rnorm(n*p),
                        nrow = n,
                        ncol = p)
    samples_tensor[, , i] <- push(new_noise,
                                weights,
                                biases,
                                functions)
  }
  return(samples_tensor)
}

```

A.5 The Auxiliary Functions

The Auxiliary functions are a group of plotting, graphing, and target computation functions. They include a plotter which reproduces the real and imaginary parts of the characteristic function matching process after a number of iterations, a grapher which is able to produce a map of the topology similar to figure 3.3, and histogram which overlays true samples onto the synthetic samples in a univariate case. The target computation function automatically computes the target matrix depending on the chosen characteristic function. These functions are given by:

```

# Characteristic function choice and compute targets logic
# -----
compute_targets <- function(characteristic_function, the_t, params){
  CF <- switch(characteristic_function,
    normal = function(t, mu, var){
      return(exp(1i*t*mu-(1/2)*t^2*var))
    },
    gamma = function(t, alpha, lambda){
      return((1-(1i*(t/lambda)))^(-alpha))
    },
    chi_square = function(t, k){
      alpha = k/2
      lambda = 1/2
      return((1-(1i*(t/lambda)))^(-alpha))
    },
    exponential = function(t, lambda){
      alpha = 1
      return((1-(1i*(t/lambda)))^(-alpha))
    },

```

```

t_dist = function(t, nu){
  A <- (sqrt(nu)*abs(t))^(nu/2)
  B <- gamma(nu/2)*2^((nu/2)-1)
  return((A*besselK(sqrt(nu)*abs(t), nu/2))/B)
},
cauchy = function(t, x0, lambda){
  return(exp(x0*1i*t - lambda*abs(t)))
},
NIG = function(t, alpha, beta, delta, mu){
  gamma <- sqrt(alpha^2 - beta^2)
  sqrt_term <- sqrt(alpha^2 - (beta+1i*t)^2)
  return(exp(1i*mu*t+delta*(gamma-sqrt_term)))
},
eq_NIG_mixture = function(t,
                           a1,
                           b1,
                           d1,
                           m1,
                           a2,
                           b2,
                           d2,
                           m2){
  # Equal weighting
  gamma1 <- sqrt(a1^2 - b1^2)
  gamma2 <- sqrt(a2^2 - b2^2)
  sqrt_term1 <- sqrt(a1^2 - (b1+1i*t)^2)
  sqrt_term2 <- sqrt(a2^2 - (b2+1i*t)^2)
  NIG1 <- exp(1i*m1*t+d1*(gamma1-sqrt_term1))
  NIG2 <- exp(1i*m2*t+d2*(gamma2-sqrt_term2))
  return(0.5*NIG1 + 0.5*NIG2)
},
mvnormal = function(t, mu, var){
  return(exp(1i*t(t)%*%mu-(1/2)*t(t)%*%var%*%t))
},
eq_mvnormal_mixture = function(t, mu1, var1, mu2, var2){
  mvnormal1 <- exp(1i*t(t)%*%mu1-(1/2)*t(t)%*%var1%*%t)
  mvnormal2 <- exp(1i*t(t)%*%mu2-(1/2)*t(t)%*%var2%*%t)
  return(0.5*mvnormal1 + 0.5*mvnormal2)
})
CF_vals <- apply(the_t, 1, function(t){do.call(CF, c(list(t = t),
                                                    params))})

CF_real <- Re(CF_vals)
CF_imag <- Im(CF_vals)
targets <- matrix(cbind(CF_real, CF_imag),
                  nrow = nrow(the_t),
                  ncol = 2)

return(targets)

```

```

}
# -----

# Plot generator
# -----

plotter <- function(the_t,
                    targets,
                    characteristic_function,
                    params,
                    empCF){
  index <- order(the_t)
  frequency <- the_t[index]
  CF_real <- targets[, 1][index]
  empCF_real <- empCF[, 1][index]
  CF_imaginary <- targets[, 2][index]
  empCF_imaginary <- empCF[, 2][index]

  layout(matrix(2:1, nrow = 2))

  Imtext <- "Imaginary part of the characteristic function of:\n"
  Retext <- "Real part of the characteristic function of:\n"

  plot(frequency, CF_imaginary,
       main = paste(Imtext,
                    characteristic_function,
                    "with params:\n",
                    paste(names(params),
                          params,
                          sep = " = ",
                          collapse = ", ")),
       xlab = "Frequency (t)",
       ylab = expression(phi(t)),
       type = "l",
       col = "black",
       lwd = 2,
       lty = 2,
       cex.main = 1)
  lines(frequency, empCF_imaginary, col = "lightpink", lwd = 4)
  legend("topright",
        legend = c("TargetCF", "FCFMG approximation"),
        col = c("black", "lightpink"),
        lwd = c(3, 3),
        lty = c(2, 1),
        cex = 0.8)

  plot(frequency, CF_real,
       main = paste(Retext,

```

```

        characteristic_function,
        "with params:\n",
        paste(names(params),
              params,
              sep = " = ",
              collapse = ", ")),
    xlab = "Frequency (t)",
    ylab = expression(phi(t)),
    type = "l",
    col = "black",
    lwd = 2,
    lty = 2,
    cex.main = 1)
lines(frequency, empCF_real, col = "lightblue", lwd = 4)
legend("topright",
      legend = c("TargetCF", "FCFMG approximation"),
      col = c("black", "lightblue"),
      lwd = c(3, 3),
      lty = c(2, 1),
      cex = 0.8)
}
# -----

# Histogram generator
# -----
histo <- function(N, n, p,
                  weights,
                  biases,
                  functions,
                  characteristic_function,
                  params){

  require(GeneralizedHyperbolic)
  require(MASS)

  target_data <- switch(characteristic_function,
normal = do.call(rnorm,
                  list(n = N*n,
                      mean = params$mu,
                      sd = sqrt(params$var))),
gamma = do.call(rgamma,
                  list(n = N*n,
                      shape = params$alpha,
                      rate = params$lambda)),
exponential = do.call(rexp,
                      list(n = N*n,
                          rate = params$lambda)),

```

```

chi_square = do.call(rchisq,
                     list(n = N*n,
                          df = params$k)),
t_dist = do.call(rt,
                  list(n = N*n,
                       df = params$nu)),
cauchy = do.call(rcauchy,
                  list(n = N*n,
                       location = params$x0,
                       scale = params$lambda)),
NIG = do.call(rnig,
              list(n = N*n,
                   alpha = params$alpha,
                   beta = params$beta,
                   delta = params$delta,
                   mu = params$mu)),
eq_NIG_mixture = {
NIG1 <- do.call(rnig,
                list(n = round(N*n*0.5),
                     alpha = params$a1,
                     beta = params$b1,
                     delta = params$d1,
                     mu = params$m1))
NIG2 <- do.call(rnig,
                list(n = N*n-round(N*n*0.5),
                     alpha = params$a2,
                     beta = params$b2,
                     delta = params$d2,
                     mu = params$m2))
      mix <- sample(c(NIG1, NIG2))
      mix
    },
mvnormal = do.call(mvrnorm,
                    list(n = N*n,
                         mu = params$mu,
                         Sigma = params$var)),
eq_mvnormal_mixture = {
mvnormal1 <- do.call(mvrnorm,
                     list(n = round(N*n*0.5),
                          mu = params$mu1,
                          Sigma = params$var1))
mvnormal2 <- do.call(mvrnorm,
                     list(n = N*n - round(N*n*0.5),
                          mu = params$mu2,
                          Sigma = params$var2))
      mix <- rbind(mvnormal1, mvnormal2)
      mix <- mix[sample(N*n), ]

```

```

    })

simulated_data <- generate_samples(N, n, p,
                                   weights,
                                   biases,
                                   functions)

layout(matrix(1:1, nrow = 1))

if ((characteristic_function != "mvnormal") & (characteristic_function != "eq_mvn
hist(simulated_data,
     probability = TRUE,
     main = paste(characteristic_function,
                   "with params:\n",
                   paste(names(params),
                         params,
                         sep = " = ",
                         collapse = ", "),
                   "\n", N*n, "samples"),
     col = "lightblue",
     xlab = "X",
     cex.main = 1,
     xlim = range(simulated_data, target_data))
lines(density(target_data), col = "black", lwd = 3, lty = 1)
legend("topright",
      legend = c("FCFMG", "Target distribution"),
      col = c("lightblue",
              "black"),
      lty = c(1, 1),
      lwd = c(3, 3),
      cex = 0.8)
}else{
  sim_x1 <- c(simulated_data[, 1, ])
  sim_x2 <- c(simulated_data[, 2, ])
  simulated_data <- cbind(sim_x1, sim_x2)
  true_x1 <- target_data[, 1]
  true_x2 <- target_data[, 2]
  simDens <- kde2d(sim_x1, sim_x2)
  trueDens <- kde2d(true_x1, true_x2)
  plot(NA,
       xlim = range(c(sim_x1, true_x1)),
       ylim = range(c(sim_x2, true_x2)),
       xlab = "Dimension 1",
       ylab = "Dimension 2",
       main = paste(characteristic_function,
                     "with params:\n",
                     paste(names(params),

```

```

        params,
        sep = " = ",
        collapse = ", "),
        "\n", N*n, "samples"),
    cex.main = 1)
contour(simDens, col = "lightblue", add = TRUE, lwd = 2)
contour(trueDens, col = "black", add = TRUE)
legend("topright",
      legend = c("FCFMG", "Target distribution"),
      col = c("lightblue",
              "black"),
      lty = c(1, 1),
      lwd = c(3, 3),
      cex = 1)
}

if ((characteristic_function != "mvnormal") & (characteristic_function != "eq_mvn
if (characteristic_function != "cauchy"){
  simulated_data_mean <- mean(simulated_data)
  simulated_data_variance <- var(simulated_data)
  cat("Mean of FCFMG data: ",
      simulated_data_mean,
      "\nVariance of FCFMG data: ",
      simulated_data_variance)
}else{
  simulated_data_location <- median(simulated_data)
  simulated_data_scale <- IQR(simulated_data)/2
  cat("Location of FCFMG data: ",
      simulated_data_location,
      "\nScale of FCFMG data: ",
      simulated_data_scale)
}
}else{
  simulated_data_mean <- colMeans(simulated_data)
  simulated_data_variance <- var(simulated_data)
  cat("Mean of FCFMG data: ",
      simulated_data_mean,
      "\nVariance of FCFMG data: ",
      simulated_data_variance)
}
}
# -----

# Network graphing logic
# -----
graph <- function(weights, biases, architecture){
  require(DiagrammerR)

```



```

# Begin the string
string <- "digraph neural_net\n {graph [rankdir = LR]\n node [shape = circle, sty

total_elements <- list() # Keep track of elements in network
total_biases <- list() # Separate biases from neurons and inputs

# Inputs and Neurons (No biases yet)
neuron_num = 1 # n1 to nk where k is total num of neurons/biases
for (i in 1:length(architecture)){
  store_elements <- c() # Collects the "local elements"
  for (j in 1:architecture[[i]]){
    if (i == 1){ # This is the input
      element <- paste("x", j, sep = "")
      fill <- "lightblue"
    }else{
      element <- paste("n", neuron_num, sep = "")
      fill <- "lightpink"
      neuron_num = neuron_num + 1
    }
    string <- paste(string,
      element,
      " [label = '",
      element,
      "'", fillcolor = ",
      fill, "]\n", sep = "")
    store_elements <- c(store_elements, element)
  }
  total_elements[[i]] <- store_elements
}

# Biases [COMMENTED OUT, THEY MAKE THE PLOT UGLY]

# Input and neuron connections and weight values
for (i in 1:(length(total_elements)-1)){
  W <- round(weights[[i]], 2)
  for (j in 1:length(total_elements[[i]])){
    for (k in 1:length(total_elements[[i+1]])){
      # Remove weight vals
      string <- paste(string,
        total_elements[[i]][j],
        " -> ",
        total_elements[[i+1]][k], "\n", sep = "")
      # Add weight vals
      # BIASES COMMENTED OUT
    }
  }
}

```

```

}

# Bias connections [COMMENTED OUT, THEY MAKE THE PLOT UGLY]

string <- paste(string, "}")
print(grViz(string))
}

```

A.6 The Function Wrapper

The function wrapper is what the individual function components described above look like inside the generator, which automatically calls each component as it is required. At the bottom of the wrapper is the main section where function wide parameters are computed. This wrapper function is given by:

```

# Feed forward Characteristic Function Matching Generator
FCFMG <- function(samples,
                    the_t,
                    characteristic_function,
                    params,
                    architecture,
                    functions,
                    loss,
                    lr,
                    passes,
                    printLoss = FALSE,
                    printGraph = FALSE,
                    printPlot = FALSE,
                    printHisto = FALSE){

  # Define network architecture
  # -----
  define_architecture <- function(samples, architecture){}
  # -----

  # Define the layer logic
  # -----
  nn_layer <- function(x, w, b, activation){}
  # -----

  # Forward pass and back propagation logic
  # -----
  execute <- function(architecture,
                      samples,
                      weights,
                      biases,

```

```

        targets,
        the_t,
        functions,
        loss,
        lr,
        passes,
        printGraph, printLoss, printPlot){}

# -----

# Generate samples logic
# -----
push <- function(samples,
                  weights,
                  biases,
                  functions){}

generate_samples <- function(N, n, p,
                             weights,
                             biases,
                             functions){}

# -----

# Characteristic function choice and compute targets logic
# -----
compute_targets <- function(characteristic_function,
                             the_t,
                             params){}

# -----

# Plot generator
# -----
plotter <- function(the_t,
                    targets,
                    characteristic_function,
                    params,
                    empCF){}

# -----

# Histogram generator
# -----
histo <- function(N, n, p,
                  weights,
                  biases,
                  functions,
                  characteristic_function,
                  params){}

# -----

```

```

# Main
# -----
n <- nrow(samples)
p <- ncol(samples)
m <- nrow(the_t)
targets <- compute_targets(characteristic_function, the_t, params)
parameters <- define_architecture(samples, architecture)
architecture <- parameters$architecture
weights <- parameters$weights
biases <- parameters$biases

result <- execute(architecture = architecture,
                  samples = samples,
                  weights = weights,
                  biases = biases,
                  targets = targets,
                  the_t = the_t,
                  functions = functions,
                  loss = loss,
                  lr = lr,
                  passes = passes,
                  printGraph = printGraph,
                  printLoss = printLoss,
                  printPlot = printPlot)

if (printHisto == TRUE){
  histo(100, n, p,
        weights = result$final_weights,
        biases = result$final_biases,
        functions,
        characteristic_function,
        params)
}

# Independent sample generator function
return(list(result = result,
            generate_samples = function(N) generate_samples(N,n,p
                                                            result$final_weights
                                                            result$final_biases
                                                            result$functions)))
}
# -----

##### Testing and set up #####
set.seed(1)
noise_samples <- 100
freq_samples <- 100

```

```

p <- 1
noise <- matrix(rnorm(noise_samples*p),
               nrow = noise_samples,
               ncol = p)
the_t <- matrix(rnorm(freq_samples*p),
               nrow = freq_samples,
               ncol = p)
##### Testing and set up #####

FCFMGout <- FCFMG(samples = noise,
                  the_t = the_t,
                  characteristic_function = "normal",
                  params = list(mu = 20, var = 10),
                  architecture = list(25, 10, 1),
                  functions = list("ReLU", "ReLU", "purelin"),
                  loss = "SEL",
                  lr = 0.001,
                  passes = 5000,
                  printLoss = TRUE,
                  printGraph = FALSE,
                  printPlot = FALSE,
                  printHisto = TRUE)

```

References

- Alayón, J. et al. (2017). Normal multivariate probability density function as best-fit approximation in portfolio theory. *Economics Bulletin*, 37(4), 243–250. Retrieved from <https://www.accessecon.com/Pubs/EB/2017/Volume37/EB-17-V37-I4-P243.pdf>
- Al-Labadi, L., & Ly, A. (2024). A Short Note on Generating a Random Sample from Finite Mixture Distributions. *Axioms*, 13(5), 307. <https://doi.org/10.3390/axioms13050307>
- Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., ... Farhan, L. (2021). Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8(1), 53. <https://doi.org/10.1186/s40537-021-00444-8>
- Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. Retrieved from <https://arxiv.org/abs/1701.07875>
- Barndorff-Nielsen, O. E. (1997). Normal Inverse Gaussian Distributions and Stochastic Volatility Modelling. *Scandinavian Journal of Statistics*, 24(1), 1–13. <https://doi.org/10.1111/1467-9469.00045>
- Billingsley, P. (1995). *Probability and Measure* (3rd ed.). New York: Wiley.
- Binkowski, K., & Kozek, A. (2010). Uniform convergence of empirical characteristic functions in a complex domain with applications to option pricing. *Statistics & Probability Letters*, 80(5-6), 270–276. <https://doi.org/10.1016/j.spl.2009.10.017>
- Brück, F. (2025). Generative Neural Networks for Characteristic Functions. *Journal of Computational and Graphical Statistics*, 1–10. <https://doi.org/10.1080/10618600.2025.2455135>
- Bürkner, P.-C., Schmitt, M., & Radev, S. T. (2025). Simulations in statistical workflows. *arXiv Preprint arXiv:2503.24011*.
- Caffo, B. S., Booth, J. G., & Davison, A. (2002). Empirical supremum rejection sampling. *Biometrika*, 89(4), 745–754.
- Cai, Z., Choi, J., & Liu, M. (2024). ReLU neural network approximation to piecewise constant functions. *arXiv Preprint arXiv:2410.16506*. Retrieved from <https://arxiv.org/abs/2410.16506>

org/abs/2410.16506

- Casella, G., Robert, C. P., & Wells, M. T. (2004). Generalized Accept-Reject Sampling Schemes. *Institute of Mathematical Statistics Lecture Notes-Monograph Series*, 342–347. Retrieved from <http://www.jstor.com/stable/4356322>
- Cheng, Y. (2021). Derivation of the Backpropagation Algorithm Based on Derivative Amplification Coefficients. *arXiv Preprint arXiv:2102.04320*. Retrieved from <https://arxiv.org/abs/2102.04320>
- Choi, S., Mittal, S., Elvira, V., Park, J., & Malkin, N. (2025). Reinforced sequential Monte Carlo for amortised sampling. *arXiv Preprint arXiv:2510.11711*. Retrieved from <https://arxiv.org/abs/2510.11711>
- Cox, A. M. G. (2020). *Probability Background Notes*. University of Bath. Retrieved from <https://www.maths.bath.ac.uk/~mapamgc/ProbBackgroundNotes.pdf>
- Csörgő, S. (1981). Multivariate Empirical Characteristic Functions. *Zeitschrift für Wahrscheinlichkeitstheorie Und Verwandte Gebiete*, 55(2), 203–229. <https://doi.org/10.1007/BF00535160>
- Cuingnet, R. (2023). On the Computation of the Logarithm of the Modified Bessel Function of the Second Kind. *arXiv Preprint arXiv:2308.11964*. Retrieved from <https://arxiv.org/abs/2308.11964>
- Cunningham, E., Cobb, A. D., & Jha, S. (2022). Principal Component Flows. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, & S. Sabato (Eds.), *Proceedings of the 39th international conference on machine learning* (pp. 4492–4519). PMLR. Retrieved from <https://proceedings.mlr.press/v162/cunningham22a.html>
- De Domenico, F., Livan, G., Montagna, G., & Nicosini, O. (2023). Modeling and simulation of financial returns under non-Gaussian distributions. *Physica A: Statistical Mechanics and Its Applications*, 622, 128886.
- Devroye, L. (2006). Nonuniform Random Variate Generation. *Handbooks in Operations Research and Management Science*, 13, 83–121. [https://doi.org/10.1016/S0927-0507\(06\)13004-2](https://doi.org/10.1016/S0927-0507(06)13004-2)
- Dodge, J., Ilharco, G., Schwartz, R., Farhadi, A., Hajishirzi, H., & Smith, N. (2020). Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping. *arXiv Preprint arXiv:2002.06305*. Retrieved from <https://arxiv.org/abs/2002.06305>
- Dolgov, S., Anaya-Izquierdo, K., Fox, C., & Scheichl, R. (2020). Approximation and sampling of multivariate probability distributions in the tensor train decomposition. *Statistics and Computing*, 30(3), 603–625. <https://doi.org/10.1007/s11222-019-09910-z>

- Du, R., & Dang, D.-M. (2025). Fourier neural network approximation of transition densities in finance. *SIAM Journal on Scientific Computing*, 47(2), C529–C557. <https://doi.org/10.1137/23M1598908>
- Dymetman, M., Bouchard, G., & Carter, S. (2012). The OS* Algorithm: a Joint Approach to Exact Optimization and Sampling. Retrieved from <https://arxiv.org/abs/1207.0742>
- Eling, M. (2012). Fitting insurance claims to skewed distributions: Are the skew-normal and skew-student good models? *Insurance: Mathematics and Economics*, 51(2), 239–248. <https://doi.org/10.1016/j.insmatheco.2012.04.001>
- Evenbly, G. (2022). A Practical Guide to the Numerical Implementation of Tensor Networks I: Contractions, Decompositions, and Gauge Freedom. *Frontiers in Applied Mathematics and Statistics*, 8, 806549. <https://doi.org/10.3389/fams.2022.806549>
- Fernandez, A., & Mali, A. (2025). TeLU Activation Function for Fast and Stable Deep Learning. Retrieved from <https://arxiv.org/abs/2412.20269>
- Ferré, L., & Whittaker, J. (2004). Application of the empirical characteristic function to compare and estimate densities by pooling information. *Computational Statistics*, 19(2), 169–192. <https://doi.org/10.1007/BF02892055>
- Fok, R., An, A., & Wang, X. (2017). Spontaneous Symmetry Breaking in Neural Networks. *arXiv Preprint arXiv:1710.06096*. Retrieved from <https://arxiv.org/abs/1710.06096>
- FPW. (1923). A Treatise on the Theory of Bessel Functions. *Science Progress in the Twentieth Century (1919-1933)*, 18(70), 304–306. Retrieved from <http://www.jstor.org/stable/43432551>
- Gabbay, F., Lev Aharoni, R., & Schweitzer, O. (2022). Deep Neural Network Memory Performance and Throughput Modeling and Simulation Framework. *Mathematics*, 10(21). <https://doi.org/10.3390/math10214144>
- Gao, T., & Jojic, V. (2016). Degrees of Freedom in Deep Neural Networks. *arXiv Preprint arXiv:1603.09260*. Retrieved from <https://arxiv.org/abs/1603.09260>
- Gao, X., Lu, L., & Liu, Q. (2024). Non-negative Tucker decomposition with double constraints for multiway dimensionality reduction. *AIMS Mathematics*, 9(8), 21755–21785. <https://doi.org/10.3934/math.20241058>
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 9, 249–256. JMLR Workshop; Conference Proceedings; PMLR. Retrieved from <https://proceedings.mlr.press/v9/glorot10a.html>
- Hanin, B. (2018). Which Neural Net Architectures Give Rise To Exploding and

- Vanishing Gradients? *Advances in Neural Information Processing Systems*, 31. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2018/file/13f9896df61279c928f19721878fac41-Paper.pdf
- Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson: Prentice Hall. Retrieved from https://www.mlsptlab-unicampania.it/images/palmieri/Signal_Processing_and_Data_Fusion/Selected_pages_CH1-4_Haykin_NeuralNetworks_3rdEd.pdf
- He, Z., & Wang, X. (2021). An Integrated Quasi-Monte Carlo Method for Handling High Dimensional Problems with Discontinuities in Financial Engineering. *Computational Economics*, 57(2), 693–718. Retrieved from <https://doi.org/10.1007/s10614-020-09976-2>
- Heckert, N., & Filliben, J. (2003). *NIST/SEMATECH e-Handbook of Statistical Methods; Chapter 1: Exploratory Data Analysis*. NIST/SEMATECH e-Handbook of Statistical Methods; Chapter 1: Exploratory Data Analysis (Accessed November 3, 2025). Retrieved from <https://www.nist.gov/publications/nistsematech-e-handbook-statistical-methods-chapter-1-exploratory-data-analysis>
- Hoeffler, T., Alistarh, D., Ben-Nun, T., Dryden, N., & Peste, A. (2021). Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 1–124. Retrieved from <http://jmlr.org/papers/v22/21-0366.html>
- Hörmann, W., & Leydold, J. (2003). Continuous Random Variate Generation by Fast Numerical Inversion. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 13(4), 347–362. <https://doi.org/10.1145/945511.945517>
- Hu, T., Chen, F., Wang, H., Li, J., Wang, W., Sun, J., & Li, Z. (2023). Complexity Matters: Rethinking the Latent Space for Generative Modeling. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in Neural Information Processing Systems* (Vol. 36, pp. 29558–29579). Retrieved from https://proceedings.neurips.cc/paper_files/paper/2023/file/5e8023f07625374c6fdf3aa08bb38e0e-Paper-Conference.pdf
- Hu, T., Chen, Z., Sun, H., Bai, J., Ye, M., & Cheng, G. (2018). Stein Neural Sampler. *arXiv Preprint arXiv:1810.03545*. Retrieved from <https://arxiv.org/abs/1810.03545>
- Huk, M. (2012). Backpropagation generalized delta rule for the selective attention Sigma-if artificial neural network. *International Journal of Applied Mathematics and Computer Science*, 22(2), 449–459. <https://doi.org/10.2478/v10006-012-0034-5>
- Hult, H., & Svensson, J. (2009). Efficient calculation of risk measures by importance sampling—the heavy tailed case. *arXiv Preprint arXiv:0909.3335*. Retrieved from <https://arxiv.org/abs/0909.3335>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical*

- learning: with applications in R* (Vol. 103). Springer.
- Joe, H., & Li, X. (2024). Likelihood Inference for Factor Copula Models with Asymmetric Tail Dependence. *Entropy*, 26(7), 610. <https://doi.org/10.3390/e26070610>
- Jordão, T. (2020). Decay of fourier transforms and generalized besov spaces. *Constructive Mathematical Analysis*, 3(1), 20–35. <https://doi.org/10.33205/cma.646557>
- Karner, C., Kazeev, V., & Petersen, P. C. (2024). Limitations of neural network training due to numerical instability of backpropagation. *Advances in Computational Mathematics*, 50(1), 14. <https://doi.org/10.1007/s10444-024-10106-x>
- Kleiber, C. (2014). The generalized lognormal distribution and the Stieltjes moment problem. *Journal of Theoretical Probability*, 27(4), 1167–1177.
- Kosiorek, A. (2024). *Normalizing Flows*. Online tutorial. Retrieved from https://akosiorek.github.io/norm_flows/
- Kuninti, S., & Rooban, S. (2021). Backpropagation algorithm and its hardware implementations: A review. *Journal of Physics: Conference Series*, 1804, 012169. IOP Publishing.
- Lamonaca, F. (2025). Introductory notes for the Acta IMEKO third issue in 2025. *Acta IMEKO*, 14(3), 1–3. <https://doi.org/10.21014/actaimeko.v14i3.2215>
- Le, Q. V., Jaitly, N., & Hinton, G. E. (2015). A simple way to initialize recurrent networks of rectified linear units. *arXiv Preprint arXiv:1504.00941*. Retrieved from <https://arxiv.org/abs/1504.00941>
- Lee, H. Y., Park, H. J., & Kim, H. M. (2014). A clarification of the Cauchy distribution. *Communications for Statistical Applications and Methods*, 21(2), 183–191.
- Lee, H., Kim, Y., Yang, S. Y., & Choi, H. (2024). Improved weight initialization for deep and narrow feedforward neural network. *Neural Networks*, 176, 106362. <https://doi.org/10.1016/j.neunet.2024.106362>
- Lee, J. D., Simchowitz, M., Jordan, M. I., & Recht, B. (2016). Gradient Descent Converges to Minimizers. Retrieved from <https://arxiv.org/abs/1602.04915>
- Li, T. B. B., & Bickel, P. (2005). Curse-of-dimensionality revisited: Collapse of importance sampling in very high-dimensional systems. *University of California, Berkeley, Department of Statistics, Tech. Rep.*, 696.
- Lu, L., Shin, Y., Su, Y., & Karniadakis, G. E. (2020). Dying ReLU and Initialization: Theory and Numerical Examples. <https://doi.org/10.4208/cicp.OA-2020-0165>
- Magnus, J. R., & Neudecker, H. (1999). *Matrix Differential Calculus with Applications in Statistics and Econometrics* (2nd ed.). John Wiley & Sons.

- Marsaglia, G., & Tsang, W. W. (2000). A simple method for generating gamma variables. *ACM Transactions on Mathematical Software (TOMS)*, 26(3), 363–372. <https://doi.org/10.1145/358407.358414>
- McLachlan, G. J., Lee, S. X., & Rathnayake, S. I. (2019). Finite mixture models. *Annual Review of Statistics and Its Application*, 6(1), 355–378.
- Nascimento, D. C. do, Ramos, P. L., Elal-Olivero, D., Cortes-Araya, M., & Louzada, F. (2021). Generalizing Normality: Different Estimation Methods for Skewed Information. *Symmetry*, 13(6). <https://doi.org/10.3390/sym13061067>
- Nguyen, D.-T. (2025). *The Little Book of Matrix Calculus*. Retrieved from <https://little-book-of.github.io/matrix-calculus/>
- Nowozin, S., Cseke, B., & Tomioka, R. (2016). f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization. *Advances in Neural Information Processing Systems*, 29. Retrieved from <https://arxiv.org/abs/1606.00709>
- Odeyar, P., Apel, D. B., Hall, R., Zon, B., & Skrzypkowski, K. (2022). A Review of Reliability and Fault Analysis Methods for Heavy Equipment and Their Components Used in Mining. *Energies*, 15(17). <https://doi.org/10.3390/en15176263>
- Olshausen, B. A. (2000). *Aliasing*. University of California. Retrieved from <https://redwood.berkeley.edu/bruno/npb261/aliasing.pdf>
- Pacheco-González, C. G. (2009). Approximation for the normal inverse Gaussian process using random sums. *Stochastic Analysis and Applications*, 27(6), 1191–1200.
- Perekrestenko, D., Eberhard, L., & Bölcskei, H. (2021). High-Dimensional Distribution Generation Through Deep Neural Networks. *Partial Differential Equations and Applications*, 2(5), 64. <https://doi.org/10.1007/s42985-021-00115-6>
- Plesner, A., Sørensen, H. H. B., & Hauberg, S. (2024). Accurate Computation of the Logarithm of Modified Bessel Functions on GPUs. *Proceedings of the 38th ACM International Conference on Supercomputing*, 213–224.
- Porkodi, S. P., Sarada, V., Maik, V., & Gurushankar, K. (2022). Generic image application using GANs (Generative Adversarial Networks): A Review. *Evolving Systems*, 1–15. <https://doi.org/10.1007/s12530-022-09464-y>
- R Core Team. (2025). *Exponential distribution*. Vienna, Austria: R Foundation for Statistical Computing. Retrieved from <https://stat.ethz.ch/R-manual/R-devel/library/stats/html/Exponential.html>
- Rangwani, H., Jaswani, N., Karmali, T., Jampani, V., & Babu, R. V. (2022). Improving GANs for Long-Tailed Data through Group Spectral Regularization. Retrieved from <https://arxiv.org/abs/2208.09932>

- Roberts, G. O., & Rosenthal, J. S. (1998). Markov-Chain Monte Carlo: Some Practical Implications of Theoretical Results. *The Canadian Journal of Statistics/La Revue Canadienne de Statistique*, 26(1), 5–20. Retrieved from <http://www.jstor.org/stable/3315667>
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv Preprint arXiv:1609.04747*. Retrieved from <https://arxiv.org/abs/1609.04747>
- Rydberg, T. H. (1997). The normal inverse Gaussian Lévy process: simulation and approximation. *Communications in Statistics. Stochastic Models*, 13(4), 887–910.
- Salman, S., & Liu, X. (2019). Overfitting Mechanism and Avoidance in Deep Neural Networks. *arXiv Preprint arXiv:1901.06566*. Retrieved from <https://arxiv.org/abs/1901.06566>
- Scherer, M., Rachev, S. T., Kim, Y. S., & Fabozzi, F. J. (2012). Approximation of skewed and leptokurtic return distributions. *Applied Financial Economics*, 22(16), 1305–1316.
- Sharma, P., Kumar, M., Sharma, H. K., & Biju, S. M. (2024). Generative adversarial networks (GANs): Introduction, taxonomy, variants, limitations, and applications. *Multimedia Tools and Applications*, 83(41), 88811–88858. <https://doi.org/10.1007/s11042-024-18767-y>
- Sitzmann, V., Martel, J., Bergman, A., Lindell, D., & Wetzstein, G. (2020). Implicit Neural Representations with Periodic Activation Functions. *Advances in Neural Information Processing Systems*, 33, 7462–7473. Retrieved from <https://arxiv.org/abs/2006.09661>
- Soch, J., Saritaş, K., Maja, Monticone, P., Faulkenberry, Thomas J., Martin, Osvaldo A., ... (2024). *StatProofBook/StatProofBook.github.io: The Book of Statistical Proofs (Version 2023)*. Zenodo. <https://doi.org/10.5281/zenodo.4305949>
- Sonoda, S., & Murata, N. (2017). Neural network with unbounded activation functions is universal approximator. *Applied and Computational Harmonic Analysis*, 43(2), 233–268. <https://doi.org/10.1016/j.acha.2015.12.005>
- Steinwart, I. (2019). A Sober Look at Neural Network Initializations. Retrieved from <https://arxiv.org/abs/1903.11482>
- Tanizaki, H. (2004). *Computational Methods in Statistics and Econometrics*. <https://doi.org/10.1201/9780203022023>
- Terada, Y., & Hirose, R. (2020). Fast generalization error bound of deep learning without scale invariance of activation functions. *Neural Networks*, 129, 344–358. <https://doi.org/10.1016/j.neunet.2020.05.033>
- Wilks, D. S. (2011). *Statistical methods in the atmospheric sciences* (Vol. 100). Academic

press.

- You, K., Long, M., Wang, J., & Jordan, M. I. (2019). How Does Learning Rate Decay Help Modern Neural Networks? *arXiv Preprint arXiv:1908.01878*. Retrieved from <https://arxiv.org/abs/1908.01878>
- Yu, J. (2004). Empirical characteristic function estimation and its applications. *Econometric Reviews*, 23(2), 93–123.
- Zammit-Mangion, A., Sainsbury-Dale, M., & Huser, R. (2025). Neural Methods for Amortized Inference. *Annual Review of Statistics and Its Application*, 12, 311–335. <https://doi.org/10.1146/annurev-statistics-112723-034123>
- Zeevi, A. J., & Meir, R. (1997). Density estimation through convex combinations of densities: approximation and estimation bounds. *Neural Networks*, 10(1), 99–109.



RHODES UNIVERSITY
Where leaders learn

Department of Statistics

RESEARCH ETHICS DECLARATION

This statement is to be included in the appendices of research papers/dissertations/theses submitted for postgraduate examination where research did **not** involve interaction with human participants, or the use of animal subjects, and therefore did not require research ethics approval. Candidates whose research did require ethics clearance must include their ethics approval letter in the appendix of their examination submission.

Name of Candidate: Nicholas Loyd Szczawinski

Name of Supervisor(s): Jeremy Baxter

Degree: Bachelor of Science Honours in Mathematical Statistics

Title of Research: Building a Feed-forward Characteristic Function Matching Generator

DECLARATION

I declare that my research did not require ethical clearance because (tick all that apply):

I used previously collected data that had already received ethics clearance.	
I analysed documents/open-access digital texts that are freely available in the public domain.	
I did a literature review/analysis of theoretical or secondary material only.	
I used human datasets of non-sensitive information that are either anonymous (identifiers were never collected) or have been deidentified (identifiers have been completely removed).	
I used commercially produced human biological material (e.g. established human cell lines).	
I observed people in public spaces and natural environments where they had no reasonable expectation of privacy and I did not interact with them or intervene in any way.	
I used non-living animal materials (e.g. bones of already deceased organisms or fossils) while complying with any custody and/or jurisdiction requirements.	
I did a content analysis of public media (newspapers, advertisements, and social media posts).	
I did a simulation study with no real-world consequences and does not involve disturbing or distressing content.	X
I observed flora, fauna, and ecosystems without interfering with or disturbing their natural state while complying with any jurisdiction requirements.	
Other (Please provide details):	

Signature of Candidate:

Date: 05/11/2025

Signature of Supervisor:

Date: 07/11/2025